



iOS SDK 1.12.0.0 for Xcode User Guide



This document is intended only for authorized individuals of AppSealing customer.
Unauthorized distribution of any parts of this document to other parties is prohibited.

The software referenced herein, this User Guide, and any associated documentation is provided to you pursuant to the agreement between your company, governmental body or other entity (“you”) and Doverrunner Corporation (“AppSealing”) under which you have received a copy of AppSealing Licensed Technology and this User Guide (such agreement, the “Agreement”). Defined terms not defined herein shall have the meanings ascribed to them in the Agreement. In the event of conflict between the terms of this User Guide and the terms of the Agreement, the terms of the Agreement shall prevail. Without limiting the generality of the remainder of this paragraph, (a) this User Guide is provided to you for informational purposes only, (b) your right to access, view, use, and copy this User Guide is limited to the rights and subject to the applicable requirements and limitations set forth in the Agreement, and (c) all of the content of this User Guide constitutes “Confidential Information” of AppSealing (or the equivalent term used in the Agreement) and is subject to all of the limitations and requirements pertinent to the use, disclosure and safeguarding of such information. Permitting anyone who is not directly involved in the authorized use of AppSealing Licensed Technology by your company or other entity to gain any access to this User Guide shall violate the Agreement and subject your company or other entity to liability therefore.

Copyright Information

Copyright © 2000-2026 Doverrunner Corporation. All rights reserved.

AppSealing® is a trademark of Doverrunner Corporation in the South Korea and/or other countries.

macOS™ and Xcode® are trademarks of Apple Inc., registered in the United States and other countries.

All other trademarks are the property of their respective owners.

Disclaimer

The remainder of this User Guide notwithstanding, this User Guide is provided “as is”, without any warranty whatsoever (including that it is error-free or complete). This User Guide contains no express or implied warranties, covenants or grants of rights or licenses, and does not modify or supplement any express warranty, covenant or grant of rights or licenses that is set forth in the Agreement. This User Guide is current as of the date set forth in the header that appears above on this page, but may be modified at any time without prior notice. Future revisions and updates of this User Guide shall be distributed as part of AppSealing SDK new releases. Doverrunner shall under no circumstances bear any responsibility for your failure to operate AppSealing Licensed Technology in compliance with the then-current version of this User Guide. Your remedies with respect to your use of this User Guide, and Doverrunner' liability for your use of this User Guide (including for any errors or inaccuracies that appear in this User Guide) are limited to those remedies expressly authorized by the Agreement (if any).

Contact Information

Address: [06109] 1F 608, Nonhyeon-ro, Gangnam-gu, Seoul, South Korea

Technical support: <https://helpcenter.appsealing.com>

Website: <https://doverrunner.com>

Table of Contents

요구 사항	4
 Part 1. AppSealing SDK의 위치	
1-1 AppSealing SDK 프로젝트 폴더에 복사	5
1-2 AppSealing SDK 압축 해제	6
1-3 프로젝트 Bundle ID와 SDK에 등록된 Bundle ID 비교	7
 Part 2. 프로젝트에 AppSealing 파일 추가	
2-1 Xcode Project 열기	8
2-2 Frameworks, Libraries and Embedded Content 조회	8
2-3 보안 프레임워크 추가	9
2-4 브리징 코드 추가	11
2-4 디버깅을 위해 Build Phase Run Script 추가	12
2-5 앱 빌드 모드에 따른 주의사항	15
 Part 3. 보안 위협 안내를 위한 GUI 추가	
3-1 앱에서 UIAlertController 출력	16
 Part 4. 프로젝트 "Build Settings" 수정	
4-1 프로젝트 메인 target 선택	20
4-2 Build Settings "User Script Sandboxing" 비활성화	20
4-3 Build Settings "C++ and Objective-C Interoperability" 변경	21
4-4 앱 무결성 및 인증서 검증 정보 생성	22
4-5 메서드 스위칭 / 메서드 후킹 감지 기능 제어	28
4-6 재서명 앱 App Store Connect 업로드	30
 Part 5. AppSealing 기기 고유 ID 획득	
5-1 기기 고유 ID 확인	35
 Part 6. 소스 코드 문자열 암호화 적용	
6-1 프로젝트 Build Phase 설정	37
6-2 문자열 암호화 태그 추가	41
6-3 Swift 문자열 암호화 주의사항	46
6-4 컴파일 오류 대응: 암호화된 문자열 복원	47

Part 7. 문제 해결

7-1 Xcode Build error (1) Cannot find type 'AppSealingInterface' in scope	49
7-2 Execution error (1) 앱이 실행되고 곧바로 종료됩니다	50
7-3 Execution error (2) 앱이 실행 중 갑자기 종료됩니다	52
7-4 무결성/인증서 스냅샷 생성 스크립트가 실행되지 않습니다	53

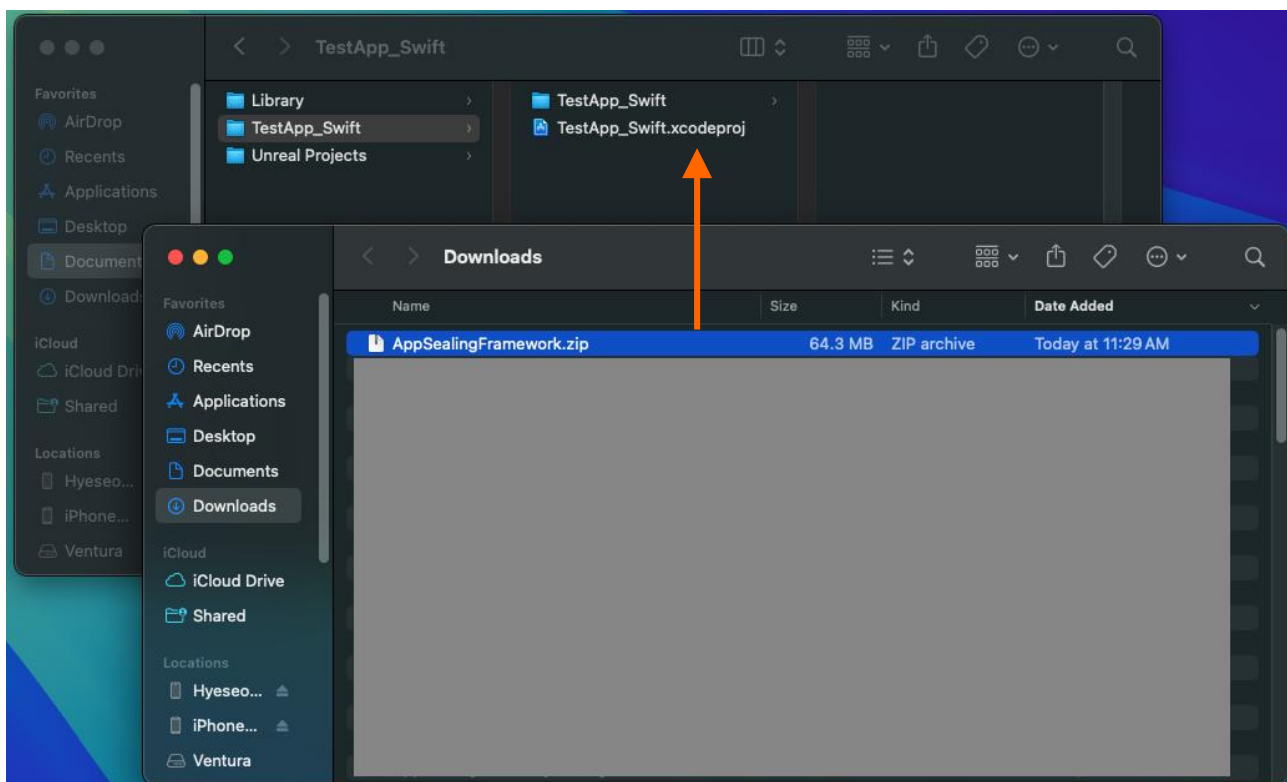
Prerequisite

- Xcode 15.0 이상 (macOS 14 이상)
(본 문서는 Xcode 16 기준으로 작성)
- iOS 15.0 이상 기기

Part 1. AppSealing SDK의 위치

AppSealing SDK zip 파일을 다운로드한 후 Xcode 프로젝트 폴더에 압축을 해제합니다. 이 문서에서 안내되는 샘플 프로젝트 'TestApp_Swift'는 "~/Documents/TestApp_Swift" 경로에 위치합니다.

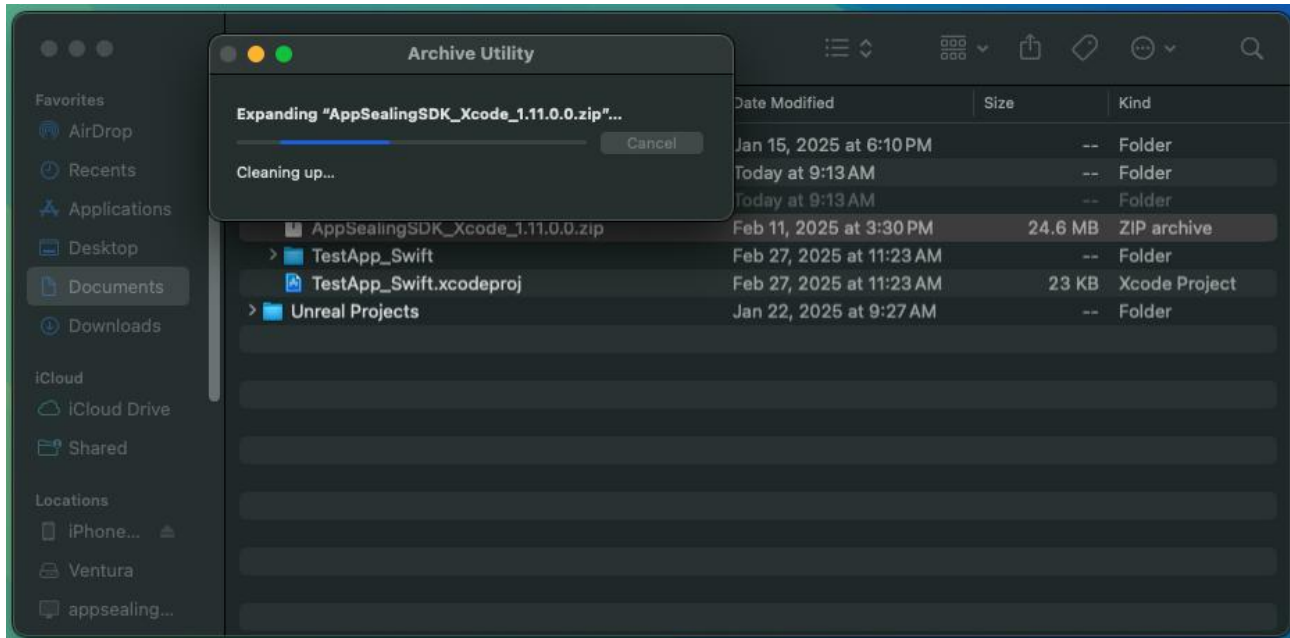
1-1 AppSealing SDK 프로젝트 폴더에 복사



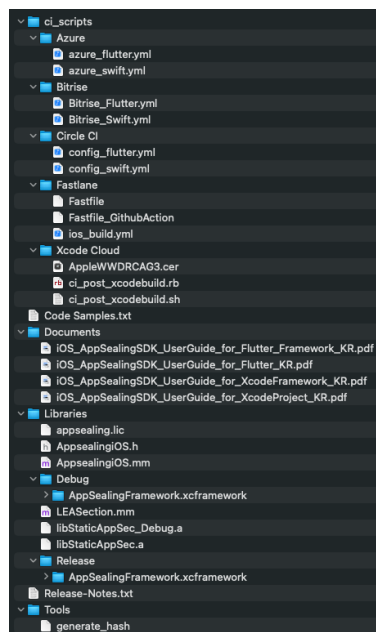
**** 주의:** SDK 압축 파일의 이름이 "AdhocEnabled"를 포함하는 버전은 공식 App Store 혹은 TestFlight 이외의 방식(Ad Hoc, enterprise, MDM 및 기타 third party store)으로 앱을 배포할 수 있도록 일부 보안 기능이 약화된 SDK입니다. App Store을 통하지 않은 배포는 공격에 취약해질 수 있으므로, 반드시 필요한 경우가 아니라면 AdhocEnabled가 아닌 버전으로 다운로드 받으시기 바랍니다.

1-2 AppSealing SDK 압축 해제

복사한 AppSealingSDK.zip 파일을 더블클릭 하여 압축을 해제합니다.



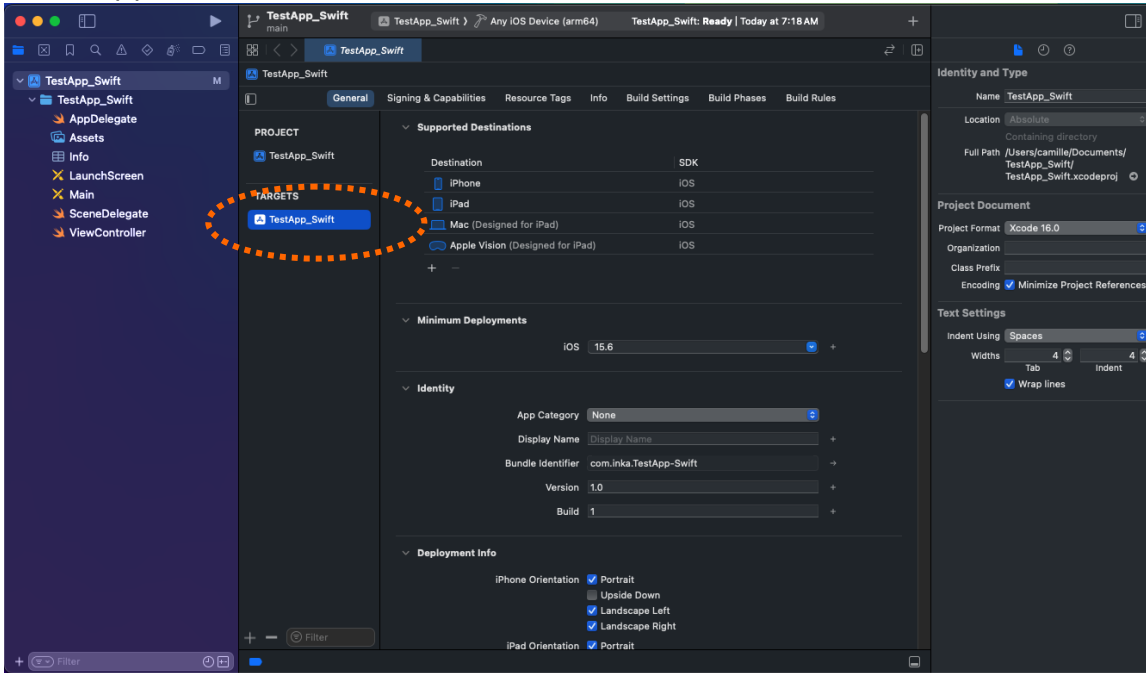
Zip 파일 압축 해제 후 다음과 같은 구조처럼 모든 파일이 생성되는지 확인하십시오.



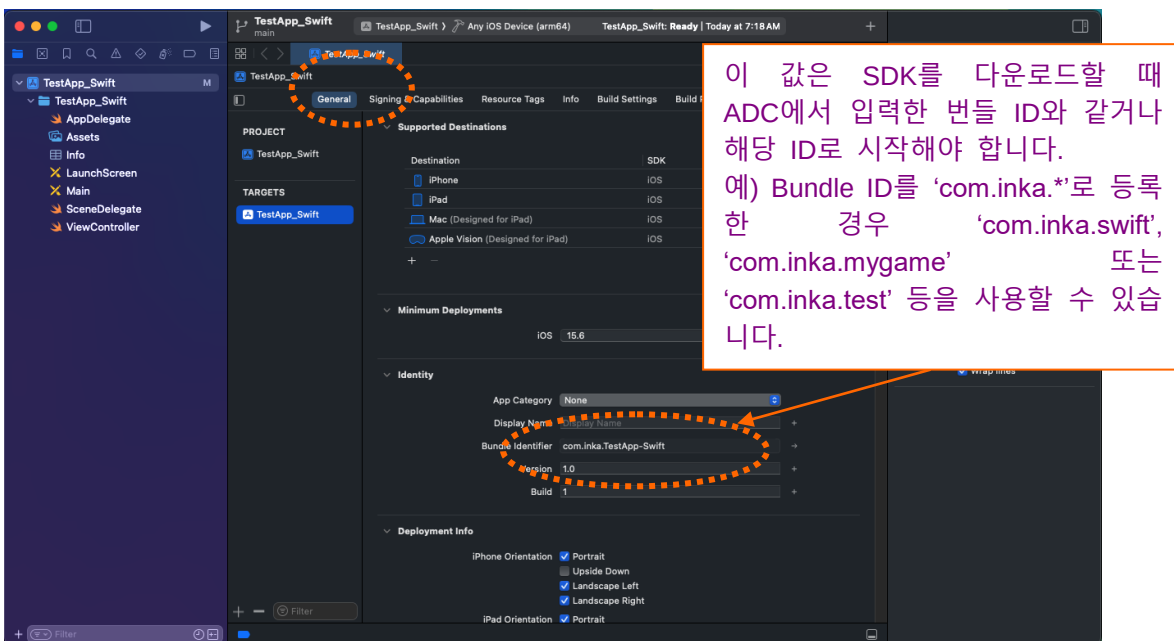
누락된 파일이 있는 경우 SDK를 다시 내려받거나 zip 파일 압축을 다시 해제해 보십시오.

1-3 프로젝트 Bundle ID와 SDK에 등록된 Bundle ID 비교

SDK zip 파일을 압축 해제하고 프로젝트의 bundle ID가 유효한지 확인하십시오. Xcode를 열고 왼쪽 패널에서 프로젝트를 선택하고 Targets 목록에서 프로젝트 이름 (TestApp_Swift)을 선택하십시오.

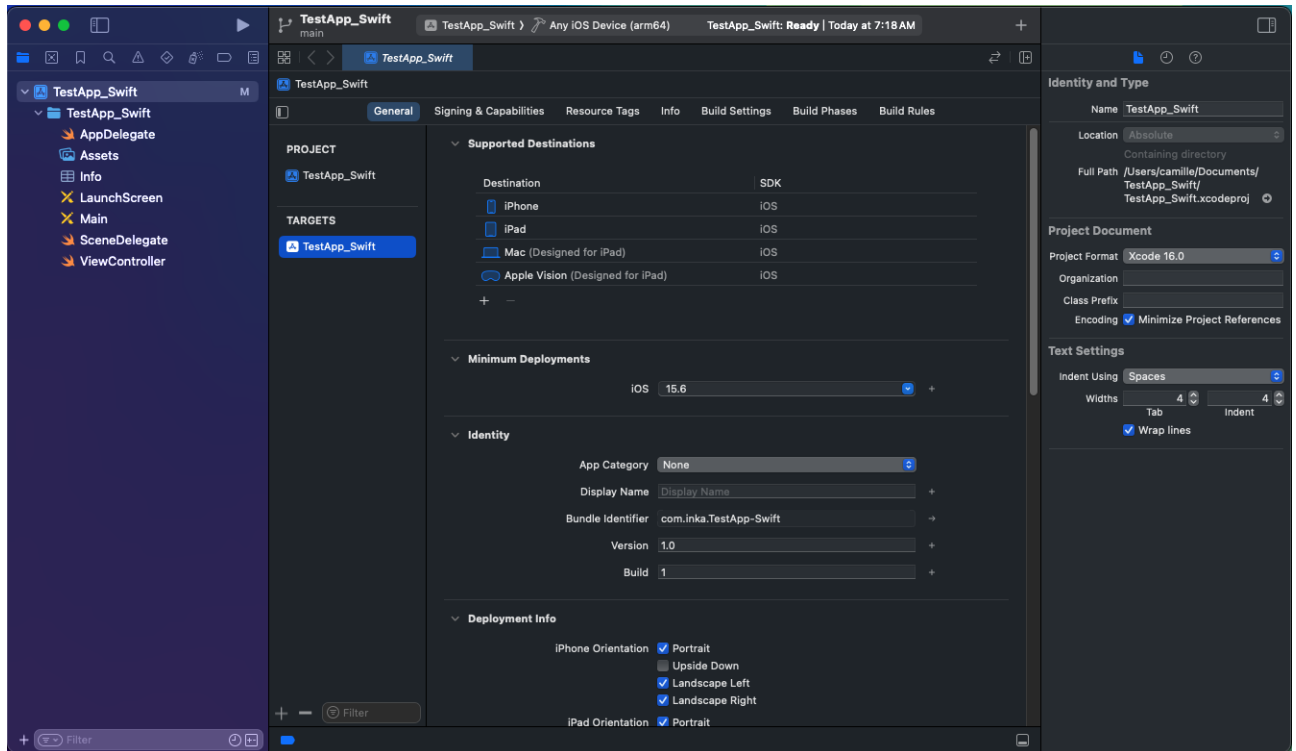


그 다음 "General" 탭을 선택하여 bundle ID가 AppSealing Developer Center (ADC)에서 SDK를 다운로드할 때 입력한 값과 같거나 해당 값으로 시작하는지 확인하십시오.

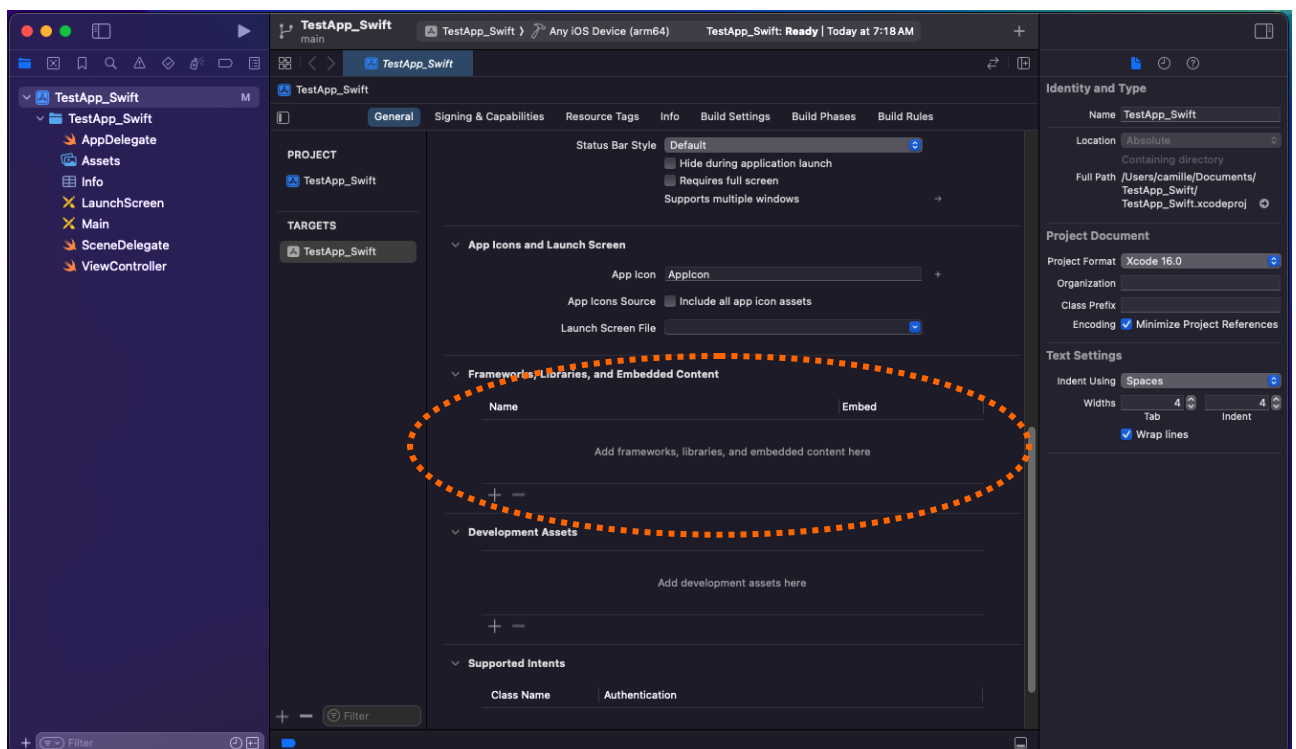


Part 2. 프로젝트에 AppSealing 파일 추가

2-1 Xcode project 열기

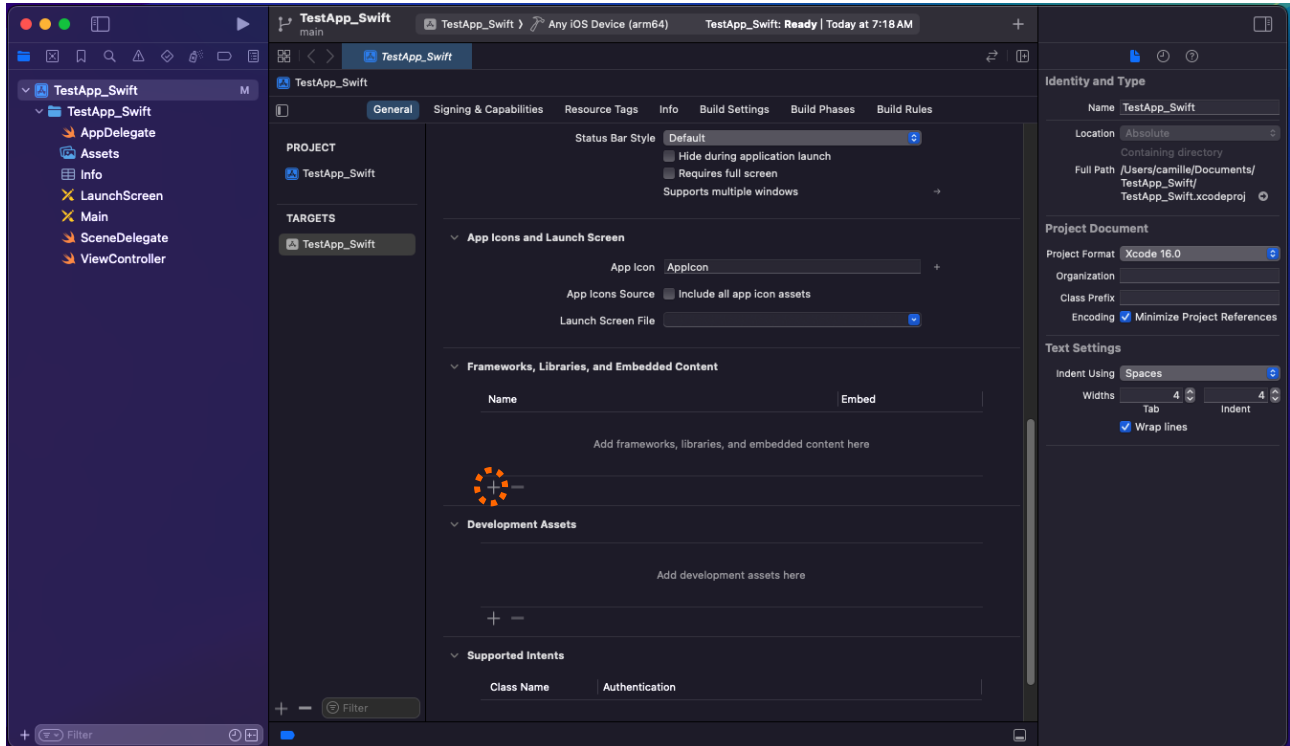


2-2 Frameworks, Libraries and Embedded Content로 스크롤

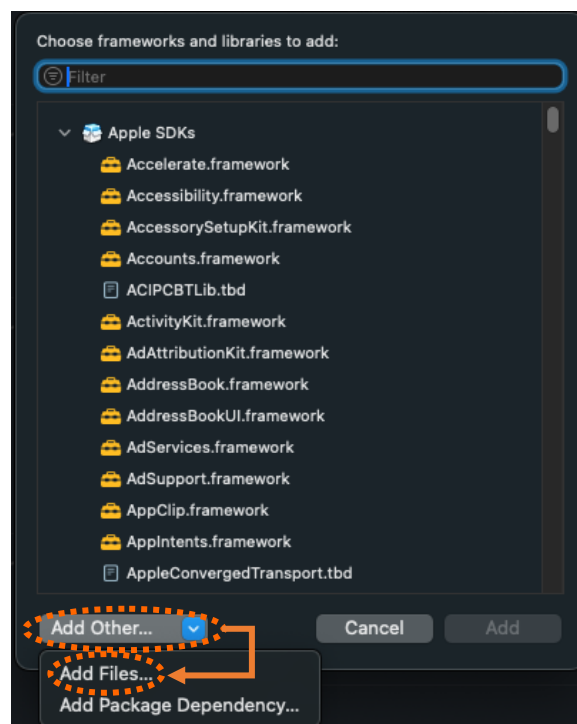


2-3 보안 Framework 추가

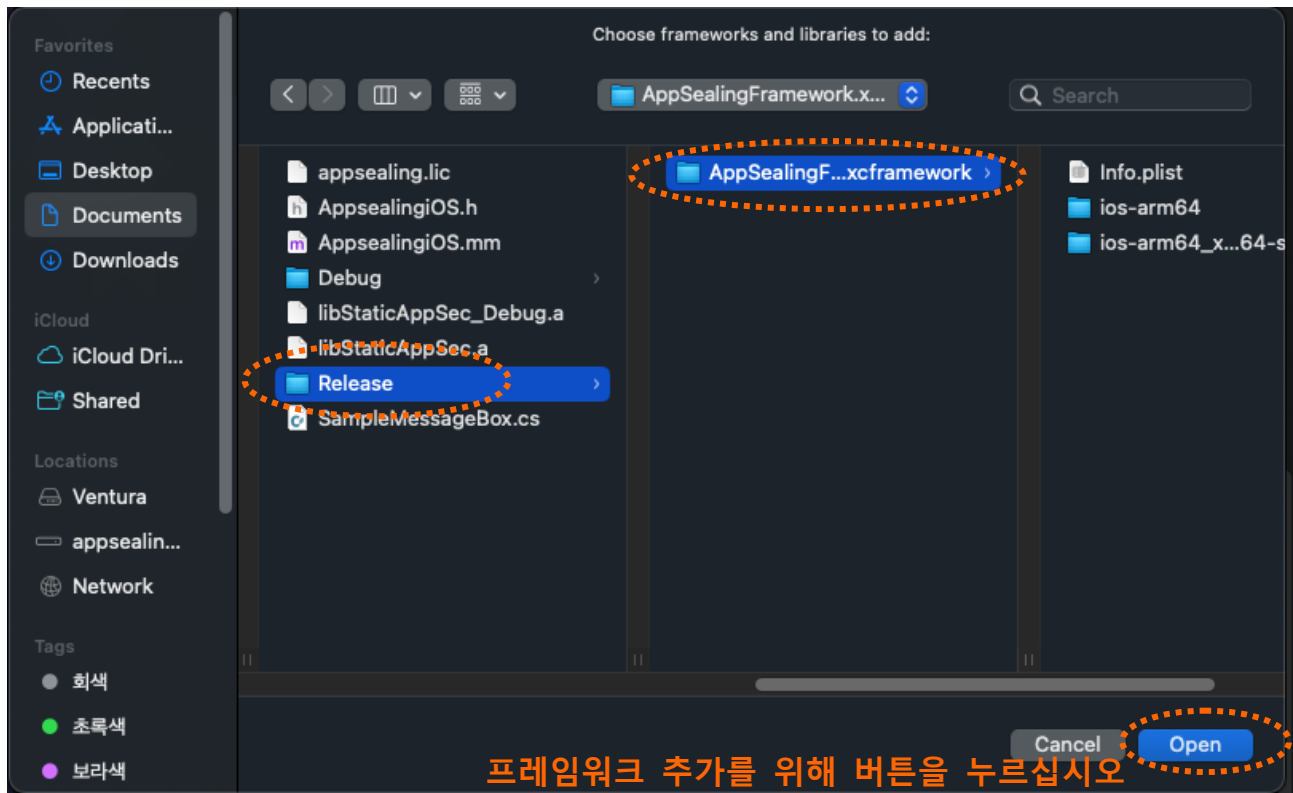
해당 부분의 + 버튼을 눌러 링크할 프레임워크를 선택합니다.



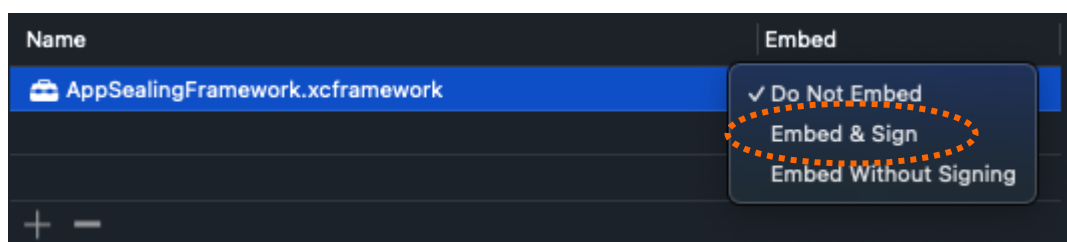
“Add Other” > “Add Files”를 눌러 로컬 Framework를 추가합니다..



대화 상자에서 “AppSealingSDK/Libraries/Release” 폴더의 “AppSealingFramework.xcframework”를 선택하고 “Open” 버튼을 눌러 추가하십시오.

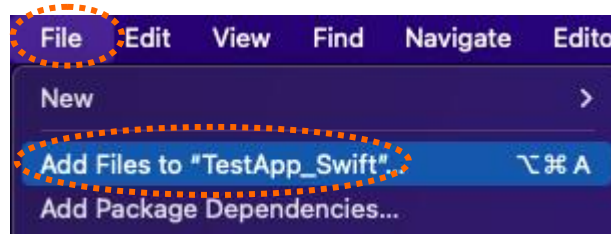


간혹 프레임워크 추가 이후 “Do Not Embed” 상태로 전환될 수 있습니다. 이를 “Embed & Sign”으로 변경하여야만 앱에 라이선스 파일과 Privacy Manifest 파일이 제대로 추가되게 됩니다.

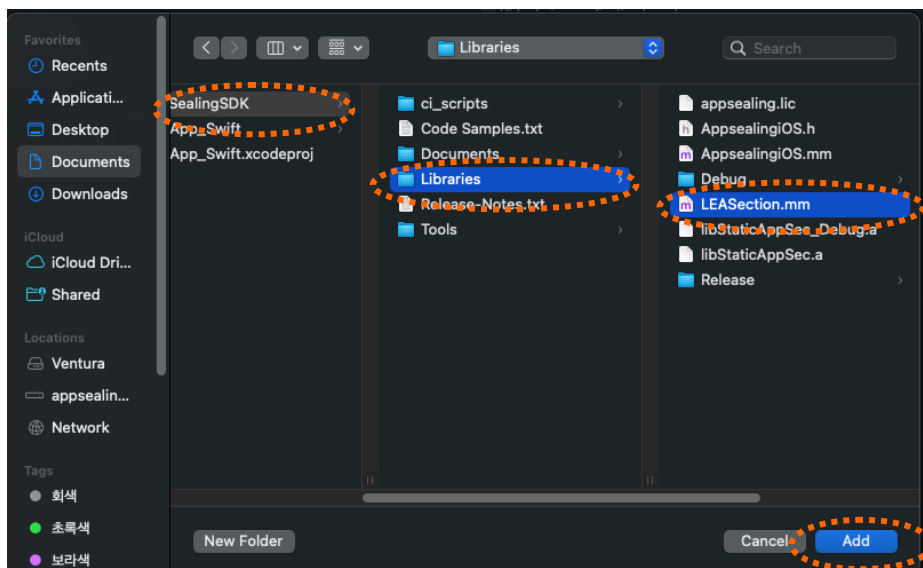


2-4 브리징 코드 추가

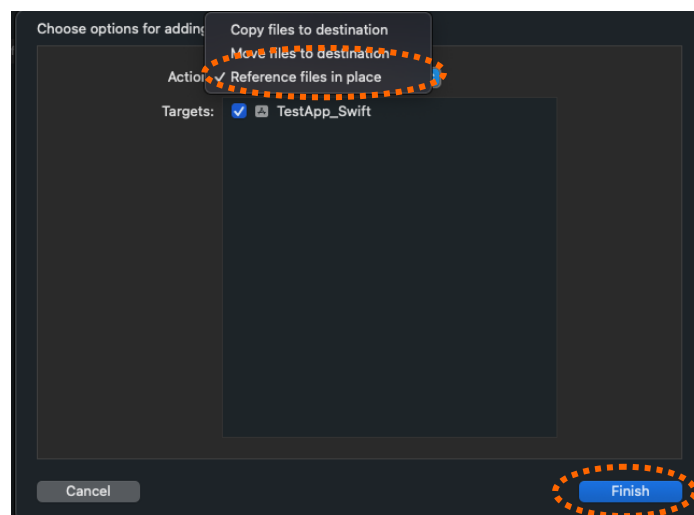
보안 프레임워크를 적용한 앱이 컴파일되기 위해 추가해야 할 브리징 코드가 있습니다. 이를 추가하기 위해 Xcode의 메뉴 바에서 'File > Add Files to "TestApp_Swift"...'를 선택하십시오.



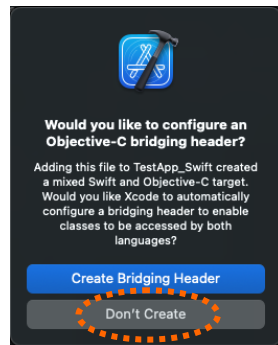
"AppSealingSDK/Libraries/LEASection.mm"을 추가하십시오.



Action으로는 "Reference files in place"를 선택하고, "Finish"를 눌러 LEASection.mm을 추가합니다.



Swift 기반 프로젝트의 경우 추가된 코드가 Objective-C 코드이기에 bridging header를 추가할 지 여부를 결정하는 창이 표시됩니다. 의문이 있을 경우, AppSealing 이외의 코드에서 Objective-C를 사용하지 않는다면 추가할 필요가 없습니다.

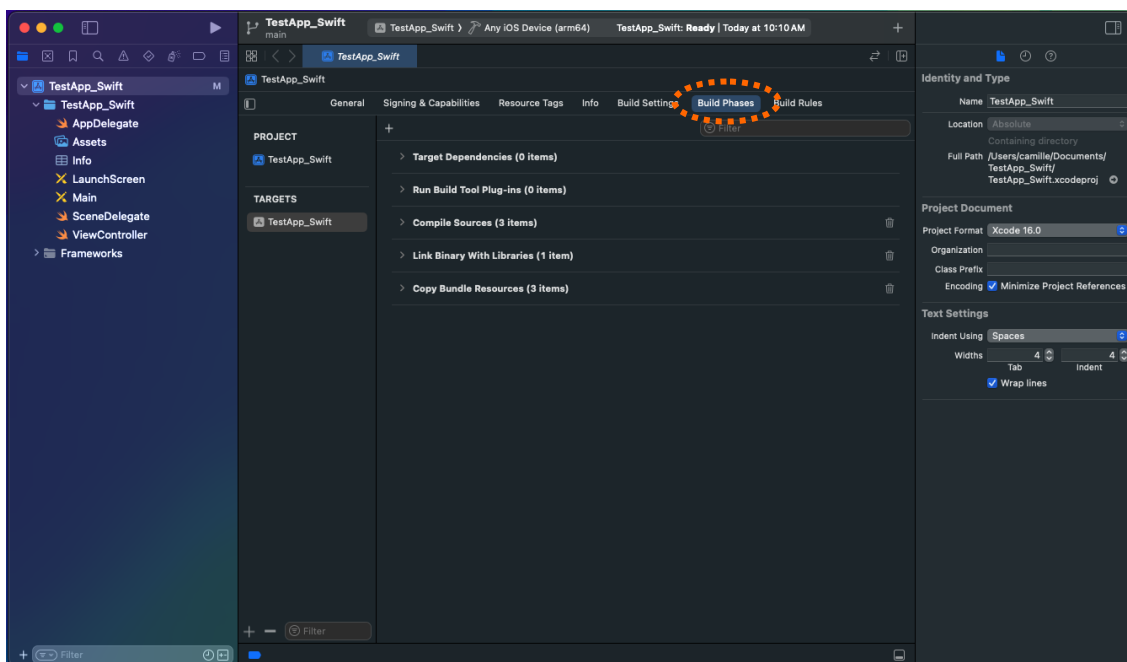


2-5 디버깅을 위해 **Run Script Phase** 추가

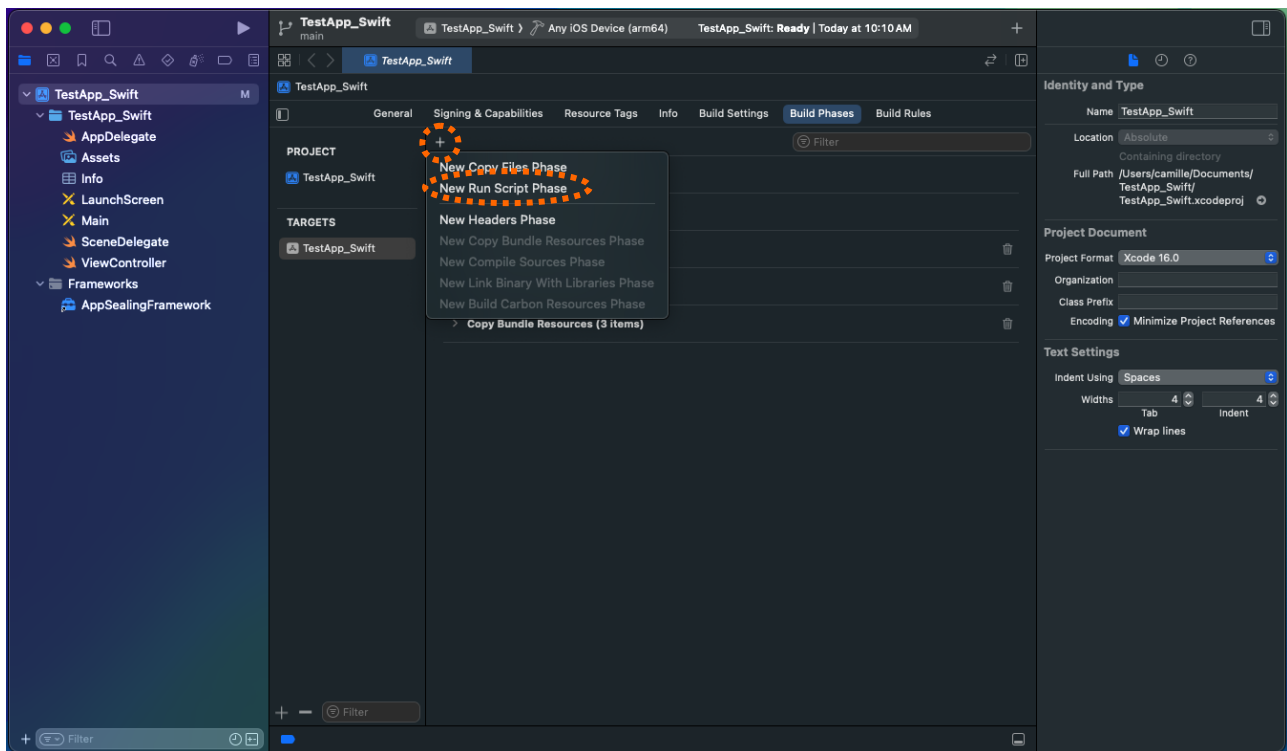
기본적으로 AppSealing 프레임워크는 릴리즈 모드에서 즉석 디버깅 중에 앱을 바로 종료시키게 되는데, 이는 디버그 모드에서는 앱이 제대로 실링되어 있지 않기 때문입니다.

이를 방지하기 위해 디버그 모드용 프레임워크를 제공하고 있습니다만, Xcode 내부적으로는 두 가지 버전의 프레임워크를 자동으로 전환할 방법을 제공하고 있지 않습니다. 일반적으로는 이 전환 작업은 유저에 의해 직접 이루어져야 하지만, 저희는 AppSealing 프레임워크가 디버그 모드일 때 전환되는 자동화된 스크립트를 제공합니다.

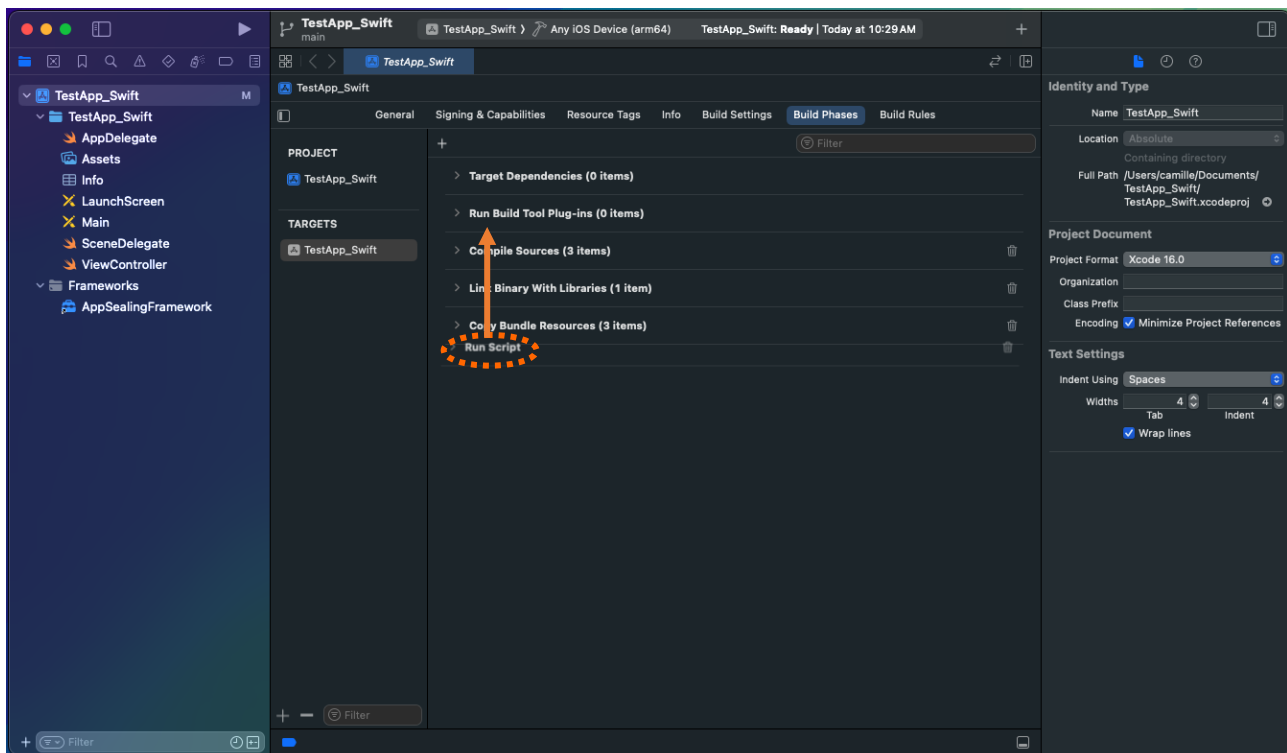
이를 프로젝트에 실 적용하기 위해 먼저 프로젝트 설정에서 "Build Phases"로 들어갑니다.

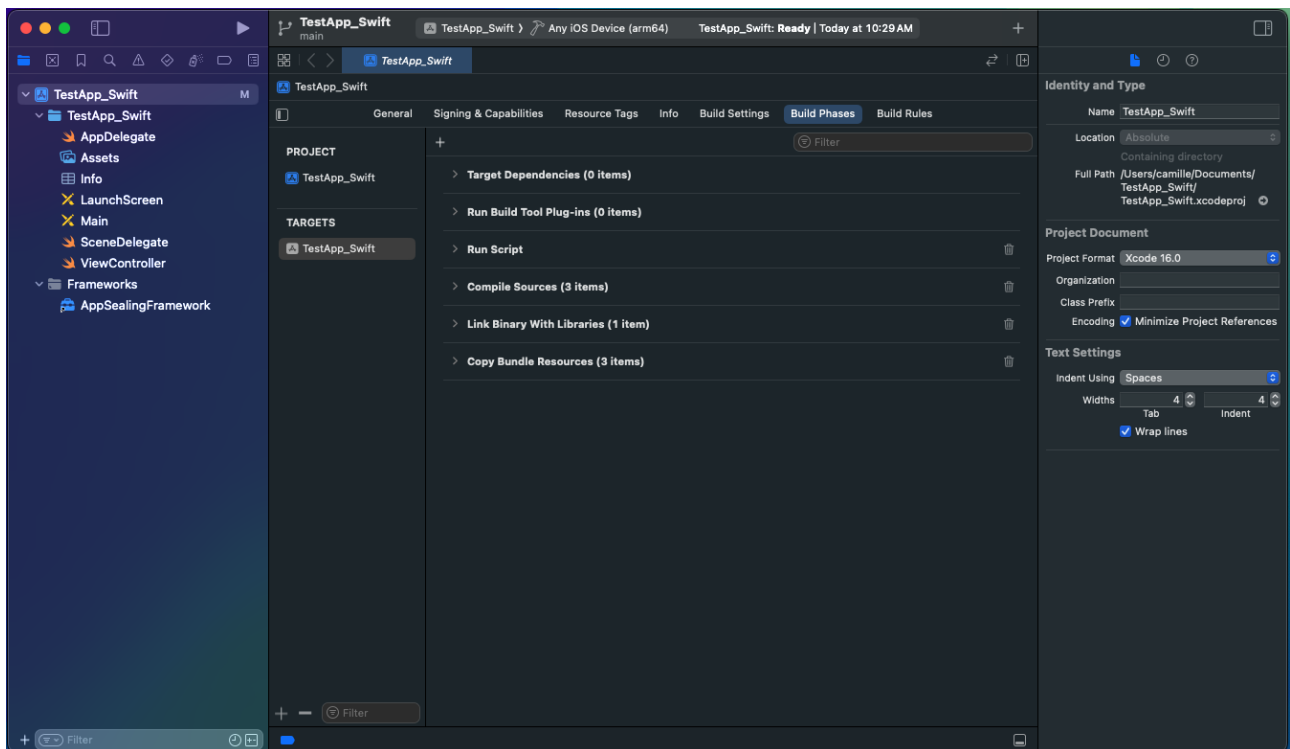


+ 버튼을 클릭하여 “New Run Script Phase” 항목을 눌러 Run Script Phase를 새로 추가합니다.

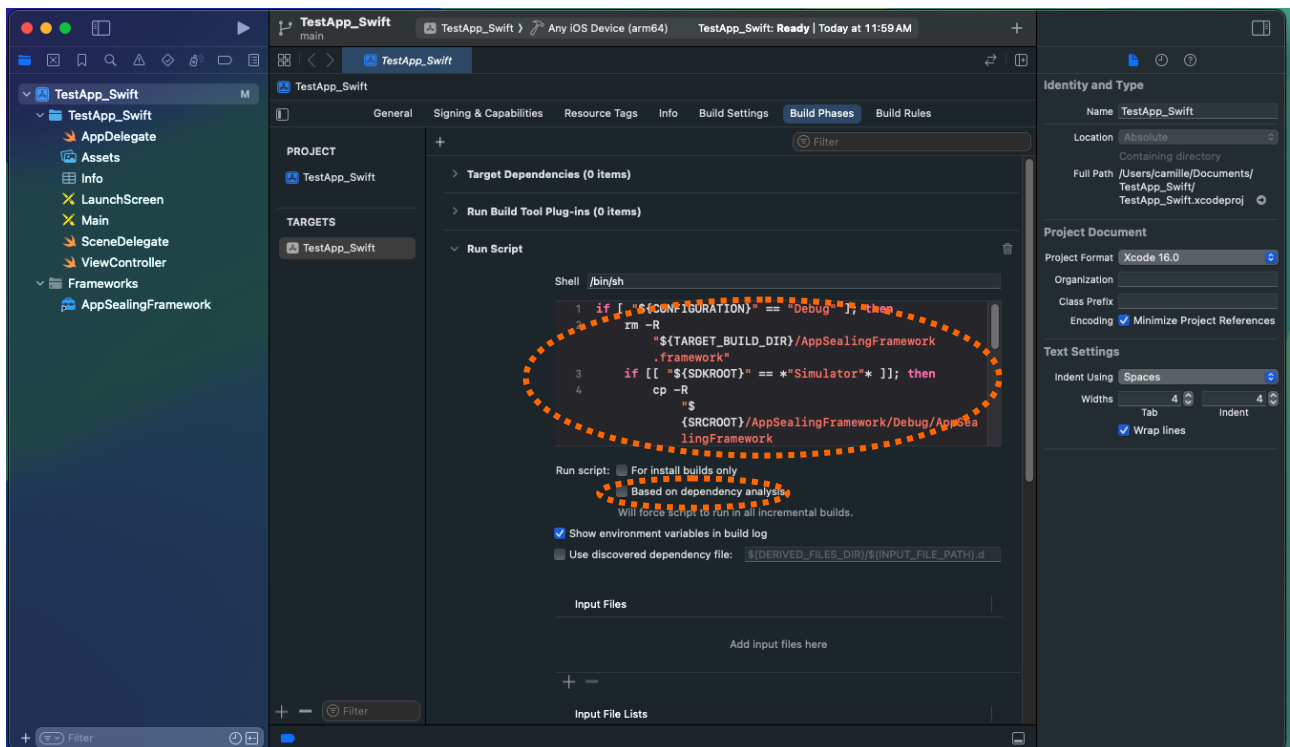


Run Script Phase가 맨 밑에 추가되었습니다. 드래그하여 “Run Build Tool Plug-ins” 아래에 위치하도록 합니다.





“Code Samples.txt” 파일에 제공된 스크립트를 Run Script 페이지에 기입하고, “Based on dependency analysis”의 체크를 해제하여 스크립트가 매번 실행되도록 합니다.



2-6 앱 빌드 모드에 따른 주의사항

* AppSealing은 디버그 모드와 릴리즈 모드에서 다르게 동작합니다.

디버그 모드로 빌드하는 경우 다음의 AppSealing 보안 기능은 원활한 개발을 위해 탐지되었더라도 앱이 종료되지 않습니다:

- 탈옥 감지 : Jailbreak detection
- 안티 디버깅 : Anti-debugging
- 안티 후킹 : Anti-hooking
- 실행 파일 암호화 감지 : Not encrypted executable file detection
- 앱 무결성 감지 : App-Integrity corruption detection
- 앱 재서명 감지 : Re-signing detection

릴리즈 모드로 빌드하면 정상적으로 탐지하고 종료됩니다.

App Store에 배포할 때 앱을 릴리즈 모드로 빌드하십시오. 릴리즈 모드로 AppSealing을 테스트할 경우 App Store 또는 TestFlight로 앱을 배포해야 합니다. 그렇지 않으면 실행 파일이 암호화되지 않은 것이 탐지되거나 앱이 변조된 것으로 탐지되어 일정 시간 후 앱이 종료됩니다.

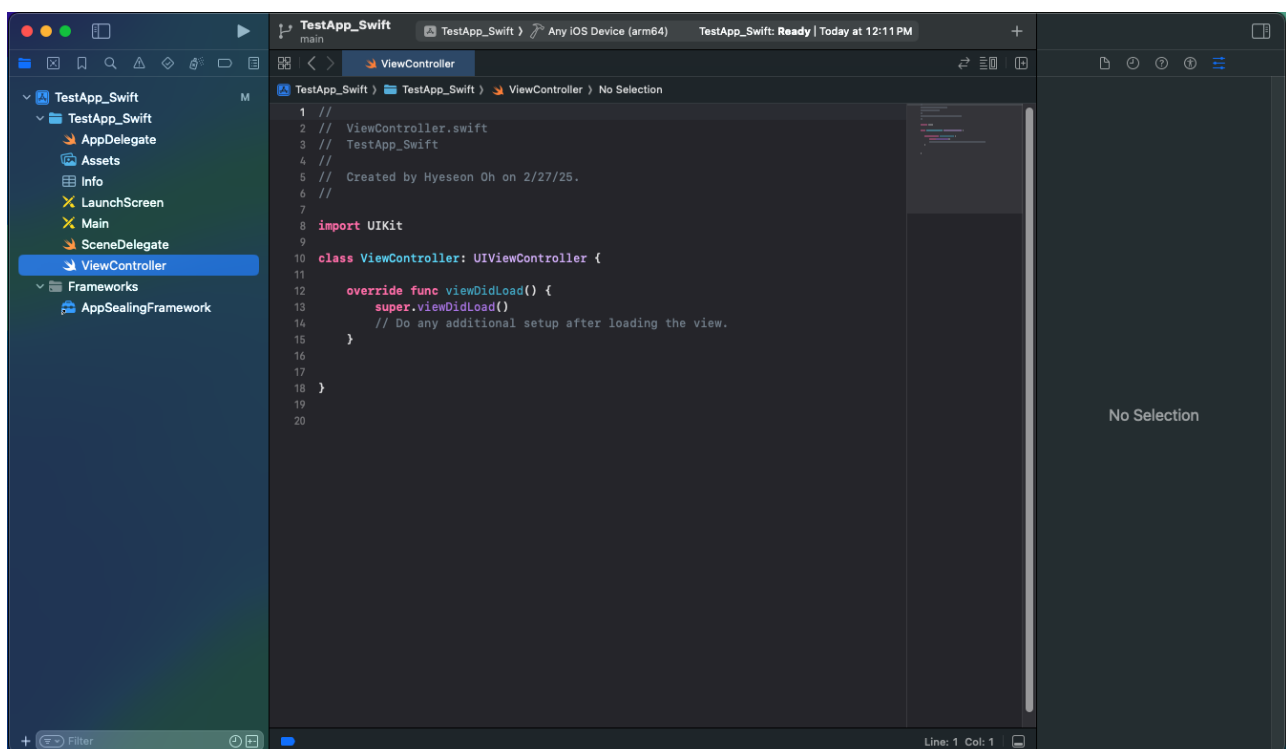
Part 3. 보안 위협 안내를 위한 GUI

AppSealing 라이브러리는 앱이 실행될 때 자동으로 활성화됩니다. AppSealing 라이브러리가 비정상적인 환경 (탈옥된 장치, 실행 파일의 복호화 된 상태, 디버거 연결)을 감지한 경우 사용자 조치와 관계없이 20초 후에 앱이 닫히도록 설계되어 있습니다. 앱 탐지 결과를 사용자에게 통지하고 사용자가 일부 잘못된 환경에 있음을 인식할 수 있도록 적절한 메시지 상자를 표시하십시오.

앱에서 해당 대화 상자를 표시하려면 아래 과정을 참고하여 "ViewController.swift" 파일에 코드를 추가하십시오. (프로젝트가 Objective-C 기반인 경우 ViewController.m 파일을 수정)

3-1 앱에서 UIAlertController 출력

먼저 Xcode project를 열고 왼쪽 프로젝트 패널에서 "ViewController.swift" 파일을 선택합니다.



ViewController.swift 파일을 연 후 해당 파일에 다음 코드를 삽입하십시오. 만일 해당 파일에 'viewDidAppear' 메서드가 이미 포함되어 있다면 최상단에 import 문을 삽입한 뒤 '`super.viewDidAppear( animated );`' 줄 바로 아래 부분의 코드를 삽입하십시오.

Simple UI code into 'ViewController.swift' for Swift project

```

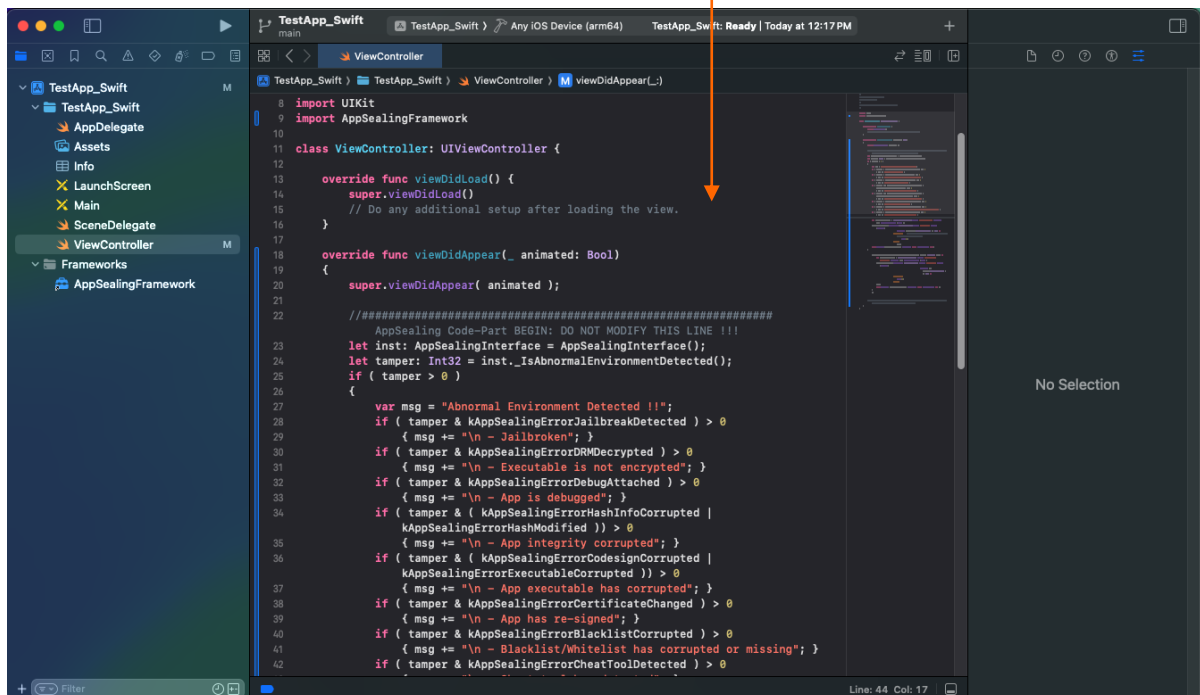
import AppSealingFramework
/* ... */
override func viewDidLoad(_ animated: Bool)
{
    super.viewDidLoadAppear( animated );

    //##### AppSealing Code-Part BEGIN: DO NOT MODIFY THIS LINE !!!
    let inst: AppSealingInterface = AppSealingInterface();
    let tamper: Int32 = inst._IsAbnormalEnvironmentDetected();
    if ( tamper > 0 )
    {
        var msg = "Abnormal Environment Detected !!";
        if ( tamper & kAppSealingErrorJailbreakDetected ) > 0
        { msg += "\n - Jailbroken"; }
        if ( tamper & kAppSealingErrorDRMDecrypted ) > 0
        { msg += "\n - Executable is not encrypted"; }
        if ( tamper & kAppSealingErrorDebugAttached ) > 0
        { msg += "\n - App is debugged"; }
        if ( tamper & ( kAppSealingErrorHashInfoCorrupted | kAppSealingErrorHashModified ) ) > 0
        { msg += "\n - App integrity corrupted"; }
        if ( tamper & ( kAppSealingErrorCodeSignCorrupted | kAppSealingErrorExecutableCorrupted ) ) > 0
        { msg += "\n - App executable has corrupted"; }
        if ( tamper & kAppSealingErrorCertificateChanged ) > 0
        { msg += "\n - App has re-signed"; }
        if ( tamper & kAppSealingErrorBlacklistCorrupted ) > 0
        { msg += "\n - Blacklist/Whitelist has corrupted or missing"; }
        if ( tamper & kAppSealingErrorCheatToolDetected ) > 0
        { msg += "\n - Cheat tool has detected"; }

        let alertController = UIAlertController(title: "AppSealing", message: msg, preferredStyle: .alert );
        alertController.addAction(UIAlertAction(title: "Confirm", style: .default,
            handler: { (action:UIAlertAction!) -> Void in
                #if !DEBUG // Debug mode does not kill app even if security threat has found
                    exit(0);
                #endif
            } ));
        self.present(alertController, animated: true, completion: nil);
    }

    AppSealingInterface._NotifySwizzlingDetected( { (msg: String?) -> () in
        let alertController = UIAlertController( title: "AppSealing Security", message: msg, preferredStyle: .alert )
        alertController.addAction( UIAlertAction( title: "Confirm", style: .default,
            handler: { ( action:UIAlertAction! ) -> Void in
                #if !DEBUG
                    exit(0);
                #endif
            } ));
        self.present( alertController, animated: true, completion: nil );
    });
}
//##### AppSealing Code-Part END: DO NOT MODIFY THIS LINE !!!

```



위의 샘플 UI 코드는 "Code Samples.txt" 파일에도 포함되어 있습니다. 코드를 복사하여 붙여넣으시면 됩니다.

프로젝트가 Objective-C 기반인 경우 다음 코드를 사용하여 UI를 표시할 수 있습니다.

Simple UI code into 'ViewController.m' for Objective-C project

```
#import <AppSealingFramework/AppSealingFramework.h>
-(void)viewDidAppear:(BOOL)animated
{
    [super viewDidAppear:animated];

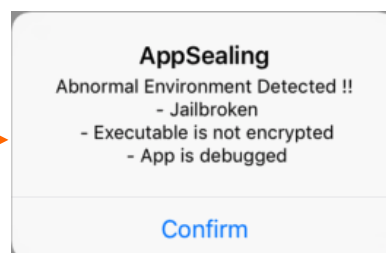
    //##### AppSealing Code-Part BEGIN: DO NOT MODIFY THIS LINE !!!
    int tamper = [[[AppSealingInterface alloc] init] _IsAbnormalEnvironmentDetected];
    if ( tamper > 0 )
    {
        NSString* msg = @"Abnormal Environment Detected !!";
        if (( tamper & kAppSealingErrorJailbreakDetected ) > 0 )
            msg = [msg stringByAppendingString:@"\n - Jailbroken"];
        if (( tamper & kAppSealingErrorDRMDecrypted ) > 0 )
            msg = [msg stringByAppendingString:@"\n - Executable is not encrypted"];
        if (( tamper & kAppSealingErrorDebugAttached ) > 0 )
            msg = [msg stringByAppendingString:@"\n - App is debugged"];
        if (( tamper & ( kAppSealingErrorHashInfoCorrupted | kAppSealingErrorHashModified ) ) > 0 )
            msg = [msg stringByAppendingString:@"\n - App integrity corrupted"];
        if (( tamper & ( kAppSealingErrorCodesignCorrupted | kAppSealingErrorExecutableCorrupted ) ) > 0 )
            msg = [msg stringByAppendingString:@"\n - App executable has corrupted"];
        if (( tamper & kAppSealingErrorCertificateChanged ) > 0 )
            msg = [msg stringByAppendingString:@"\n - App has re-signed"];
        if (( tamper & kAppSealingErrorBlacklistCorrupted ) > 0 )
            msg = [msg stringByAppendingString:@"\n - Blacklist/Whitelist has corrupted or missing"];
        if (( tamper & kAppSealingErrorCheatToolDetected ) > 0 )
            msg = [msg stringByAppendingString:@"\n - Cheat tool has detected"];

        UIAlertController *alert = [UIAlertController alertControllerWithTitle:@"AppSealing"
            message:msg
            preferredStyle:UIAlertControllerStyleAlert];
        UIAlertAction *confirm = [UIAlertAction actionWithTitle:@"Confirm"
            style:UIAlertActionStyleDefault
            handler:^(UIAlertAction * _Nonnull action) {
                #if !DEBUG && !defined(DEBUG) // Debug mode does not kill app even if security threat has found
                    exit(0);
                #endif
            }];
        [alert addAction:confirm];
        [self presentViewController:alert animated:YES completion:nil];
    }

    [AppSealingInterface _NotifySwizzlingDetected:^(NSString* msg){
        UIAlertController *alert = [UIAlertController alertControllerWithTitle:@"AppSealing"
            message:msg
            preferredStyle:UIAlertControllerStyleAlert];
        UIAlertAction *confirm = [UIAlertAction actionWithTitle:@"Confirm"
            style:UIAlertActionStyleDefault
            handler:^(UIAlertAction * _Nonnull action) {
                #if !DEBUG && !defined(DEBUG) // Debug mode does not kill app even if security threat has found
                    exit(0);
                #endif
            }];
        [alert addAction:confirm];
        [self presentViewController:alert animated:YES completion:nil];
    }];
    //##### AppSealing Code-Part END: DO NOT MODIFY THIS LINE !!!
}
```

Xcode나 gdb를 사용하여 앱을 디버깅하거나 비정상적인 장치에서 앱을 실행하면 앱에서 아래와 같은 간단한 경고 상자가 표시됩니다. 이런 상황에서 사용자 조치와 상관없이 앱은 20초 후에 자동으로 종료됩니다.

iOS 기기에서 보안 침입이 감지되면 경고창이 출력되고 20초 후 또는 사용자가 "Confirm" 버튼을 누르면 앱이 닫힙니다.





프로젝트가 Flutter 기반인 경우 ViewController.swift 파일이 생성되지 않으므로 AppDelegate.swift 파일에 다음과 같이 수정된 코드를 추가하여 GUI를 생성할 수 있습니다.

Simple UI code into 'AppDelegate.swift' for Flutter project

```
func application(_ application: UIApplication, didFinishLaunchingWithOptions
launchOptions: [UIApplication.LaunchOptionsKey: Any]?) -> Bool
{
    // Override point for customization after application launch.

    ##### AppSealing Code-Part BEGIN: DO NOT MODIFY THIS LINE !!!
    let inst: AppSealingInterface = AppSealingInterface();
    let tamper: Int32 = inst._IsAbnormalEnvironmentDetected();
    if ( tamper > 0 )
    {
        var msg = "Abnormal Environment Detected !!";
        if ( tamper & kAppSealingErrorJailbreakDetected ) > 0
        { msg += "\n - Jailbroken"; }
        if ( tamper & kAppSealingErrorDRMDecrypted ) > 0
        { msg += "\n - Executable is not encrypted"; }
        if ( tamper & kAppSealingErrorDebugAttached ) > 0
        { msg += "\n - App is debugged"; }
        if ( tamper & ( kAppSealingErrorHashInfoCorrupted | kAppSealingErrorHashModified ) ) > 0
        { msg += "\n - App integrity corrupted"; }
        if ( tamper & ( kAppSealingErrorCodeSignCorrupted | kAppSealingErrorExecutableCorrupted ) ) > 0
        { msg += "\n - App executable has corrupted"; }
        if ( tamper & kAppSealingErrorCertificateChanged ) > 0
        { msg += "\n - App has re-signed"; }
        if ( tamper & kAppSealingErrorBlacklistCorrupted ) > 0
        { msg += "\n - Blacklist/Whitelist has corrupted or missing"; }
        if ( tamper & kAppSealingErrorCheatToolDetected ) > 0
        { msg += "\n - Cheat tool has detected"; }

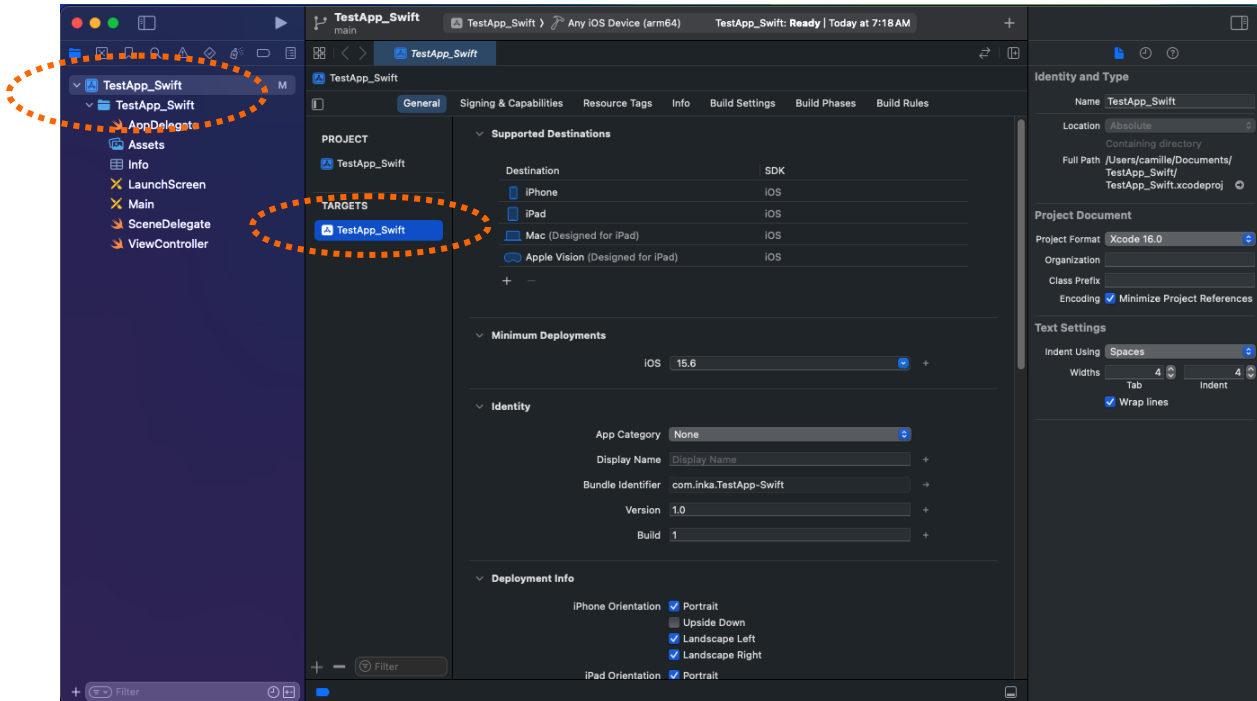
        let alertController = UIAlertController(title: "AppSealing", message: msg, preferredStyle: .alert );
        alertController.addAction(UIAlertAction(title: "Confirm", style: .default,
            handler: { (action: UIAlertAction!) -> Void in
                #if !DEBUG // Debug mode does not kill app even if security threat has found
                    exit(0);
                #endif
            } ));
        DispatchQueue.main.async {
            self.window?.rootViewController?.present( alertController, animated: true, completion: nil)
        };
    }

    AppSealingInterface._NotifySwizzlingDetected( { (msg: String?) -> () in
        let alertController = UIAlertController( title: "AppSealing Security", message: msg, preferredStyle: .alert );
        alertController.addAction( UIAlertAction( title: "Confirm", style: .default,
            handler: { ( action: UIAlertAction! ) -> Void in
                #if DEBUG
                #else
                    exit(0);
                #endif
            } ));
        DispatchQueue.main.async {
            self.window?.rootViewController?.present( alertController, animated: true, completion: nil )
        };
    });

    ##### AppSealing Code-Part END: DO NOT MODIFY THIS LINE !!!
    return true;
}
```

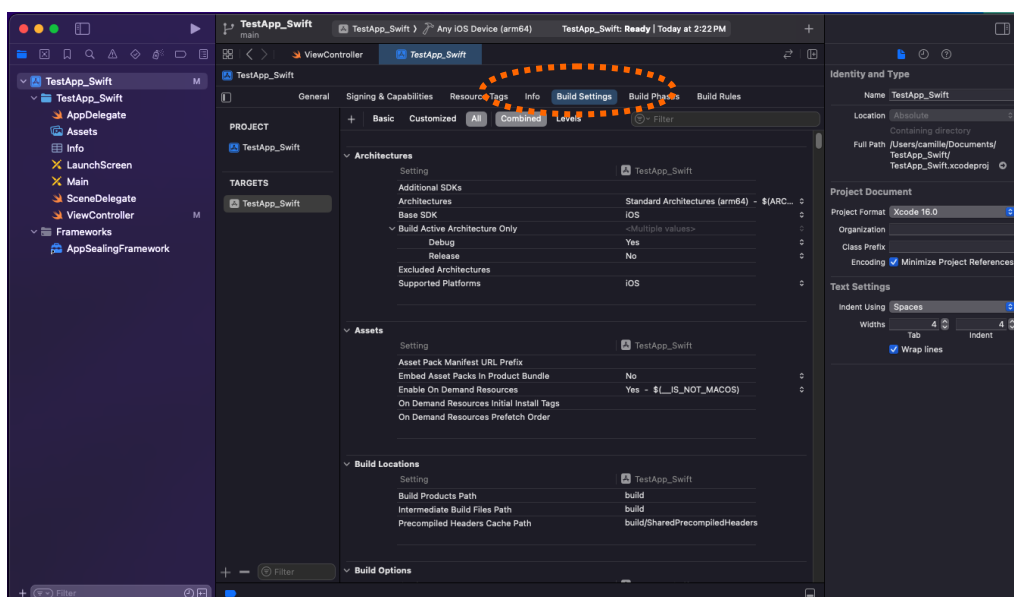
Part 4. 프로젝트 “Build Settings” 수정

4-1 프로젝트 메인 target 선택

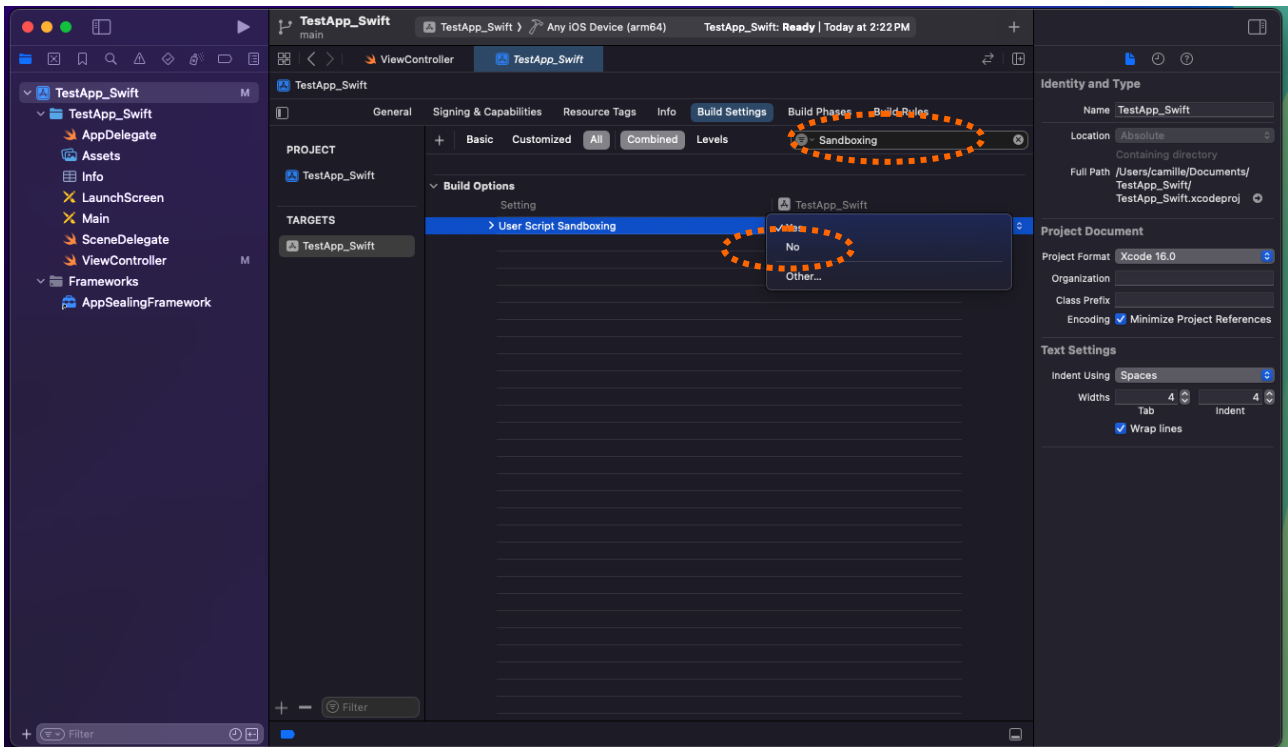


4-2 User Script Sandboxing 비활성화

2-4단계에서 추가한 스크립트는 User Script Sandboxing 환경 하에서 잘 동작하지 않습니다. 이를 수정하기 위해 Build Settings로 갑니다.



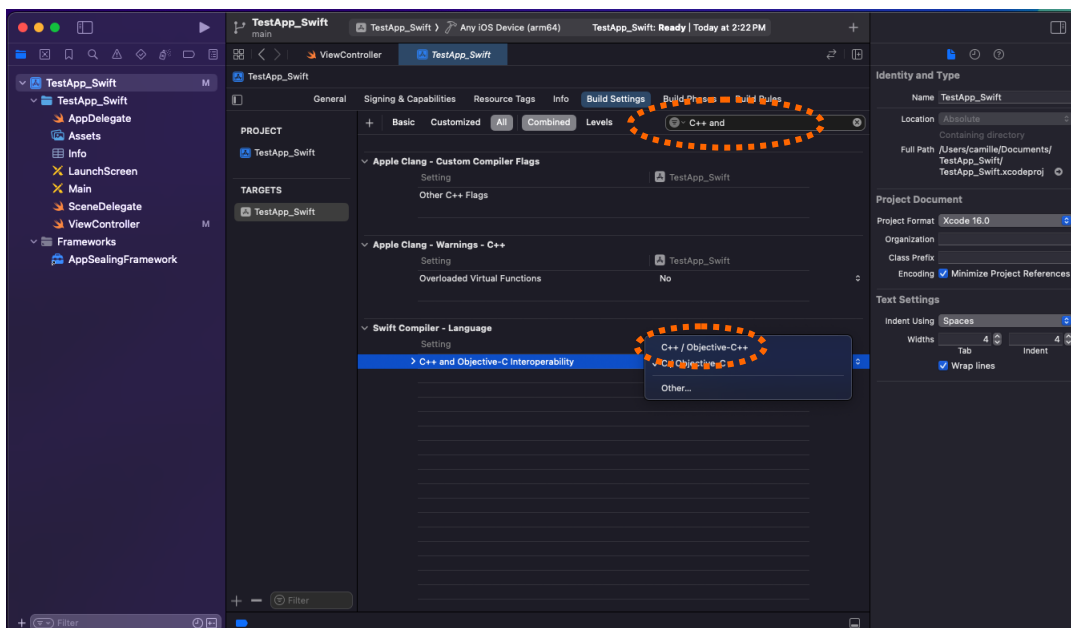
Sandboxing을 검색하여 **"User Script Sandboxing"** 필드가 나타나면 필드 값을 눌러 **No**로 변경하십시오.



4-3 C++ and Objective-C Interoperability 변경

본 보안 프레임워크에는 Objective-C와 C++ 코드가 같이 들어 있으므로, 또 다른 빌드 세팅을 바꾸어야 빌드 에러가 발생하지 않습니다.

"C++ and"를 검색한 뒤, **"C++ and Objective-C Interoperability"** 필드가 나타나면 그 값을 "C++ / Objective-C++"로 변경하십시오.

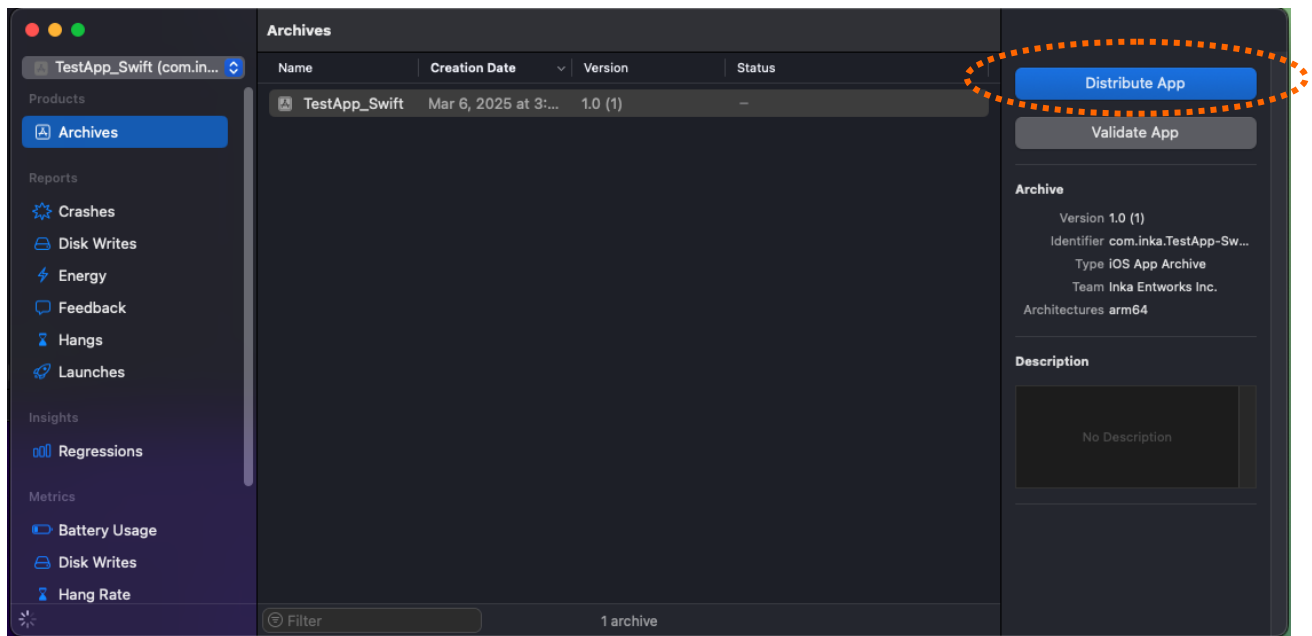


4-4 앱 무결성 및 인증서 검증 정보 생성

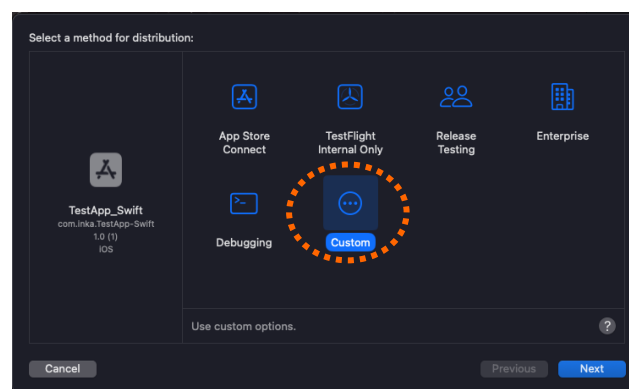
앱을 테스트하거나 스토어에 배포할 경우 앱의 무결성 검증을 위한 추가 작업을 진행해야 합니다. 이 단계를 건너뛰는 경우 단말에서 앱을 실행하면 무결성 검증에 실패하여 일정 시간 후 앱이 종료됩니다.

TestFlight나 App Store 배포 시, 혹은 AdhocEnabled SDK를 사용하여 Ad Hoc 배포 시에는 반드시 이 단계를 수행해야 합니다.

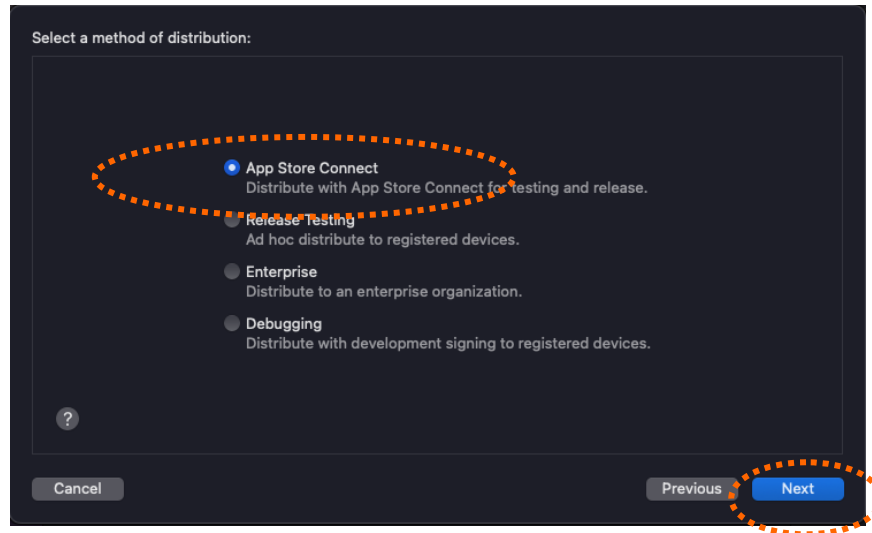
앱스토어나 TestFlight에 앱을 업로드 하기 위한 과정을 단계별로 살펴보겠습니다. 아래는 Archive가 완료된 상태의 Organizer 화면입니다.



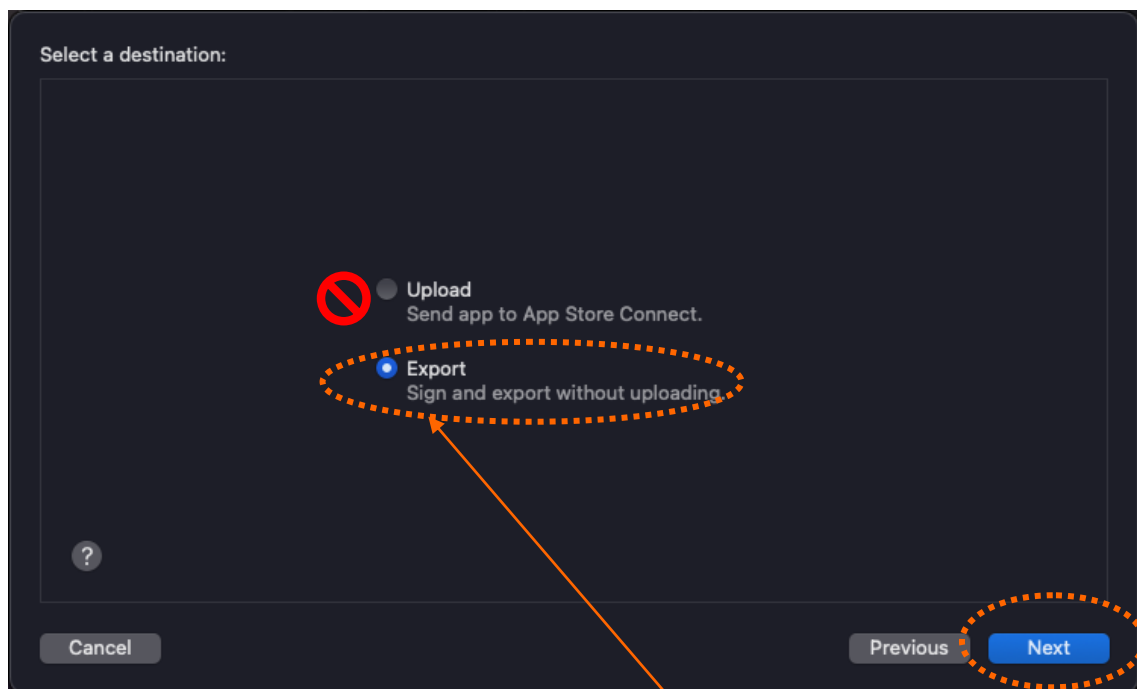
이 상태에서 업로드 할 IPA를 생성하기 위해 "Distribute App" 버튼을 클릭합니다.



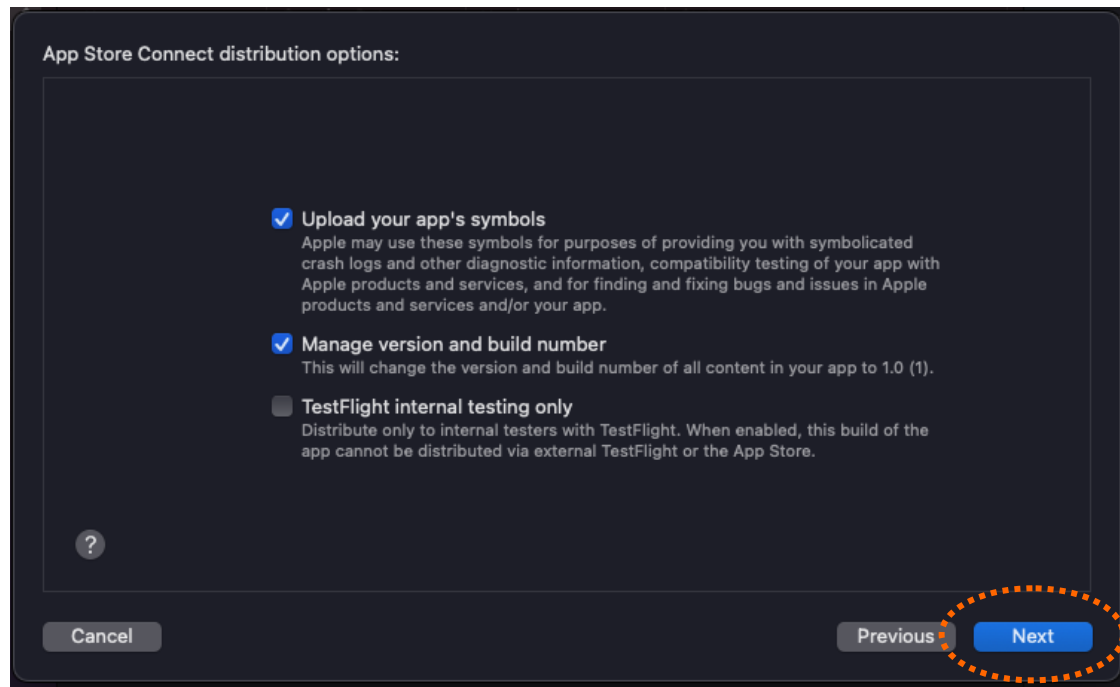
일반적으로는 "App Store Connect"를 바로 선택하게 되겠지만, 지금은 "Custom"의 다른 배포 옵션들이 필요합니다. "Custom"을 선택하고 "Next"를 클릭합니다.



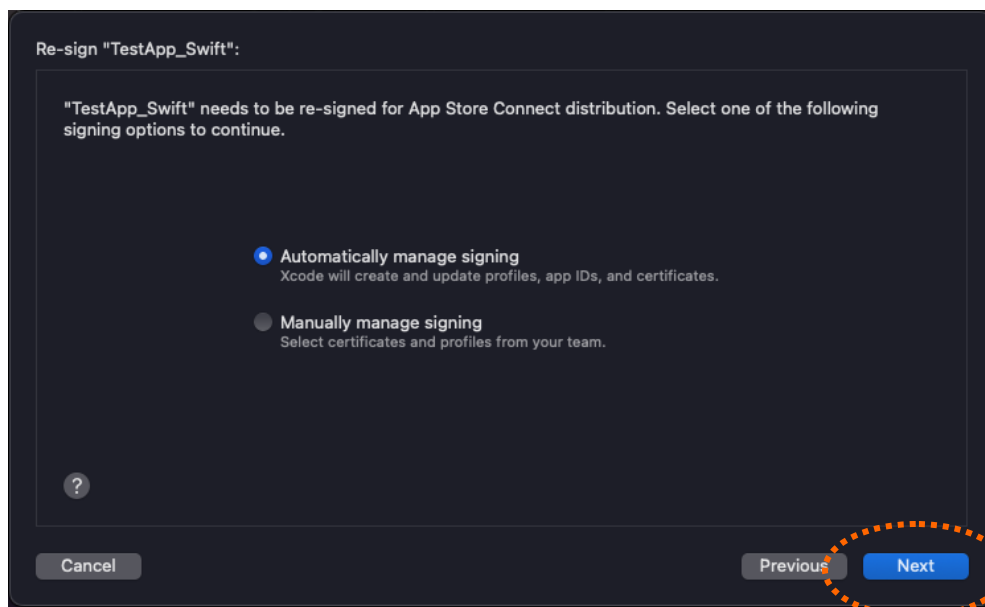
"App Store Connect"가 선택된 상태에서 "Next" 버튼을 클릭합니다. (AdhocEnabled SDK를 사용하는 경우 다른 선택지도 선택이 가능합니다.)



"Upload"를 선택하는 경우가 대부분이지만 **AppSealing 적용을 위해서는 반드시 "Export"를 선택**해야 합니다. 이는 업로드 전 무결성 및 인증서 스냅샷 처리 작업이 필요하기 **때문으로 이 과정이 잘못되거나 생략되면 앱이 정상적으로 실행되지 않으므로 유의해야 합니다.** "Export"가 선택된 상태에서 "Next" 버튼을 클릭합니다.

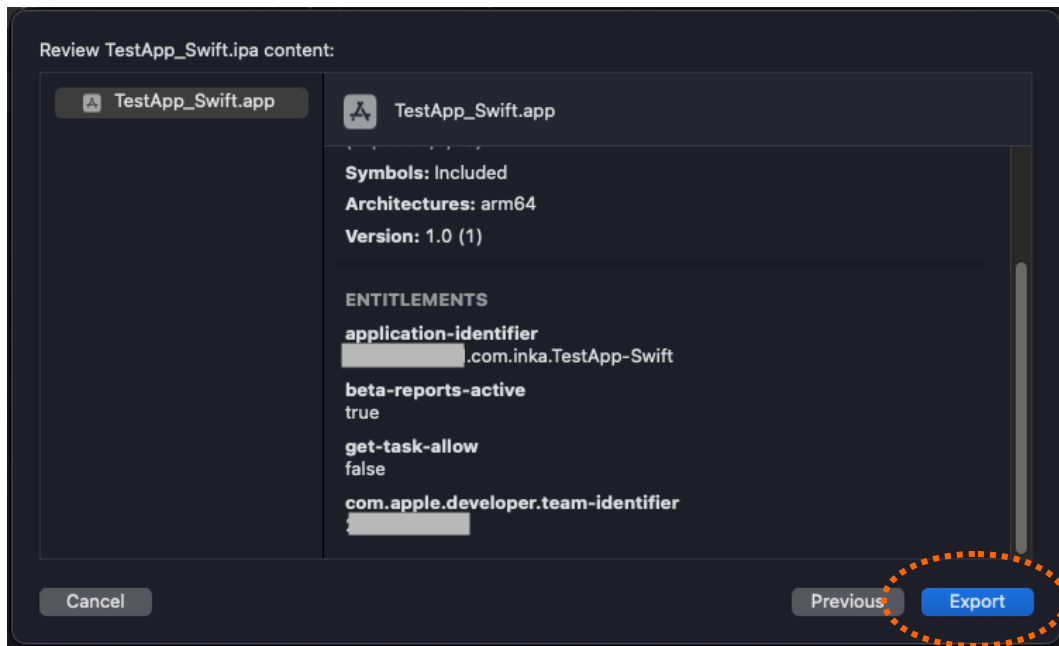


"Next" 버튼을 클릭하거나, 필요한 옵션을 조정합니다.

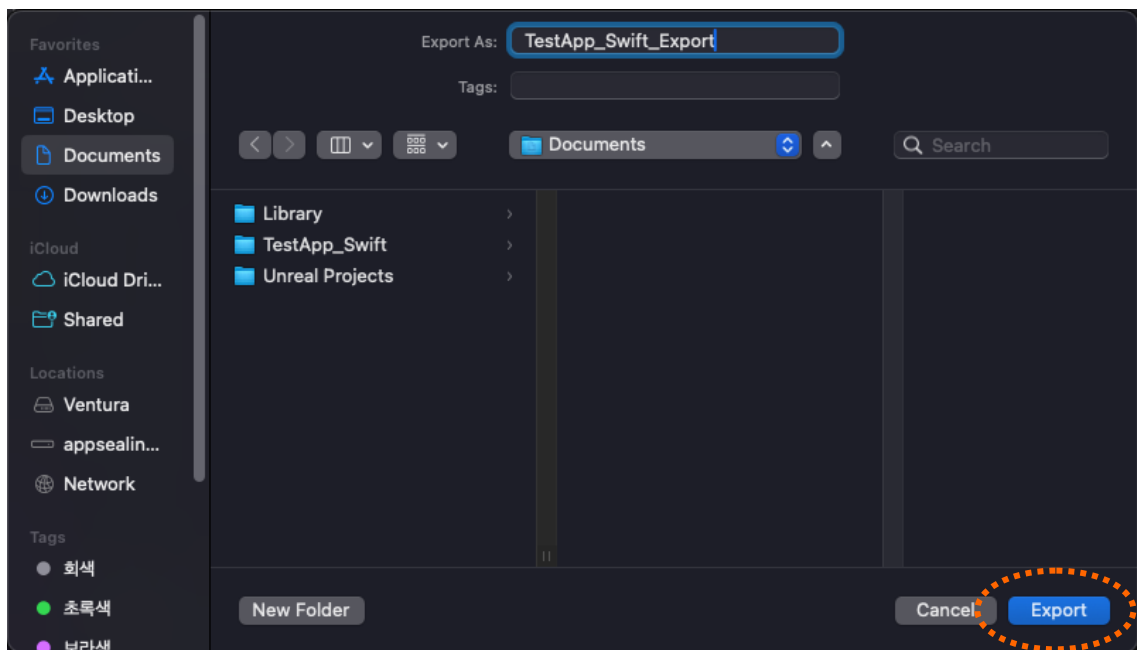


프로비저닝 프로파일을 사용하는지 자동 서명을 사용하는지 여부에 따라 일반적으로 사용하던 값을 선택합니다. 여기서는 자동으로 서명을 관리한다 간주하고 "Next"를 누르겠습니다.

이제 다음과 같이 IPA 파일로 export할 수 있는 창이 나타납니다.

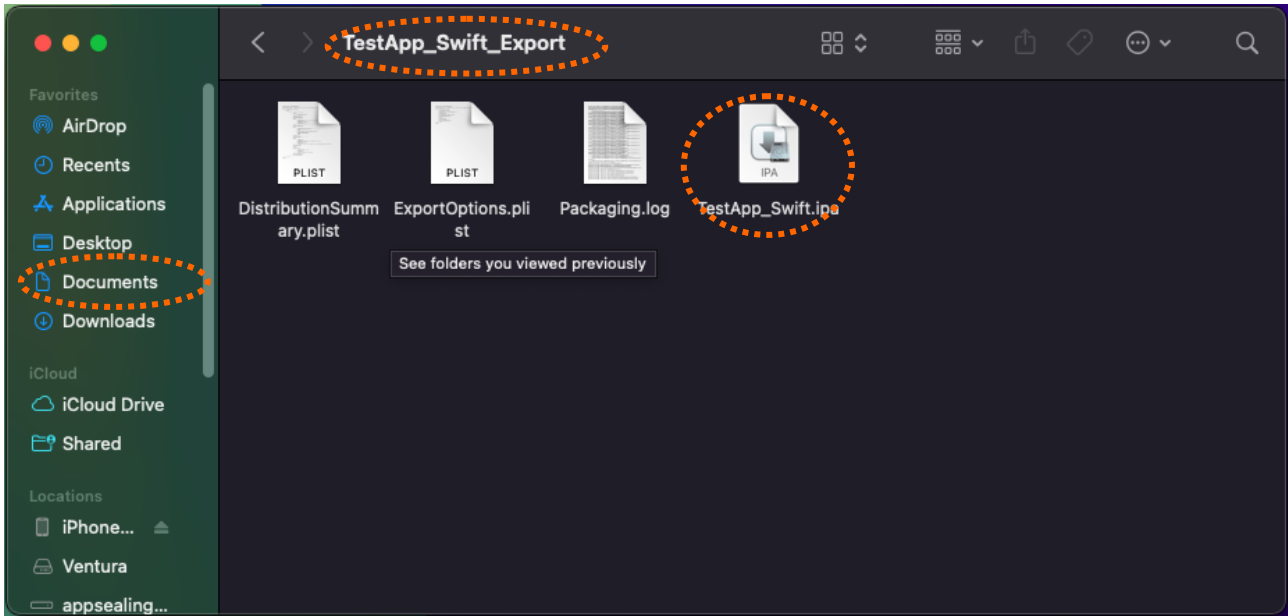


내용을 확인한 후 "Export" 버튼을 클릭합니다.

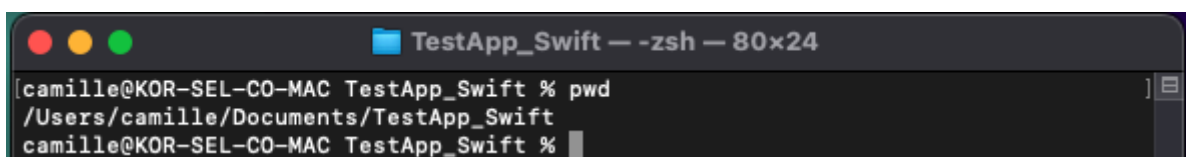


Export 위치를 지정하는 창이 표시되면 저장 위치를 선택하고 "Export"를 클릭합니다. 여기서는 ~/Downloads/TestApp_Swift_Export 라는 이름의 폴더를 사용했습니다.

“Export”를 클릭하면 해당 폴더에 IPA 파일이 생성됩니다. 아래와 같이 Finder에서 확인할 수 있습니다. 이제 이 폴더의 위치를 잘 기억하거나 또는 Finder에서 해당 폴더로 이동한 후 창을 열어 놓습니다.



이제 Export된 IPA 파일을 가지고 작업을 진행해야 합니다. 터미널 앱을 실행하고 Xcode 프로젝트 폴더로 이동합니다.



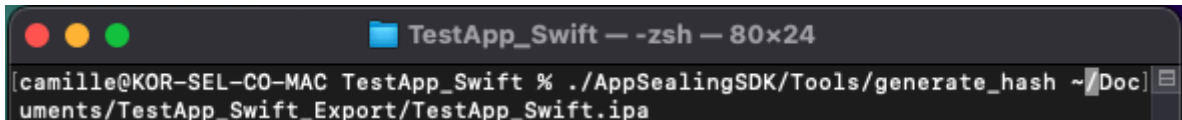
이 문서에서는 Xcode 프로젝트 경로로 ~/Documents/TestApp_Swift를 사용했습니다. 위 그림처럼 해당 위치로 이동해서 pwd를 실행하면 경로가 출력되는 것을 확인할 수 있습니다.

다음 명령으로 Tools 폴더 안의 파일들에 실행 권한을 부여합니다.

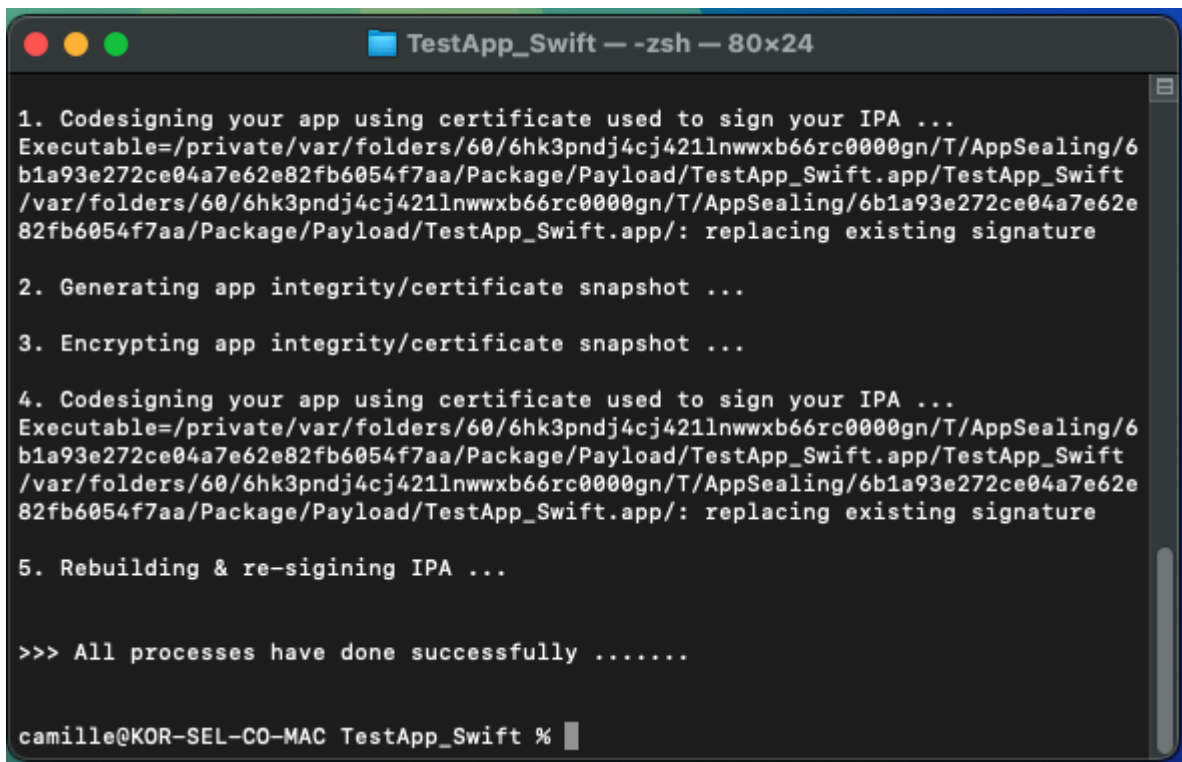
```
$ chmod +x AppSealingSDK/Tools/*
```

이제 아래와 같이 generate_hash 스크립트를 실행합니다. 파라미터는 이전 단계에서 Export 했던 IPA 파일의 경로입니다. 경로를 직접 타이핑 하지 않고 열어 두었던 Finder 창에서 TestApp_Swift.ipa 파일을 터미널 창으로 드래그&드롭 해도 됩니다.

```
$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Export/TestApp_Swift.ipa
```



스크립트를 실행하면 아래 화면과 같이 진행 상황이 표시되면서 지정한 IPA에 무결성 검증 정보와 인증서 검증 정보를 추가하게 됩니다.



이 과정은 "Ad Hoc", "Enterprise", "Development"로 배포할 경우에도 동일하게 적용됩니다.

4-5 메서드 스위즐링 / 메서드 후킹 감지 기능 제어

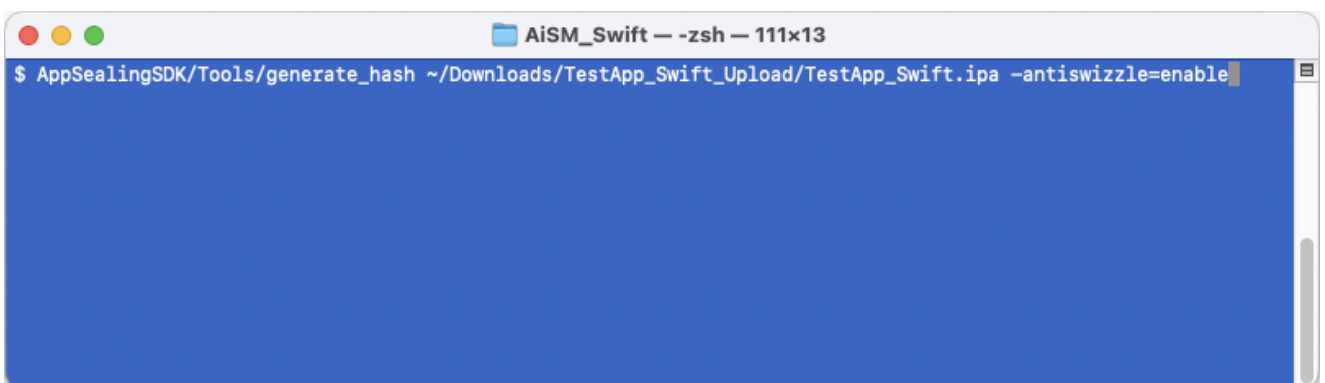
메서드 스위즐링은 특정 클래스의 기존 메소드의 기능을 새로운 메소드의 기능으로 교환하여 원래의 메서드를 호출할 때 실제로는 새로운 메서드의 코드가 실행되도록 하는 기술로 주로 Objective-C에서 사용되며 이를 통해 기존 코드의 동작을 유연하게 수정하거나 확장할 수 있게 해줍니다.

메서드 스위즐링은 유용한 기술이지만 공격자는 메서드 스위즐링을 통해 기존의 메소드를 자신의 악성 코드로 교환할 수도 있습니다. 예를 들어 사용자의 입력을 처리하는 메서드를 수정하여 데이터를 탈취하거나, 원하지 않는 행동을 실행하도록 만들 수 있습니다. 또한 시스템의 로깅 메서드를 스위즐링하여 중요한 이벤트가 기록되지 않도록 할 수 있고 이를 통해 악의적인 행동이 감지되지 않게 할 수 있습니다.

AppSealing SDK에는 이러한 메서드 스위즐링 동작을 감지할 수 있는 기능이 추가되어 있지만 메서드 스위즐링이 항상 공격의 수단으로만 사용되는 것이 아니기에 이를 감지할 것인지 하지 않을 것인지를 직접 지정할 수 있습니다.

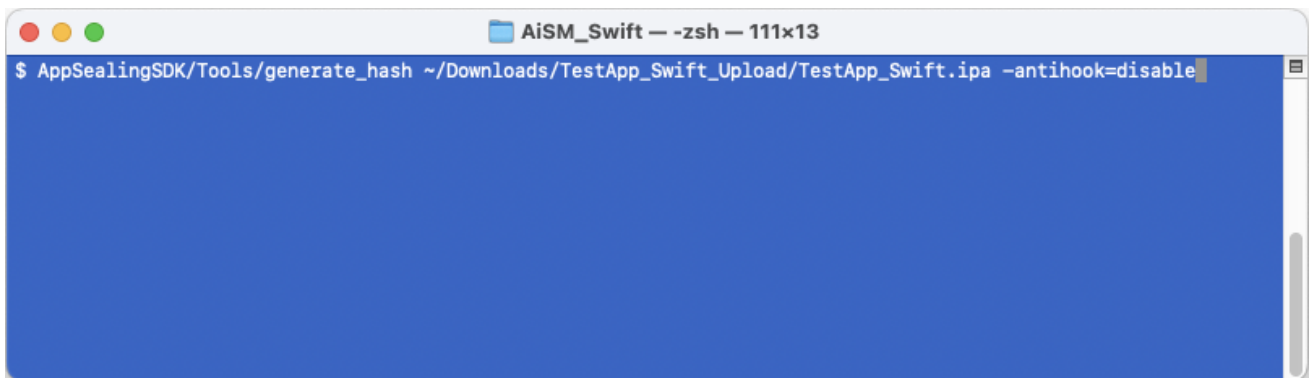
기본적으로 오동작을 막기 위해 메서드 스위즐링 감지 기능이 비활성화 된 상태로 SDK가 배포되지만 generate_hash 스크립트를 실행하는 과정에서 이를 활성화 시킬 수 있습니다. 만약 앱 테스트 단계에서 메서드 스위즐링이 감지되었다는 보안 경고가 발생할 경우 앱에 포함된 다른 라이브러리에서 의도적으로 메서드 스위즐링을 사용하는 경우가 있을 수 있기 때문에 이 경우는 메서드 스위즐링 감지를 활성화 시키면 안 됩니다.

generate_hash 스크립트 실행 시 메서드 스위즐링 감지 기능을 활성화 하려면 다음과 같이 **antiswizzle=enable** 파라미터를 추가한 상태로 generate_hash를 실행하면 됩니다.

A screenshot of a macOS terminal window titled "AiSM_Swift - zsh - 111x13". The terminal shows a command being executed: "\$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa -antiswizzle=enable". The command is highlighted with a blue background. The terminal output area is currently empty.

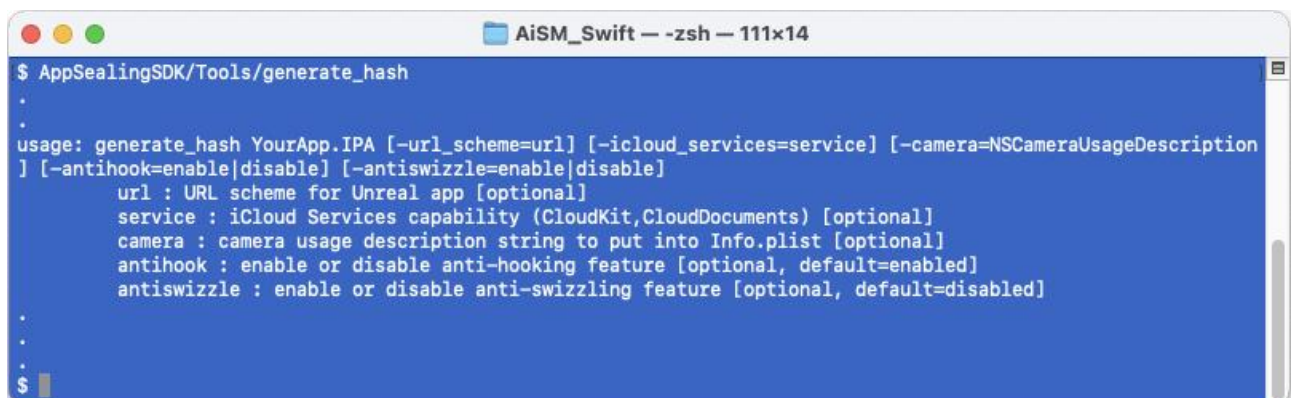
메서드 스위즐링과 함께 사용되는 공격 방법으로 메서드 후킹이 있습니다. 메서드 후킹은 메서드 스위즐링과 유사하지만 메서드를 서로 바꾸는 것이 아니라 메서드의 실행 흐름을 직접 조작하고 제어한다는 차이점이 있습니다. 메서드 스위즐링은 필요에 따라 앱이나 특정 라이브러리에서 사용될 수 있지만 정상적인 앱과 라이브러리에서 메서드 후킹이 사용되는 경우는 없기 때문에 AppSealing SDK에는 메서드 후킹 감지 기능이 기본으로 활성화되어 있지만 필요한 경우 이 기능을 비활성화 시킬 수 있습니다.

메서드 후킹 감지 기능을 비활성화하려면 다음과 같이 “**antihook=disable**” 파라미터를 추가하여 generate_hash 스크립트를 실행하면 됩니다.



```
AiSM_Swift - zsh - 111x13
$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa -antihook=disable
```

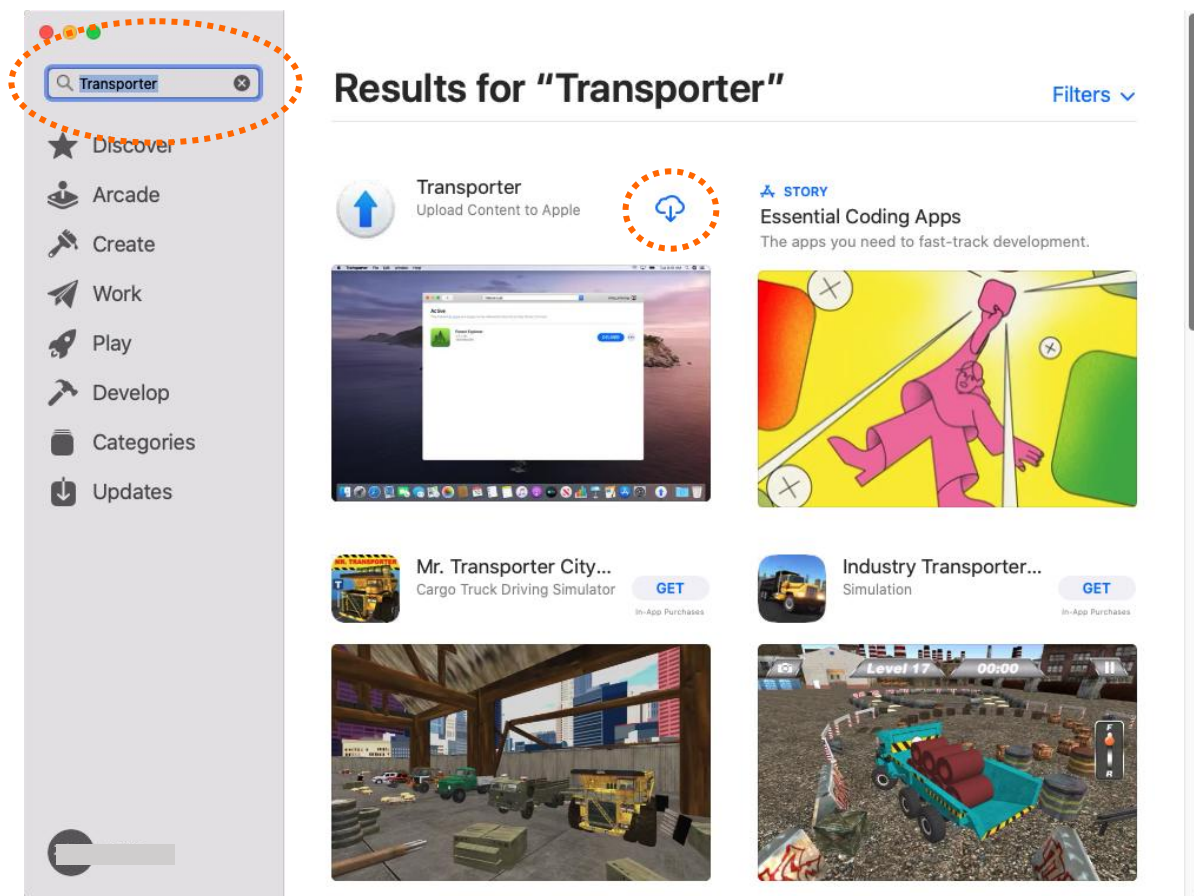
generate_hash 스크립트의 모든 파라미터를 확인하고 싶을 경우는 IPA 경로명 없이 스크립트만 실행하면 아래 화면과 같이 사용 가능한 모든 파라미터를 확인할 수 있습니다.



```
AiSM_Swift - zsh - 111x14
$ AppSealingSDK/Tools/generate_hash
.
.
usage: generate_hash YourApp.IPA [-url_scheme=url] [-icloud_services=service] [-camera=NSCameraUsageDescription]
[-antihook=enable|disable] [-antiswizzle=enable|disable]
    url : URL scheme for Unreal app [optional]
    service : iCloud Services capability (CloudKit,CloudDocuments) [optional]
    camera : camera usage description string to put into Info.plist [optional]
    antihook : enable or disable anti-hooking feature [optional, default=enabled]
    antiswizzle : enable or disable anti-swizzling feature [optional, default=disabled]
.
.
.
$
```

4-6 재서명 앱 App Store Connect 업로드

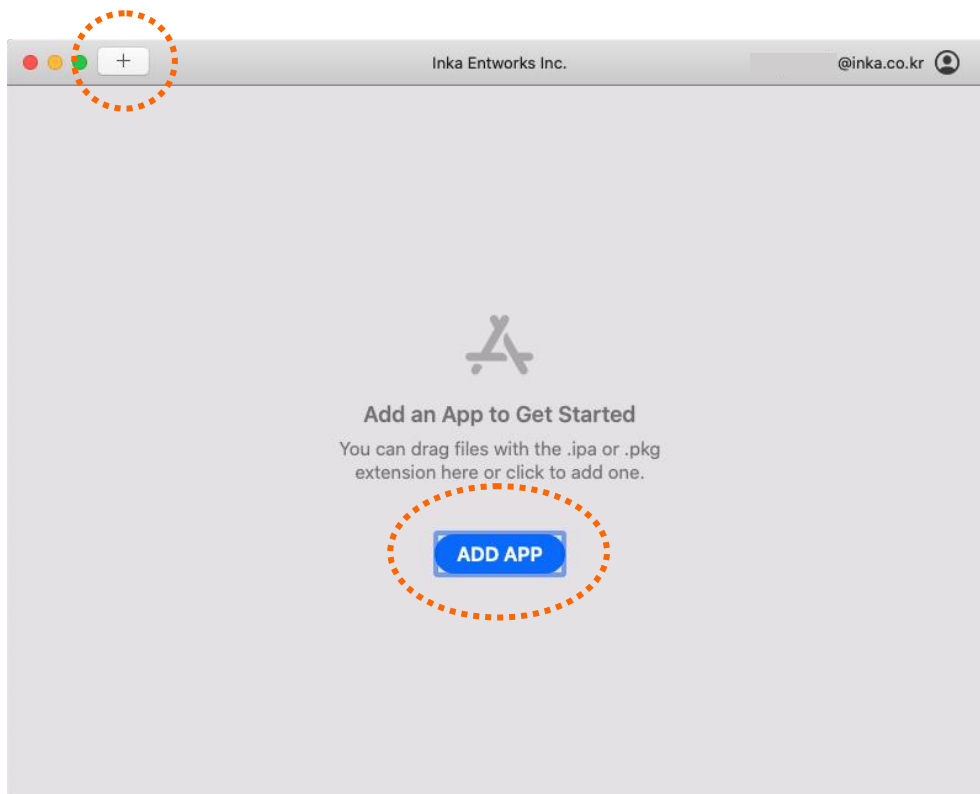
이제 재서명이 완료된 IPA를 App Store Connect에 업로드 합니다. (물론 AdhocEnabled SDK를 사용한 Ad Hoc 배포라면 이 과정이 필요 없습니다.) 간편한 업로드를 위해 Transporter 앱을 사용하겠습니다. macOS에 Transporter가 설치되어 있지 않은 경우 App Store를 실행하고 "Transporter"를 검색하여 설치합니다.

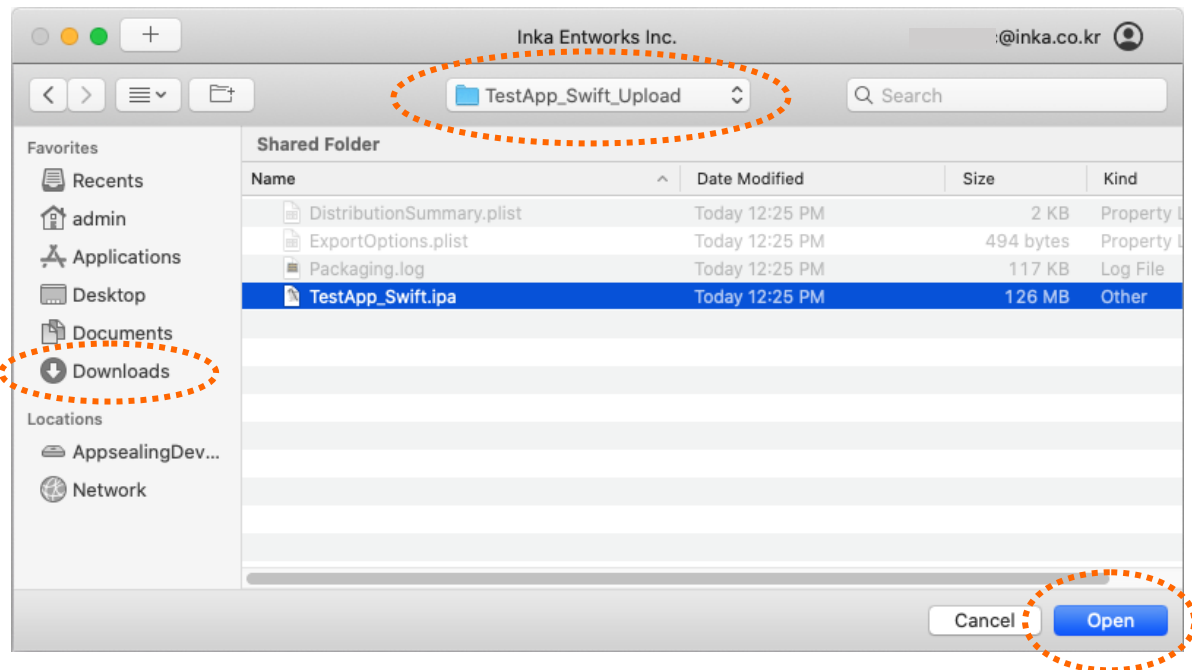


설치를 완료하고 실행하면 아래와 같이 Apple ID를 입력하는 창이 나타납니다. Apple 계정 ID와 비밀번호를 입력합니다. (이 과정은 최초 1회만 수행하면 됩니다)

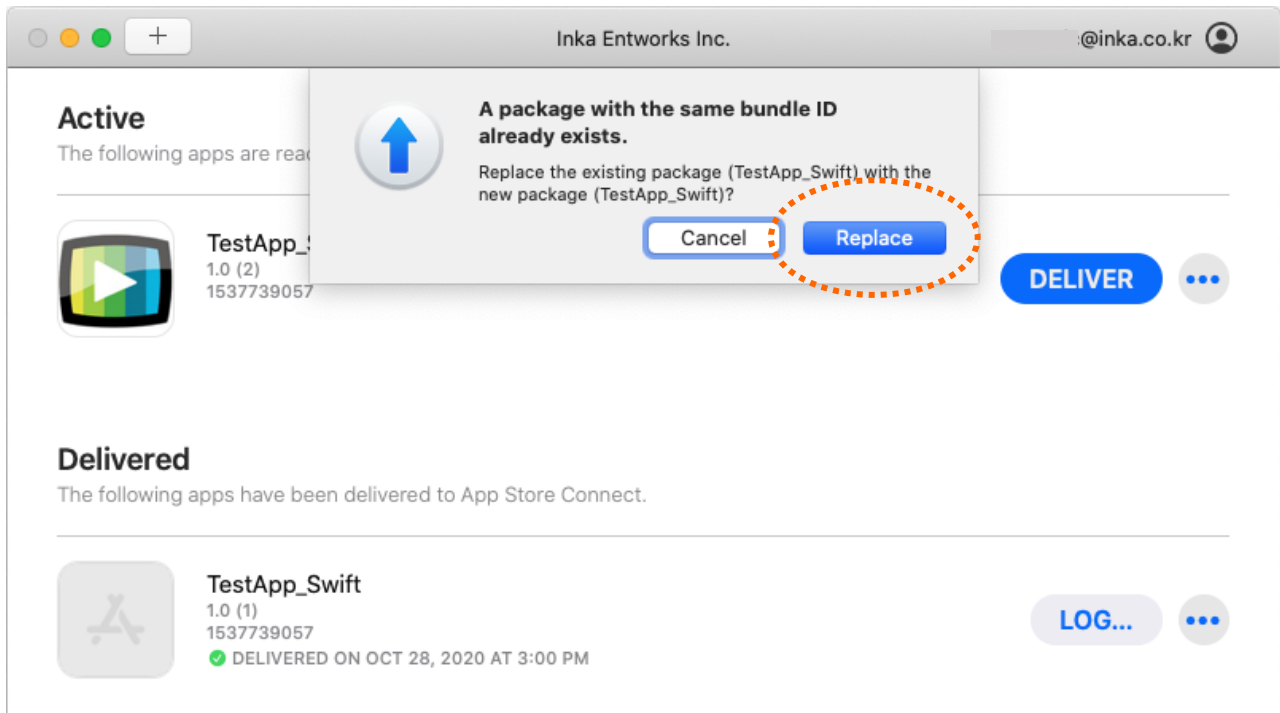


ID와 비밀번호를 입력하고 로그인하면 아래와 같은 창이 나타나며 여기서 업로드할 IPA를 선택하기 위해 좌측 상단의 + 버튼 또는 창 중앙의 "ADD APP" 버튼을 클릭하고 이전 단계에서 재서명이 완료된 IPA 파일을 선택합니다.

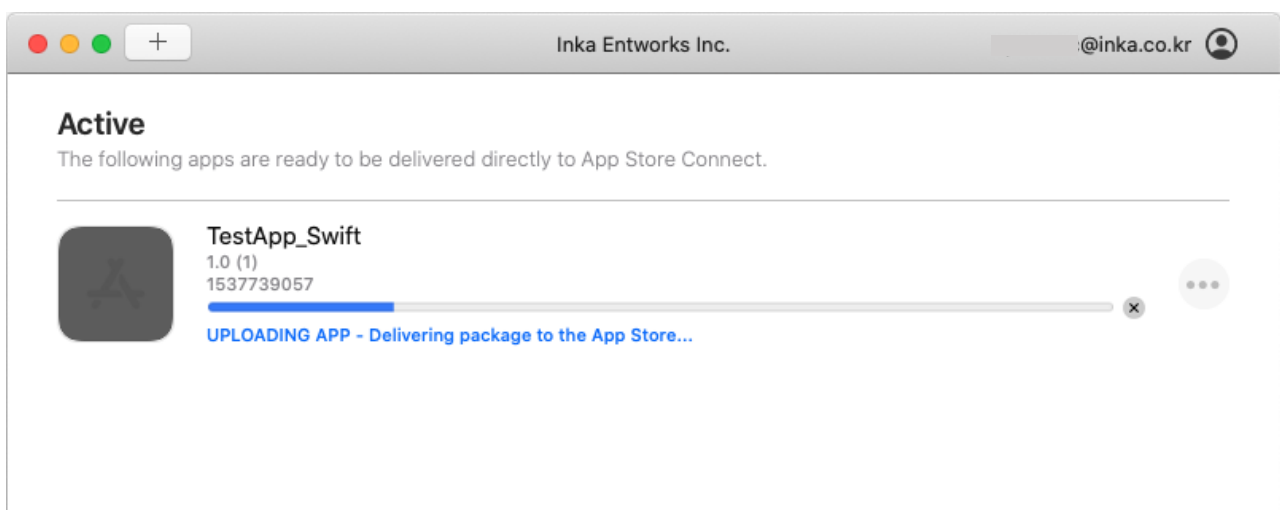
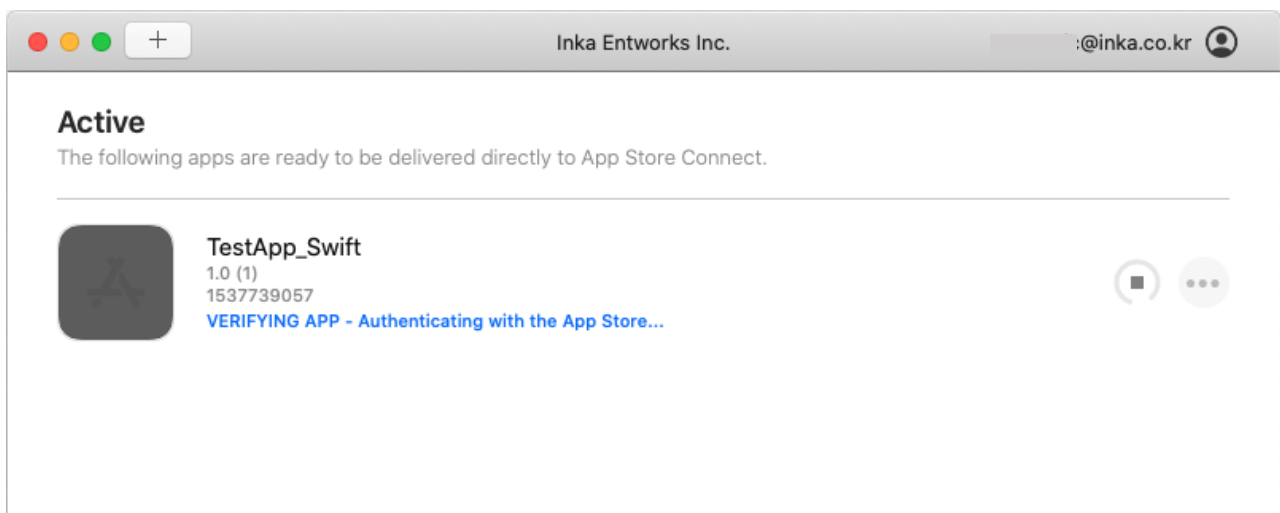
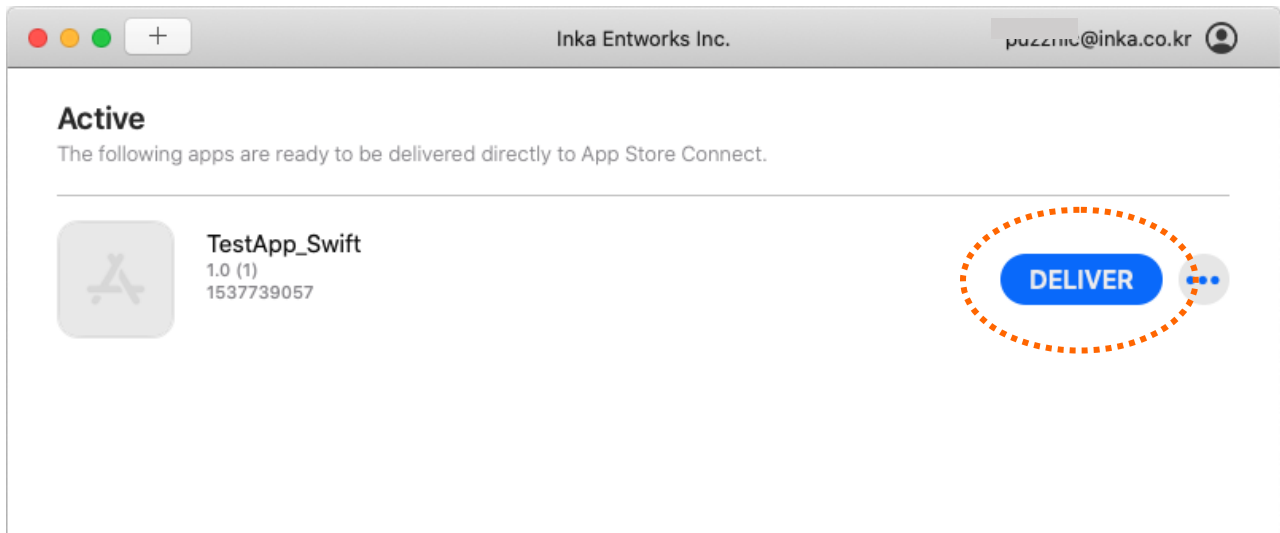


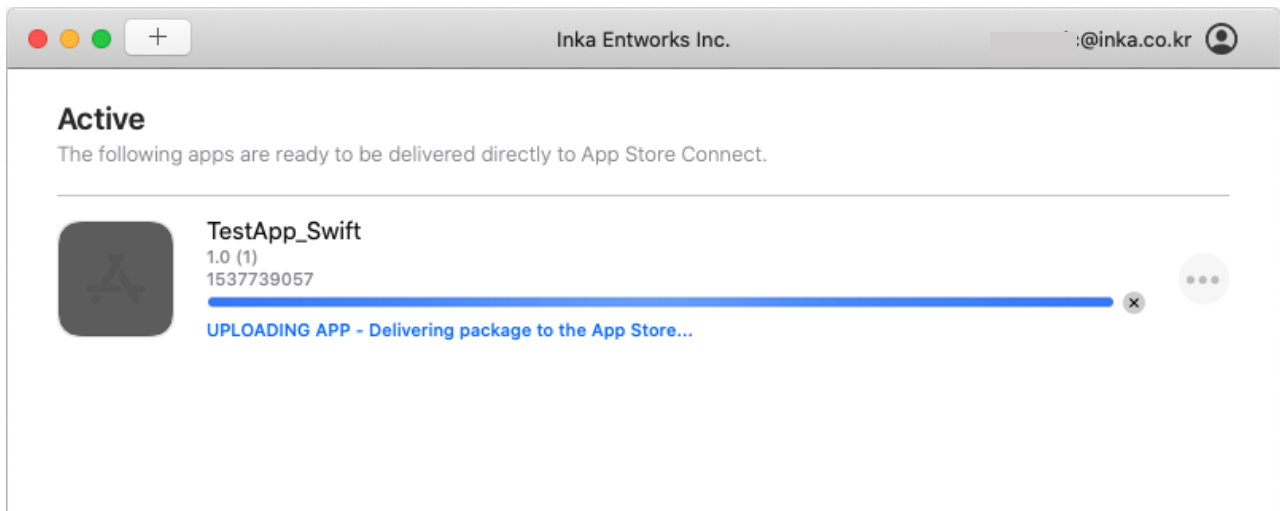


앱 최초 전송 이후 앱의 버전이나 빌드 번호를 증가시키고 새로 업로드하려고 할 경우 IPA 파일을 추가하면 번들 ID가 동일하기 때문에 아래와 같은 확인 창이 나타날 수 있습니다. 이 경우 "Replace" 버튼을 클릭합니다.

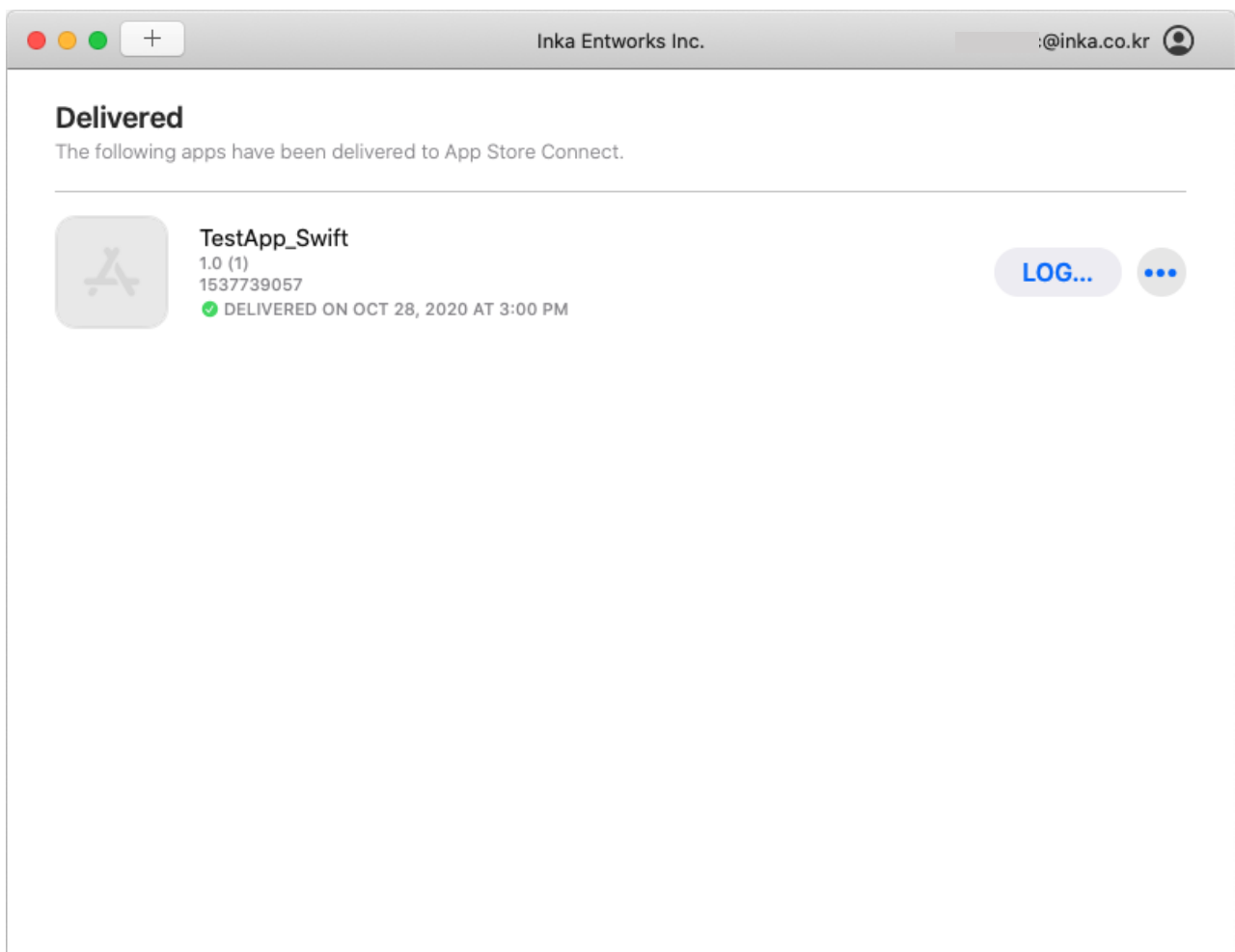


IPA가 추가된 상태에서 "DELIVER" 버튼을 클릭하면 앱 검증과 App Store Connect 업로드 과정이 진행됩니다.





아래 화면과 같이 전송이 완료되면 App Store Connect에서 심사를 위한 빌드 제출이나 TestFlight용 빌드 배포를 진행할 수 있습니다.



Part 5. AppSealing 기기 고유 ID 획득

AppSealing SDK는 기기별로 고유 ID를 생성하고 관리합니다. AppSealing SDK를 사용하는 고객은 필요한 경우 AppSealing에서 제공하는 인터페이스를 이용하여 기기 고유 ID를 확인할 수 있습니다. 확인된 고유 ID는 AppSealing에서 제공하는 해킹 데이터 서비스와 연계하여 비즈니스에 활용할 수 있습니다.

5-1 기기 고유 ID 확인

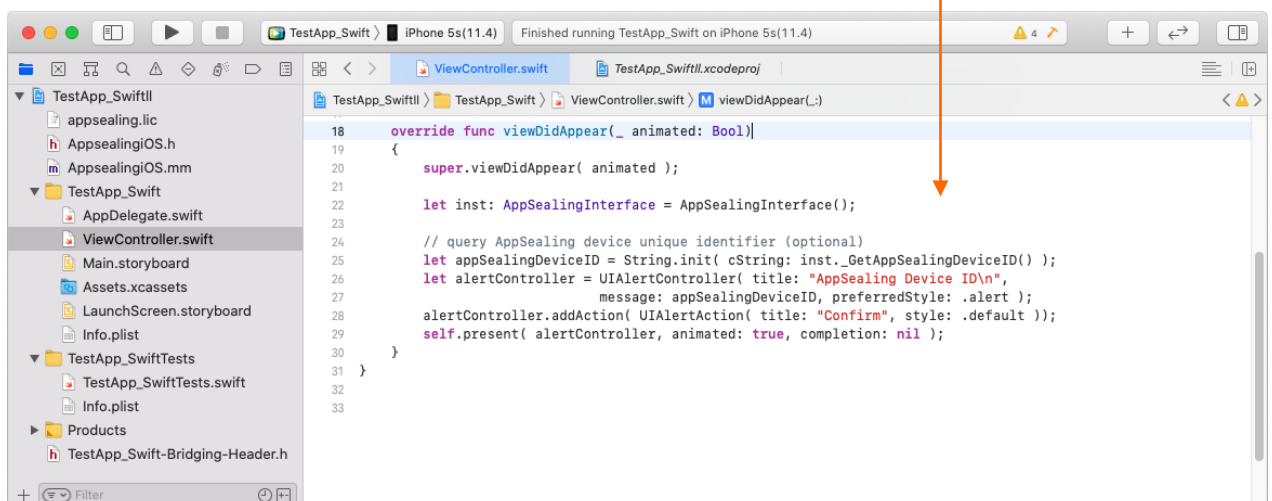
다음 예시에서는 앱이 구동될 때 고유 ID를 확인합니다. 프로젝트의 "ViewController.swift" 파일을 열고 다음 코드를 삽입하십시오. 만약 ViewController.swift 파일에 'viewDidAppear' 메서드가 이미 있다면 'super.viewDidAppear(animated);' 줄 아래에 다음 코드를 삽입하십시오.)

Simple UI code into 'ViewController.swift' for Swift project

```
override func viewDidAppear(_ animated: Bool)
{
    super.viewDidAppear( animated );

    let inst: AppSealingInterface = AppSealingInterface();
    let appSealingDeviceID = String.init( cString: inst._GetAppSealingDeviceID() );
    let alertController = UIAlertController( title: "AppSealing DeviceID",
                                           message: appSealingDeviceID, preferredStyle: .alert );

    alertController.addAction( UIAlertAction( title: "Confirm", style: .default ));
    self.present( alertController, animated: true, completion: nil );
}
```



프로젝트가 Objective-C 기반인 경우 다음 코드를 사용하여 UI를 표시할 수 있습니다.

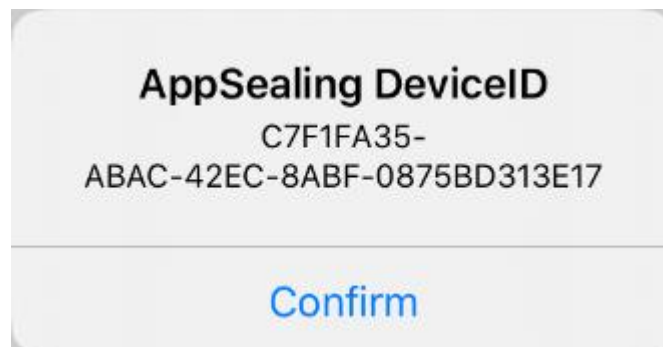
Simple UI code into 'ViewController.mm' for **Objective-C** project

```
#include "AppsealingiOS.h"

- (void)viewDidAppear:(BOOL)animated
{
    [super viewDidAppear:animated];

    AppSealingInterface* inst = [[AppSealingInterface alloc] init];
    char* _appSealingDeviceID = [inst GetAppSealingDeviceID:_appSealingDeviceID];
    UIAlertController *alert = [UIAlertController alertControllerWithTitle:@"AppSealing DeviceID"
                                                                    message:[NSString alloc] initWithUTF8String:_appSealingDeviceID
                                                                    preferredStyle:UIAlertControllerStyleAlert];
    UIAlertAction *confirm = [UIAlertAction actionWithTitle:@"Confirm"
                                                            style:UIAlertActionStyleDefault
                                                            handler:^(UIAlertAction * _Nonnull action) { }];
    [alert addAction:confirm];
    [self presentViewController:alert animated:YES completion:nil];
}
```

앱을 실행하면 아래와 같이 메시지 박스에서 기기 고유 ID를 확인할 수 있습니다.



Part 6. 소스 코드 문자열 암호화 적용

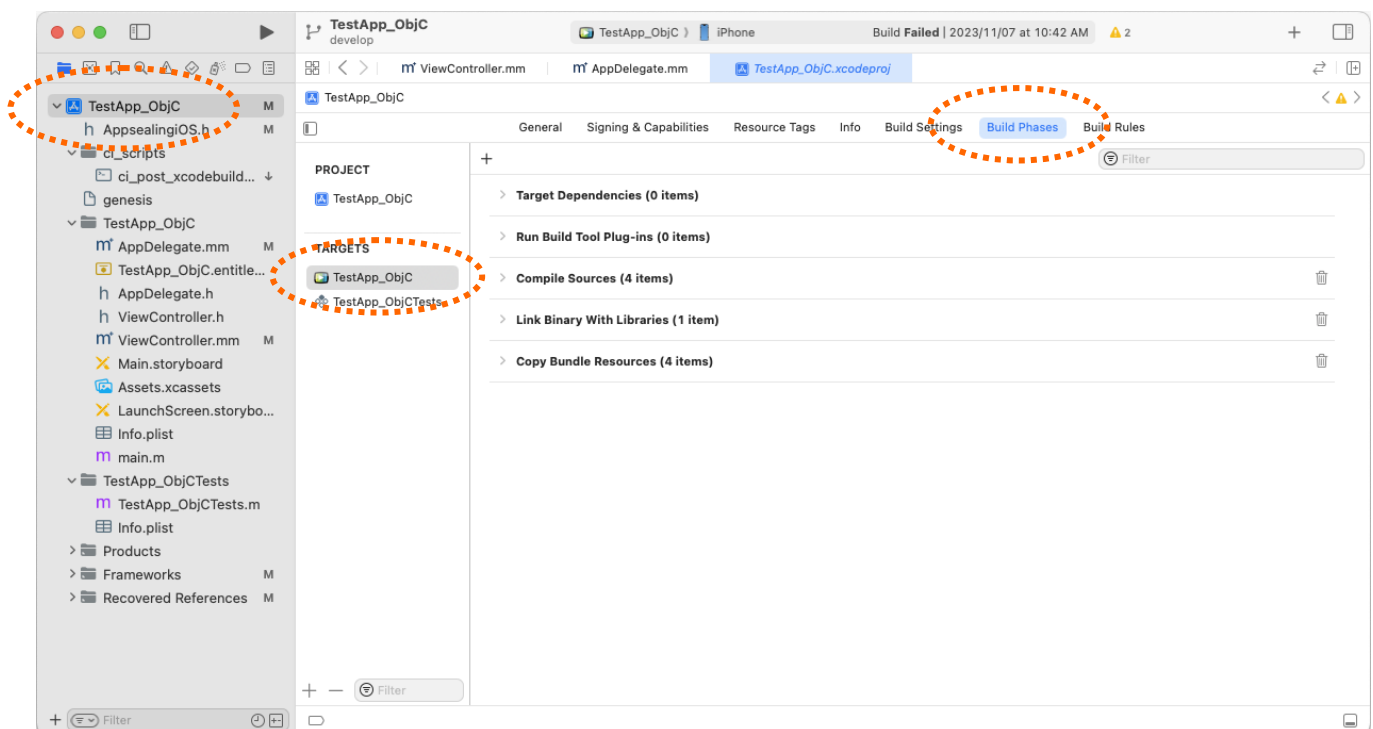
6-1 프로젝트 Build Phase 설정

AppSealing SDK는 부가 기능으로 소스 코드 문자열을 암호화하여 실행 파일에서 문자열을 완전히 숨길 수 있는 기능을 제공합니다. 일반적으로 소스 코드의 모든 문자열을 암호화 할 필요는 없지만 일련번호나 중요 정보를 포함한 문자열의 경우, 또는 문자열 복사를 하지 못하도록 막을 필요가 있을 경우 암호화를 통해 문자열을 보호할 수 있습니다.

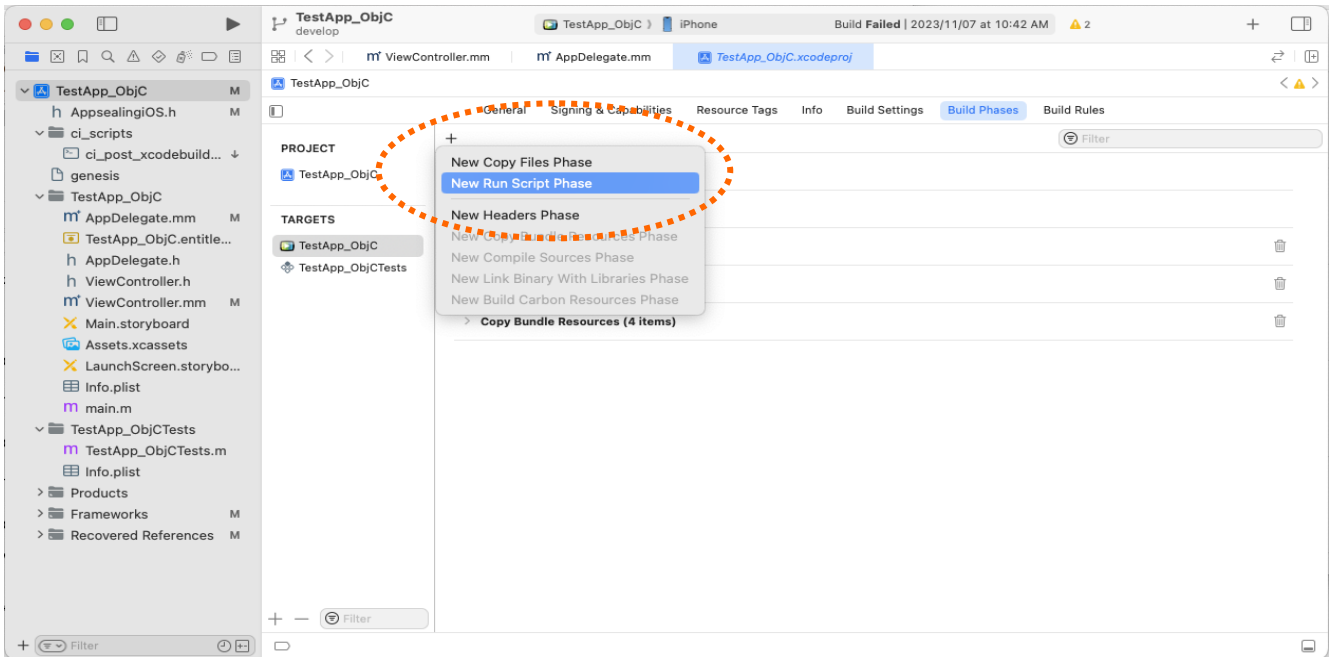
따라서 AppSealing의 문자열 암호화 기능은 소스 코드의 모든 문자열을 암호화 하는 것이 아니라 소스 코드 중에서 보호가 필요한 문자열에 한해 개발자가 직접 지정된 태그를 추가 하는 방식으로 진행됩니다. 이렇게 태그가 추가된 문자열은 원래의 문자열 형태를 그대로 보존하고 있다가 컴파일 직전 암호화된 문자열로 교체되고 컴파일 되며 컴파일이 완료되면 다시 태그가 추가된 원래 문자열 형태로 복구됩니다. 다만 이를 위해 Xcode 빌드 과정에 추가적인 설정이 필요합니다. 아래 설명된 단계를 따라 빌드 스크립트를 설정해 줍니다.

이 장에서는 샘플 프로젝트로 Objective-C 기반의 TestApp_ObjC 라는 이름의 프로젝트를 사용합니다.

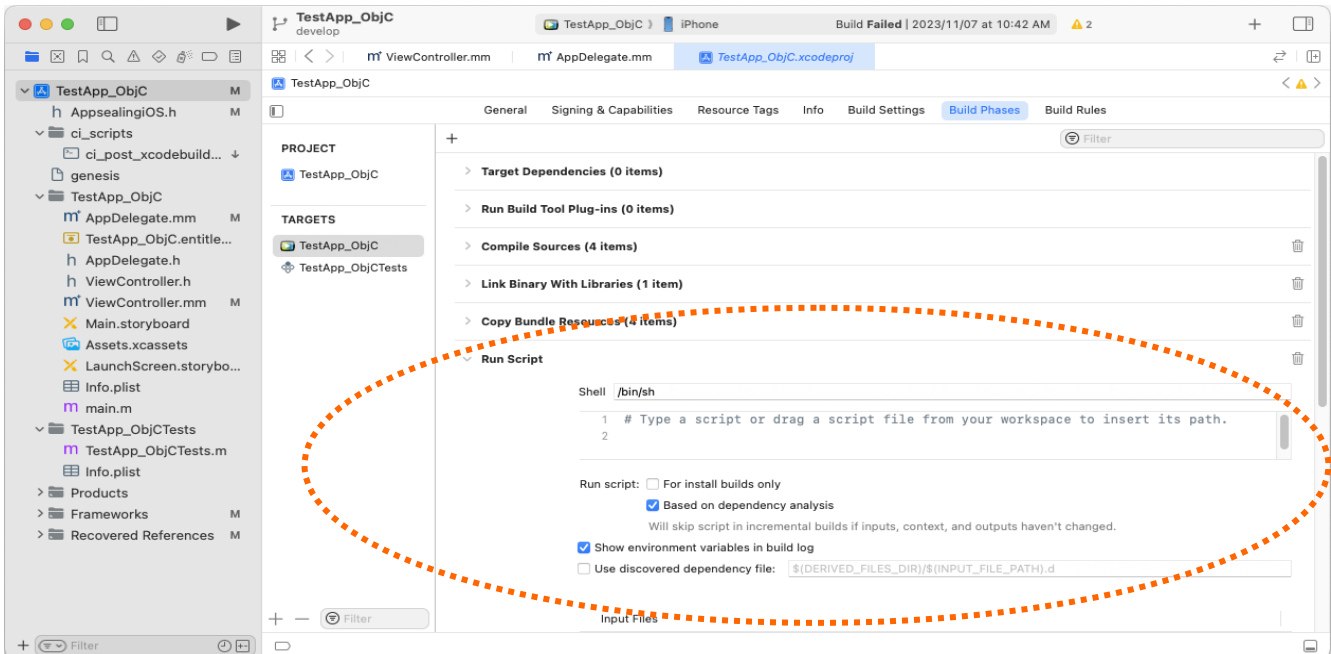
먼저 프로젝트 내비게이터에서 프로젝트를 선택하고 TARGETS에서 메인 타겟 "TestApp_ObjC"를 선택합니다. 우측에 표시되는 탭 중에서 "Build Phase" 탭을 선택합니다.



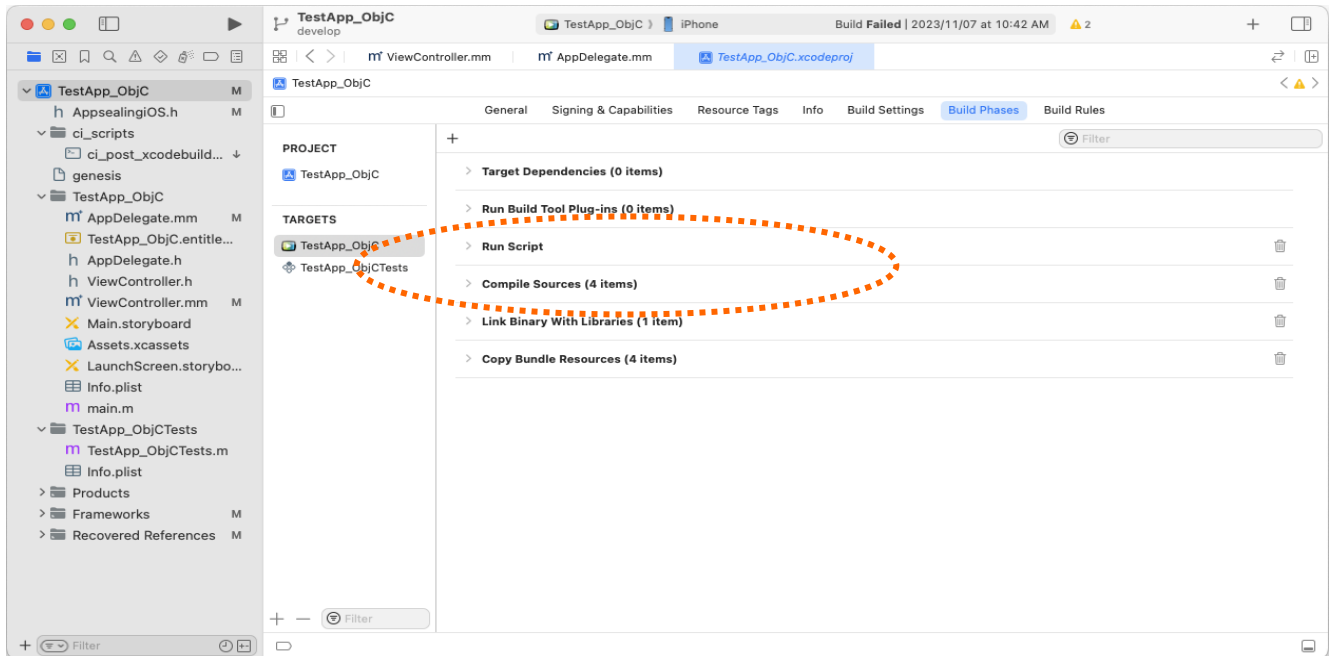
“Build Phase” 탭 화면이 표시되면 좌측 상단의 “+” 아이콘을 클릭하여 메뉴를 표시하고 여기서 “New Run Script Phase”를 선택합니다.



아래 화면과 같이 “Run Script” 단계가 추가되고 그 안에 스크립트 명령을 추가할 수 있는 박스가 표시됩니다.

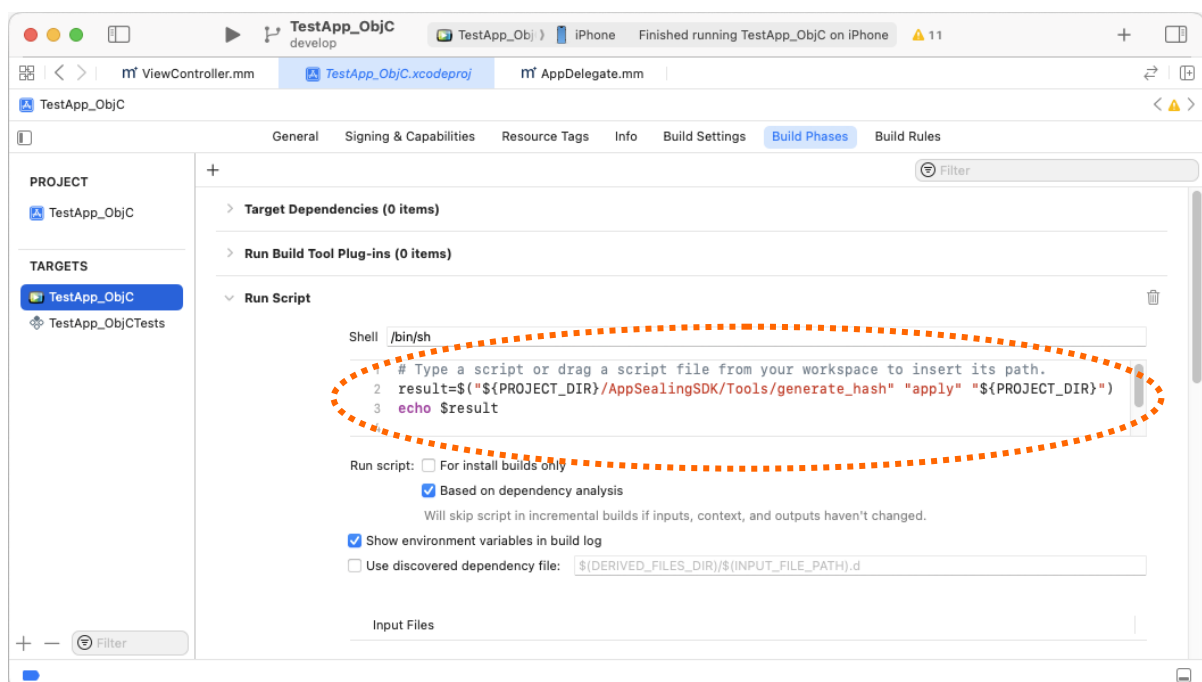


"Run Script" 좌측의 뒤집힌 캐럿(∨) 아이콘을 클릭해 펼쳐진 스크립트 내용을 접은 후 마우스 왼쪽 버튼으로 "Run Script" 항목을 잡아 끌어 "Compile Sources (## items)" 항목 위로 이동시킵니다. 아래 화면과 같이 되어야 합니다.

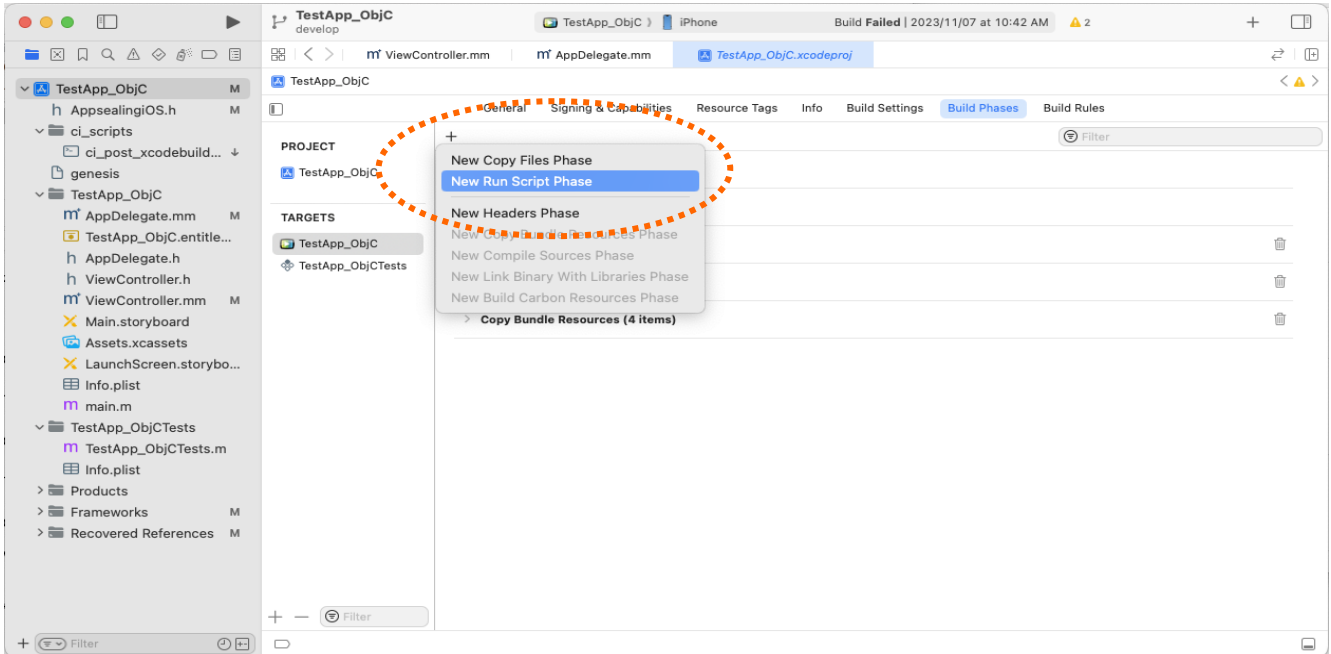


다시 "Run Script" 좌측의 ">" 아이콘을 클릭해 스크립트 항목을 펼친 후에 스크립트 박스에 아래 내용을 복사하여 붙여넣기로 입력합니다.

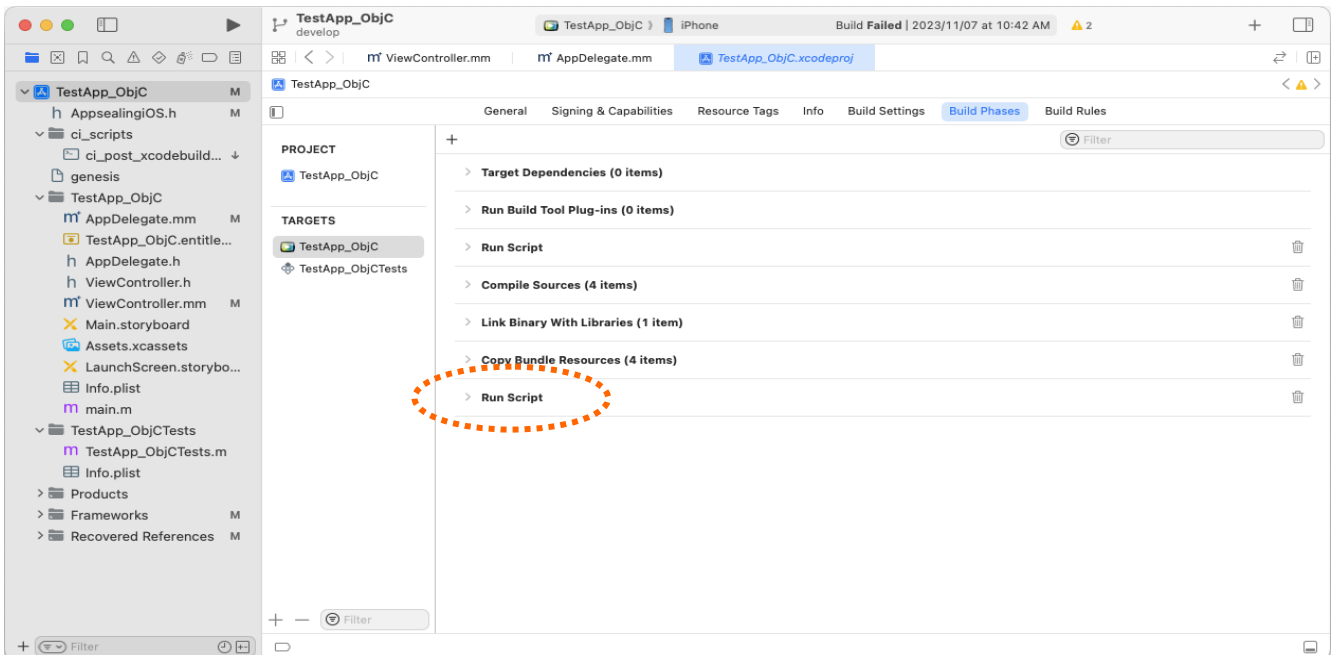
```
result=$(("${PROJECT_DIR}/AppSealingSDK/Tools/generate_hash" "apply" "${PROJECT_DIR}")
echo $result
```



다음은 좌측 상단의 "+" 아이콘을 한번 더 클릭하여 메뉴를 표시하고 "New Run Script Phase" 항목을 추가로 선택합니다.

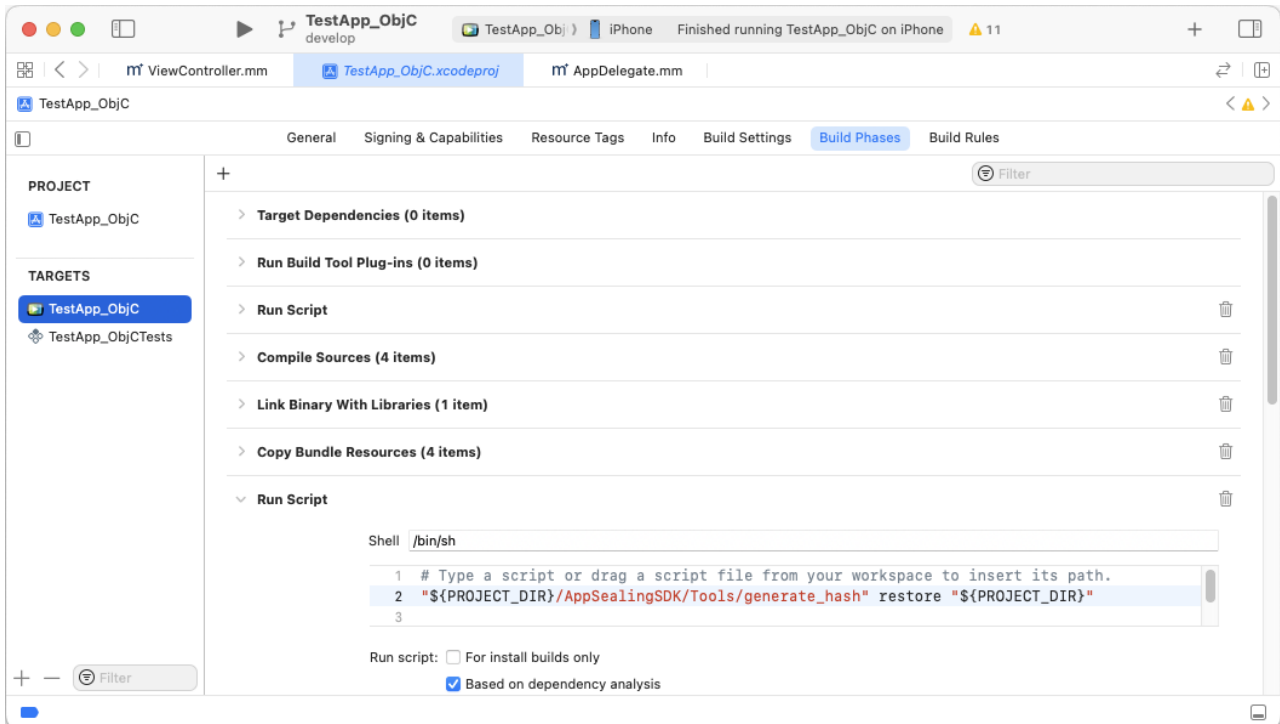


이제 아래 화면과 같이 맨 아래 쪽에 "Run Script" 항목이 하나 더 추가가 되어야 합니다.



"Run Script" 좌측의 ">" 아이콘을 클릭해 스크립트 항목을 펼친 후 스크립트 박스에 아래 내용을 복사하여 붙여넣기로 입력합니다.

```
"${PROJECT_DIR}/AppSealingSDK/Tools/generate_hash " "restore" "${PROJECT_DIR}"
```



6-2 문자열 암호화 태그 추가

이제 문자열 암호화를 위한 Build Phase 설정 작업은 모두 완료됐습니다. 지금부터는 실제 소스 코드에서 암호화를 적용할 문자열에 태그를 추가하는 작업을 진행해야 합니다. 암호화 태그는 "\0x1\0x1\0x1\0x1"가 사용되며 문자열 맨 끝 부분에 추가해야 됩니다.

e.g)

"This is my string" → "This is my string\0x1\0x1\0x1\0x1"

"Special char !@#\$%^&'\\" → "Special char !@#\$%^&'\\" \0x1\0x1\0x1\0x1"

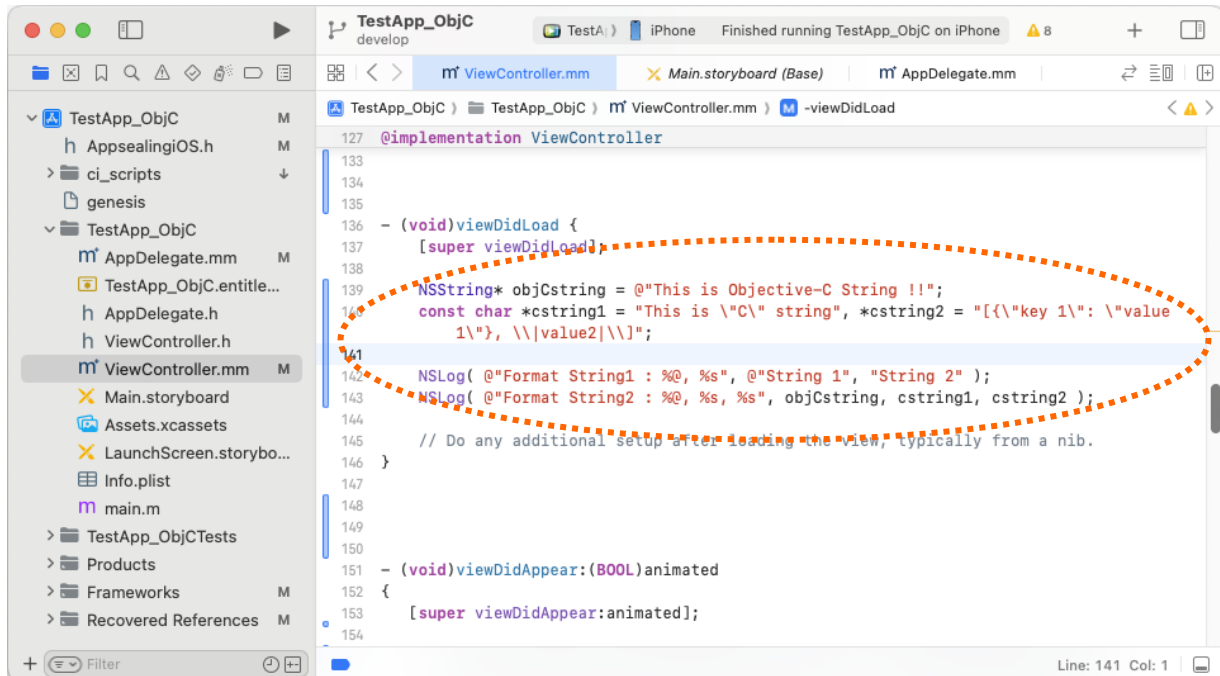
"문자열을 암호화 합니다" → "문자열을 암호화 합니다\0x1\0x1\0x1\0x1"

"文字列を暗号化します" → "文字列を暗号化します\0x1\0x1\0x1\0x1"

"加密字符串" → "加密字符串\0x1\0x1\0x1\0x1"

"Cifrar la cadena" → "Cifrar la cadena\0x1\0x1\0x1\0x1"

실제 소스 코드에서 문자열에 태그를 적용해 보겠습니다. TestApp_ObjC 프로젝트의 경우 Objective-C 언어를 기반으로 하고 있어서 소스 코드 상에 C/C++ 문자열과 Objective-C 문자열을 모두 포함할 수 있습니다.



위 코드의 viewDidLoad 함수에 있는 4개의 Objective-C 문자열과 3개의 C 문자열에 암호화를 적용해 보겠습니다.

[Objective-C 문자열]

@**"This is Objective-C String !"** → @**"This is Objective-C String !\0x1\0x1\0x1\0x1"**

@**"Format String1 : %@, %s"** → @**"Format String1 : %@, %s\0x1\0x1\0x1\0x1"**

@**"String 1"** → @**"String 1\0x1\0x1\0x1\0x1"**

@**"Format String2 : %@, %s, %s"** → @**"Format String2 : %@, %s, %s\0x1\0x1\0x1\0x1"**

[C 문자열]

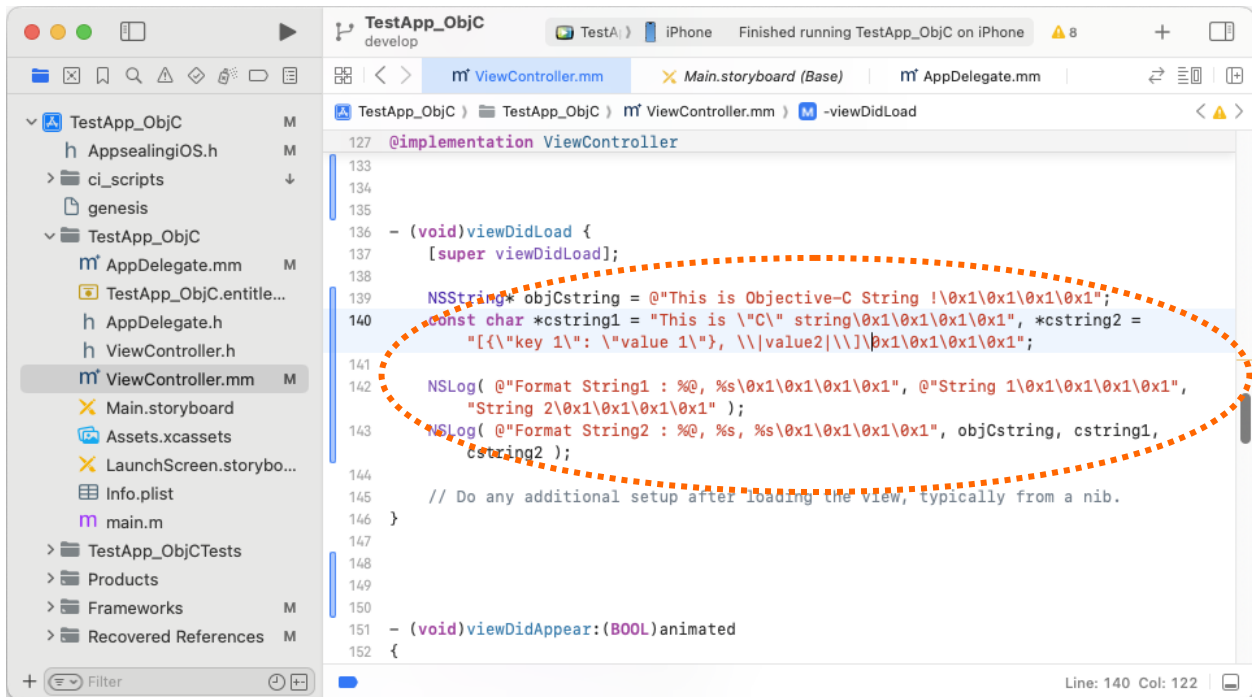
"This is \"C\" string" → **"This is \"C\" string\0x1\0x1\0x1\0x1"**

"[{\"key 1\": \"value 1\"}, \\\|value2|\\]" →

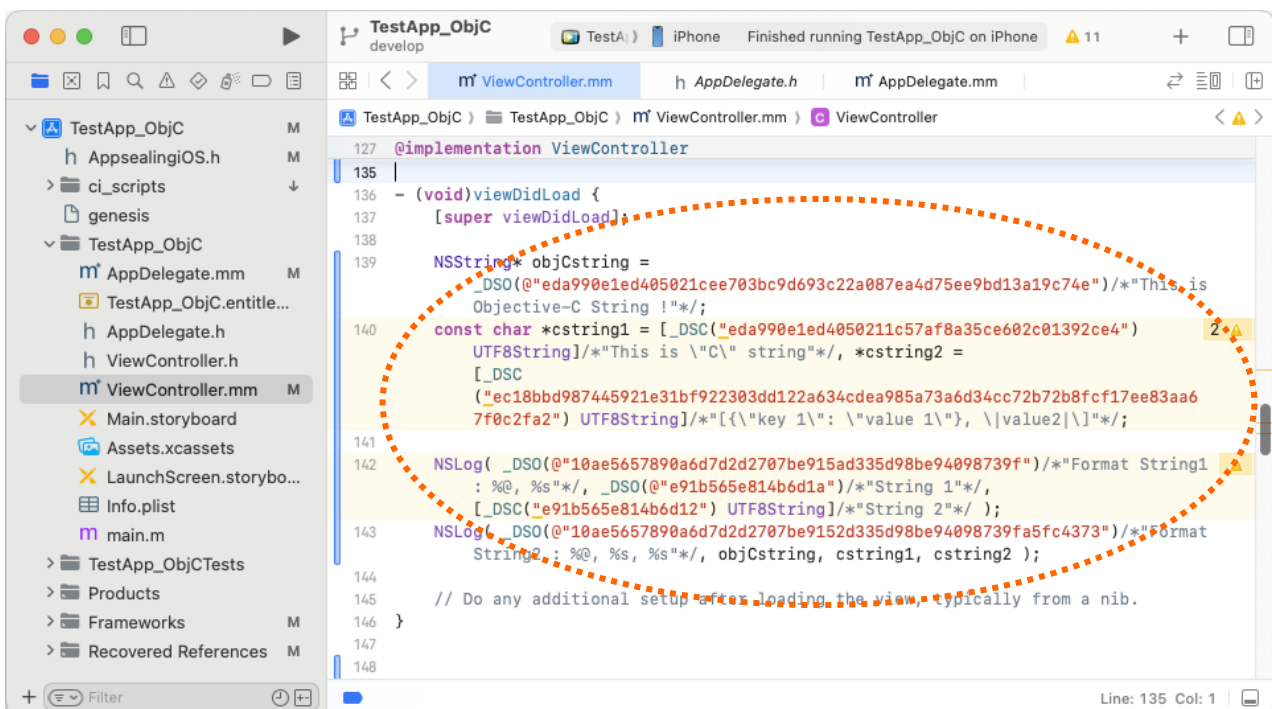
"[{\"key 1\": \"value 1\"}, \\\|value2|\\]\0x1\0x1\0x1\0x1"

"String 2" → **"String 2\0x1\0x1\0x1\0x1"**

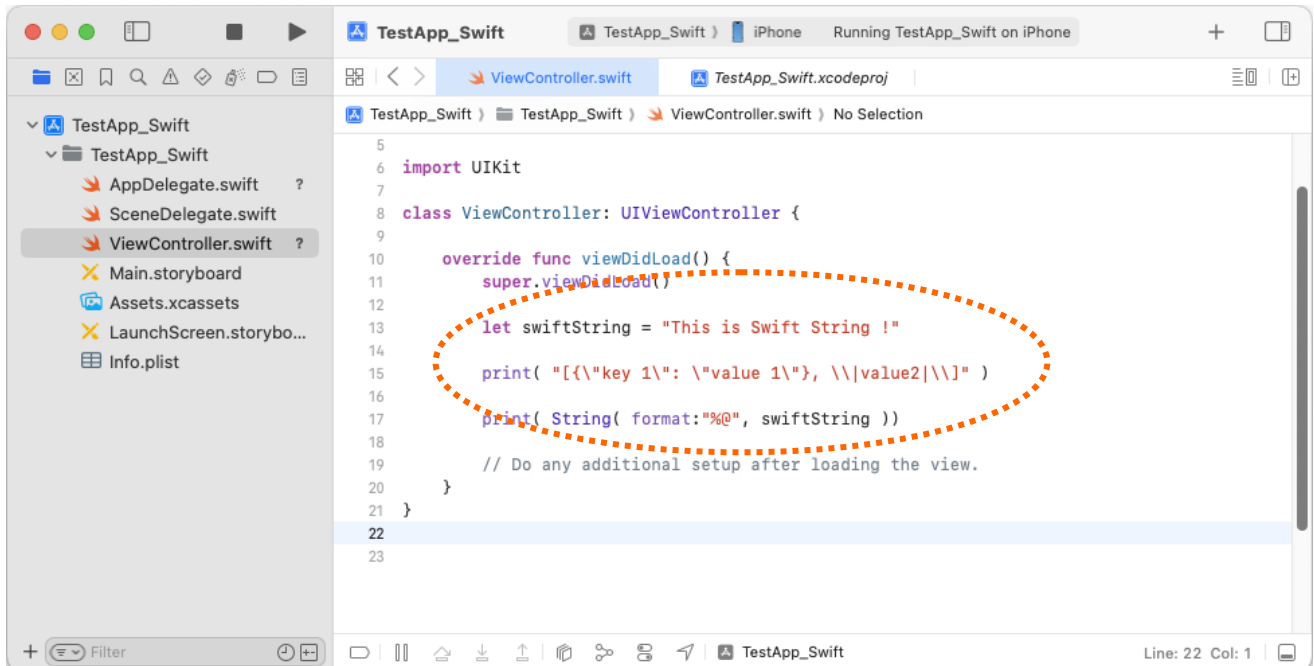
태그를 모두 적용한 코드의 모습은 아래와 같습니다.



이렇게 태그가 적용된 Objective-C 코드는 컴파일 시점에 아래와 같이 암호화 된 문자열로 변환되어 컴파일 되고 컴파일이 완료되면 다시 위 그림과 같은 원래의 태그 달린 문자열로 복원됩니다. (따라서 아래와 같은 암호화 된 문자열이 포함된 Objective-C 코드는 일반적인 상황에서는 볼 수 없습니다)



Swift 소스 코드의 경우에도 태그 적용은 동일한 방법으로 이루어집니다.

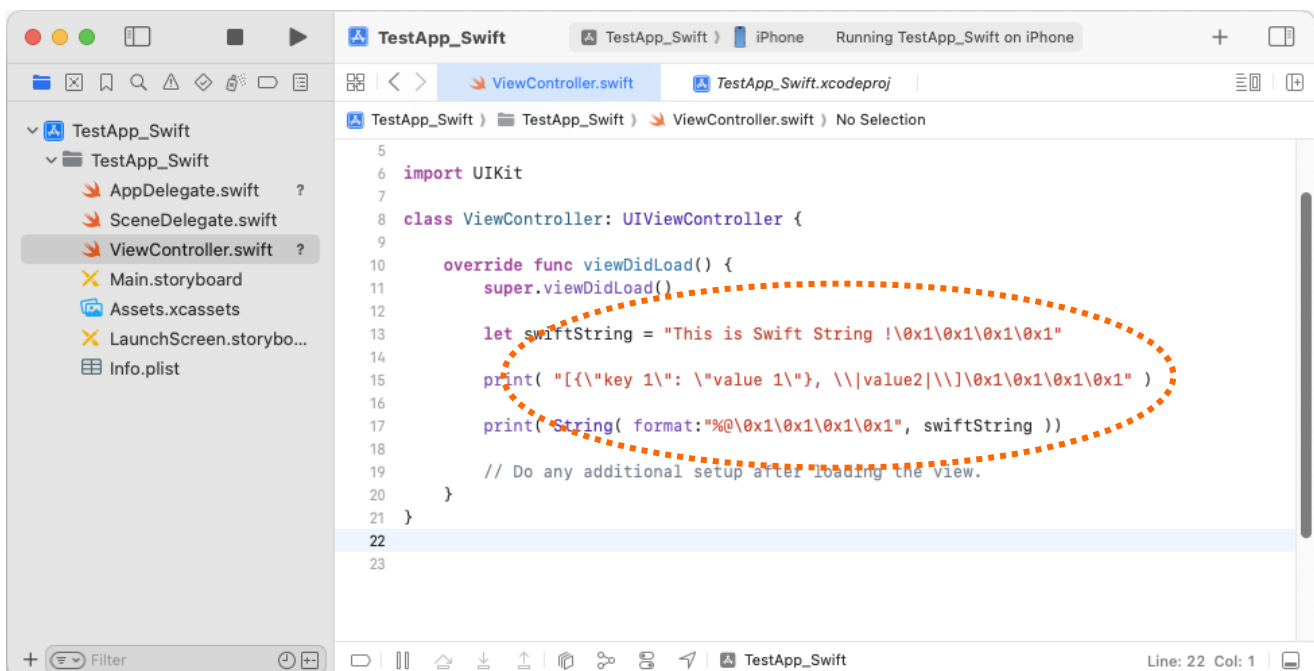


"This is Swift string" → "This is Swift string\0x1\0x1\0x1\0x1"

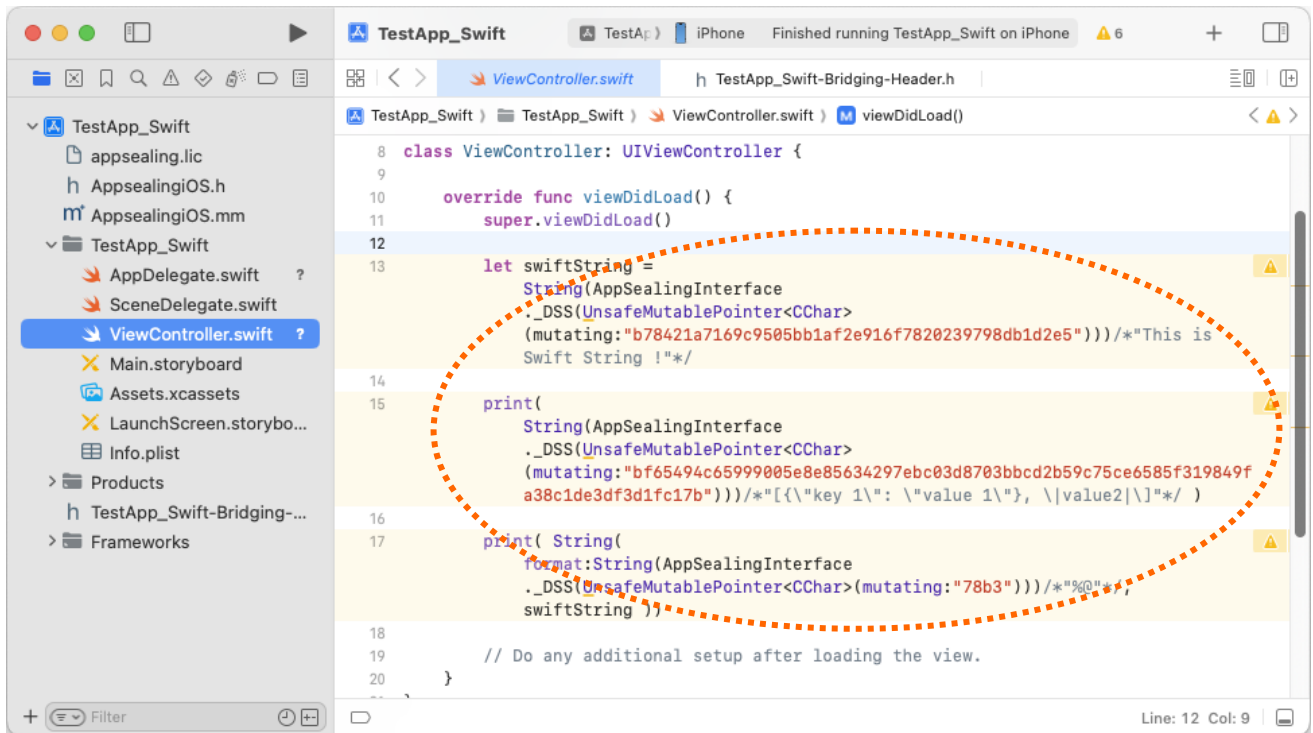
"[\\\"key 1\\": \\\"value 1\\", \\\\"value2\\"]" →

"[\\\"key 1\\": \\\"value 1\\", \\\\"value2\\"]\0x1\0x1\0x1\0x1"

"%@" → "%@\0x1\0x1\0x1\0x1"

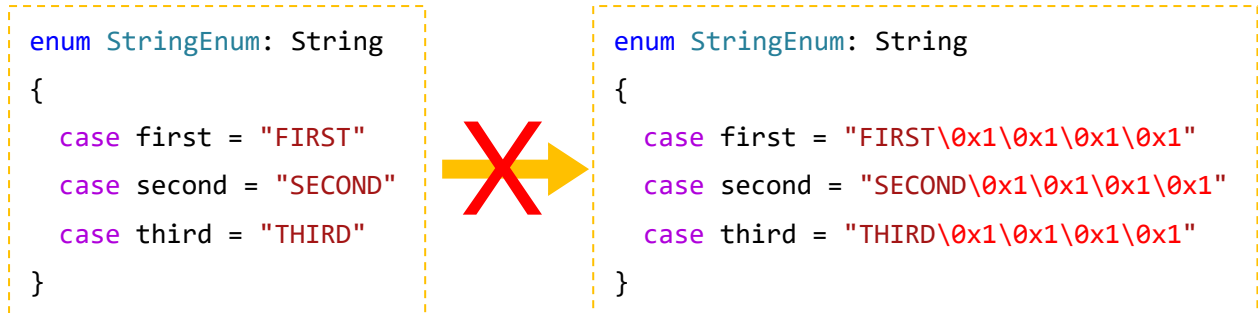


태그가 적용된 Swift 코드는 컴파일 시점에 아래와 같이 암호화 된 문자열로 변환되어 컴파일 되고 컴파일이 완료되면 다시 위 그림과 같은 원래의 태그 달린 문자열로 복원됩니다. (마찬가지로 아래와 같은 암호화 된 문자열이 포함된 Swift 코드도 일반적인 상황에서는 볼 수 없습니다)

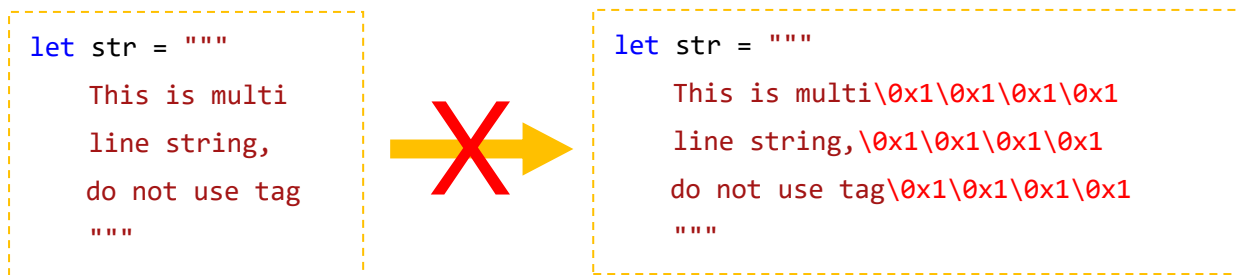


6-3 Swift 문자열 암호화 주의 사항

- 다음과 같은 문자열 열거형을 사용할 경우 원시값으로 지정된 문자열에는 태그를 추가할 수 없습니다. 태그를 추가할 경우 컴파일 오류가 발생하게 됩니다.



- 다음과 같은 멀티라인 문자열에는 태그를 추가할 수 없습니다.



- 다음과 같이 문자열 보간법이 적용된 문자열에는 태그를 추가할 수 없습니다.



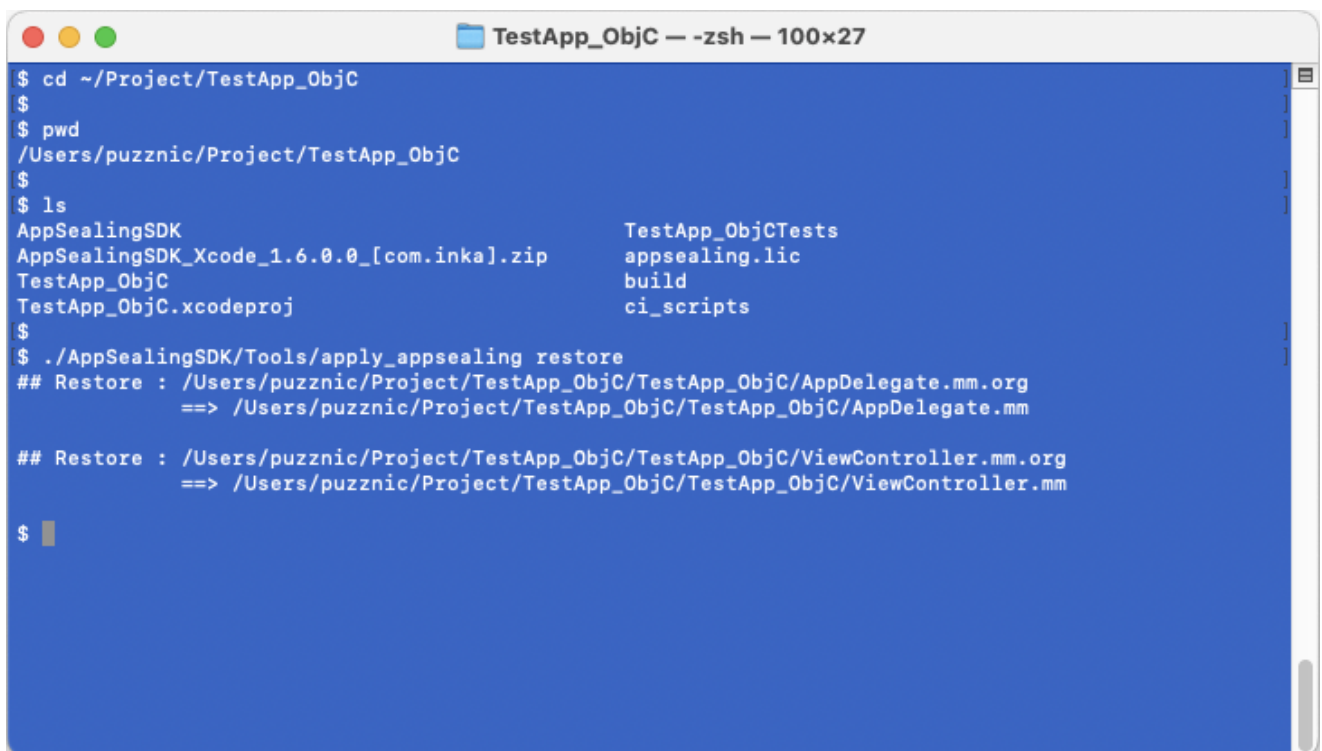
6-4 컴파일 오류 대응 : 암호화 된 문자열 복원

태그가 추가된 문자열은 컴파일 직전 암호화 된 문자열로 변경되고 컴파일 이후 원래의 문자열로 복원됩니다(8-1 Run Script). 그러나 컴파일 오류가 발생하거나 어떤 원인으로 컴파일 과정이 중단 되었을 경우 컴파일 이후의 스크립트가 실행되지 않고 이는 암호화 된 문자열이 복원되지 않은 채 소스 코드 상에 그대로 남아 있는 문제로 이어지게 됩니다.

이 경우 암호화 된 문자열을 원래대로 복원하는 방법은 두가지가 있습니다.

첫번째는 스크립트 실행을 통한 복원입니다. 터미널 앱을 실행하고 프로젝트 디렉토리로 이동합니다. 그리고 나서 AppSealingSDK/Tools/generate_hash 스크립트를 "restore" 파라미터와 함께 실행하면 원본 코드가 복원됩니다.

```
$ ./AppSealingSDK/Tools/generate_hash restore
```

A screenshot of a macOS terminal window titled "TestApp_ObjC — zsh — 100x27". The terminal shows the following commands and output:

```
$ cd ~/Project/TestApp_ObjC
$
$ pwd
/Users/puzznic/Project/TestApp_ObjC
$
$ ls
AppSealingSDK                               TestApp_ObjCTests
AppSealingSDK_Xcode_1.6.0.0_[com.inka].zip   appsealing.lic
TestApp_ObjC                                 build
TestApp_ObjC.xcodeproj                       ci_scripts
$
$ ./AppSealingSDK/Tools/apply_appsealing restore
## Restore : /Users/puzznic/Project/TestApp_ObjC/TestApp_ObjC/AppDelegate.mm.org
==> /Users/puzznic/Project/TestApp_ObjC/TestApp_ObjC/AppDelegate.mm

## Restore : /Users/puzznic/Project/TestApp_ObjC/TestApp_ObjC/ViewController.mm.org
==> /Users/puzznic/Project/TestApp_ObjC/TestApp_ObjC/ViewController.mm

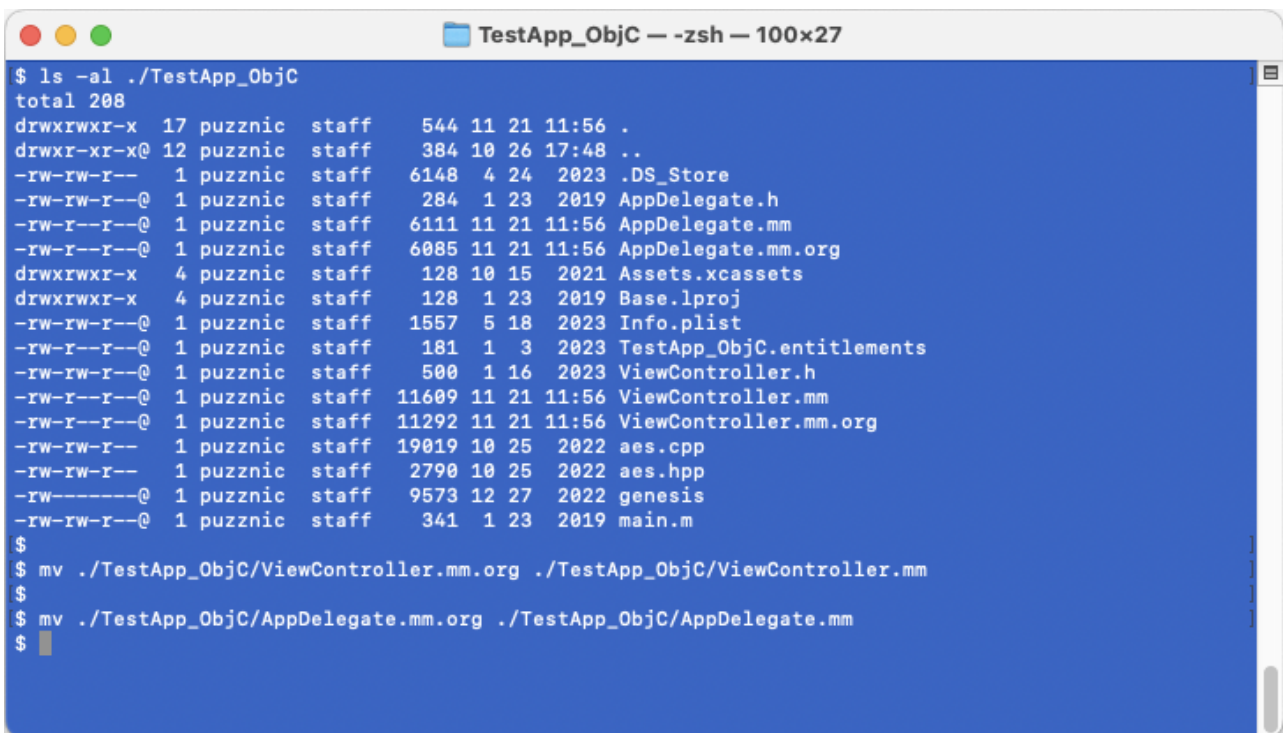
$
```

스크립트를 실행하면 예제로 사용된 TestApp_ObjC 프로젝트의 AppDelegate.mm 및 ViewController.mm 두 개의 소스코드가 복원되었다는 메시지가 출력되는 것을 볼 수 있습니다.

두번째 방법은 터미널에서 셸 명령을 직접 입력해 백업되어 있는 코드 파일을 원래 이름으로 되돌리는 것입니다. TestApp_ObjC 프로젝트를 예로 들면 TestApp_ObjC 폴더 아래 있는 TestApp_ObjC 서브 폴더 내용을 출력해 보면 아래 화면과 같이 두 개의 소스 코드가 .org 라는 확장자를 이용해 백업되어 있는 것을 확인할 수 있습니다.

이제 아래의 커맨드를 입력해서 두 개의 파일을 원래 이름으로 되돌립니다.

```
$ mv ./TestApp_ObjC/ViewController.mm.org ./TestApp_ObjC/ViewController.mm
$ mv ./TestApp_ObjC/AppDelegate.mm.org ./TestApp_ObjC/AppDelegate.mm
```



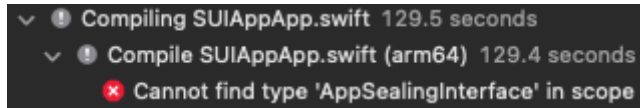
The image shows a macOS terminal window titled "TestApp_ObjC — zsh — 100x27". The terminal displays the output of the command `ls -al ./TestApp_ObjC`, which lists the contents of the directory. The files listed include `ViewController.mm.org` and `AppDelegate.mm.org`. Below the listing, the commands `mv ./TestApp_ObjC/ViewController.mm.org ./TestApp_ObjC/ViewController.mm` and `mv ./TestApp_ObjC/AppDelegate.mm.org ./TestApp_ObjC/AppDelegate.mm` are entered to restore the original files.

```
$ ls -al ./TestApp_ObjC
total 208
drwxrwxr-x  17 puzznic  staff   544 11 21 11:56 .
drwxr-xr-x@  12 puzznic  staff   384 10 26 17:48 ..
-rw-rw-r--   1 puzznic  staff  6148  4 24  2023 .DS_Store
-rw-rw-r--@   1 puzznic  staff   284  1 23  2019 AppDelegate.h
-rw-r--r--@   1 puzznic  staff  6111 11 21 11:56 AppDelegate.mm
-rw-r--r--@   1 puzznic  staff  6085 11 21 11:56 AppDelegate.mm.org
drwxrwxr-x   4 puzznic  staff   128 10 15  2021 Assets.xcassets
drwxrwxr-x   4 puzznic  staff   128  1 23  2019 Base.lproj
-rw-rw-r--@   1 puzznic  staff  1557  5 18  2023 Info.plist
-rw-r--r--@   1 puzznic  staff   181  1  3  2023 TestApp_ObjC.entitlements
-rw-rw-r--@   1 puzznic  staff   500  1 16  2023 ViewController.h
-rw-r--r--@   1 puzznic  staff 11609 11 21 11:56 ViewController.mm
-rw-r--r--@   1 puzznic  staff 11292 11 21 11:56 ViewController.mm.org
-rw-rw-r--   1 puzznic  staff 19019 10 25  2022 aes.cpp
-rw-rw-r--   1 puzznic  staff  2790 10 25  2022 aes.hpp
-rw-----@   1 puzznic  staff  9573 12 27  2022 genesis
-rw-rw-r--@   1 puzznic  staff   341  1 23  2019 main.m

$
$ mv ./TestApp_ObjC/ViewController.mm.org ./TestApp_ObjC/ViewController.mm
$
$ mv ./TestApp_ObjC/AppDelegate.mm.org ./TestApp_ObjC/AppDelegate.mm
$
```


Part 7. 문제 해결

10-1 Xcode Build error (1) **Use of undeclared type 'AppSealingInterface'**

A screenshot of Xcode's build log showing three lines of output. The first line is 'Compiling SUIAppApp.swift 129.5 seconds' with a green checkmark icon. The second line is 'Compile SUIAppApp.swift (arm64) 129.4 seconds' with a green checkmark icon. The third line is 'Cannot find type 'AppSealingInterface' in scope' with a red 'x' icon, indicating an error.

```
✓ [i] Compiling SUIAppApp.swift 129.5 seconds
✓ [i] Compile SUIAppApp.swift (arm64) 129.4 seconds
✗ [x] Cannot find type 'AppSealingInterface' in scope
```

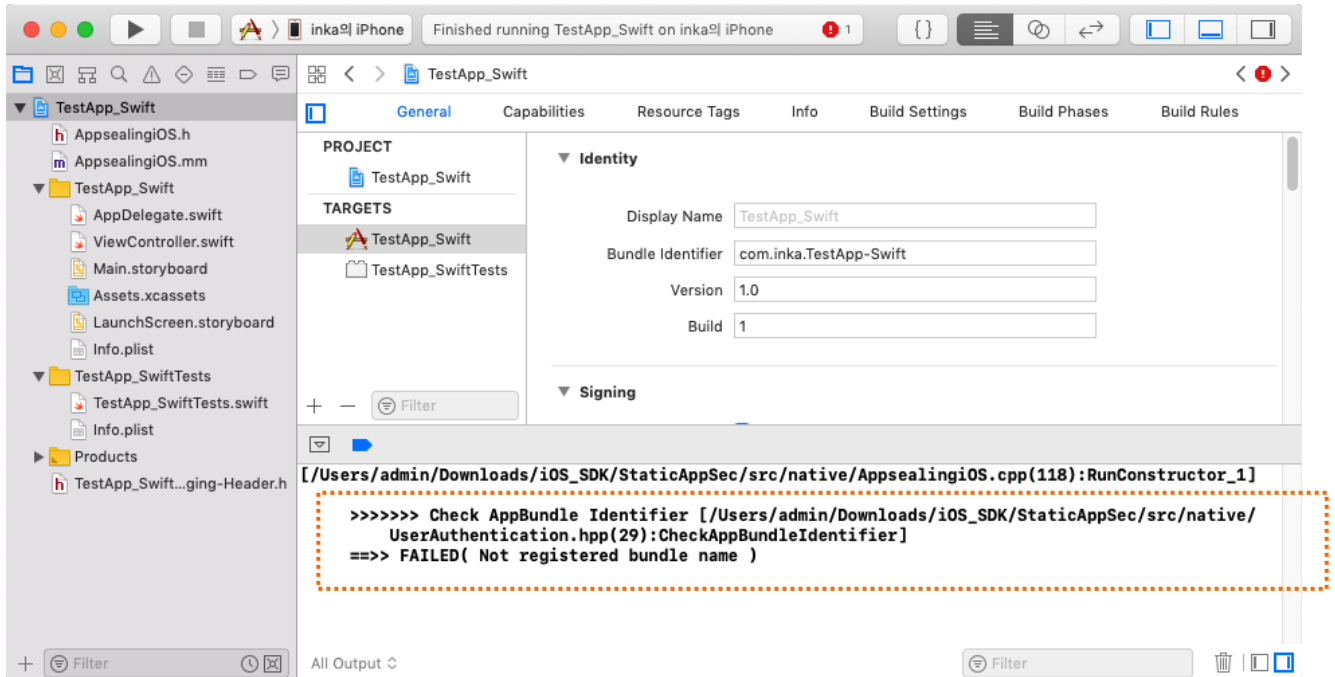
AppSealing 프레임워크 가져오기가 제대로 되어 있지 않을 때 발생하는 오류입니다.

- Swift: **import** AppSealingFramework
- Objective-C: **#import** <AppSealingFramework/AppSealingFramework.h>

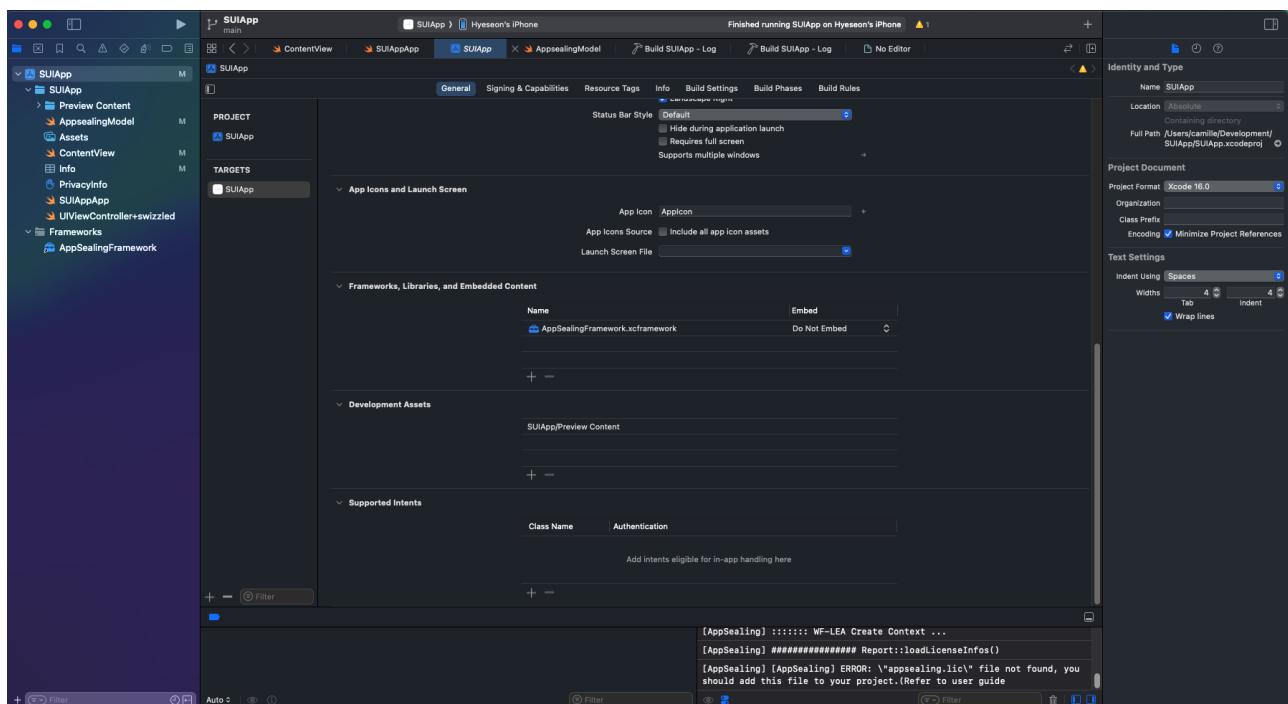
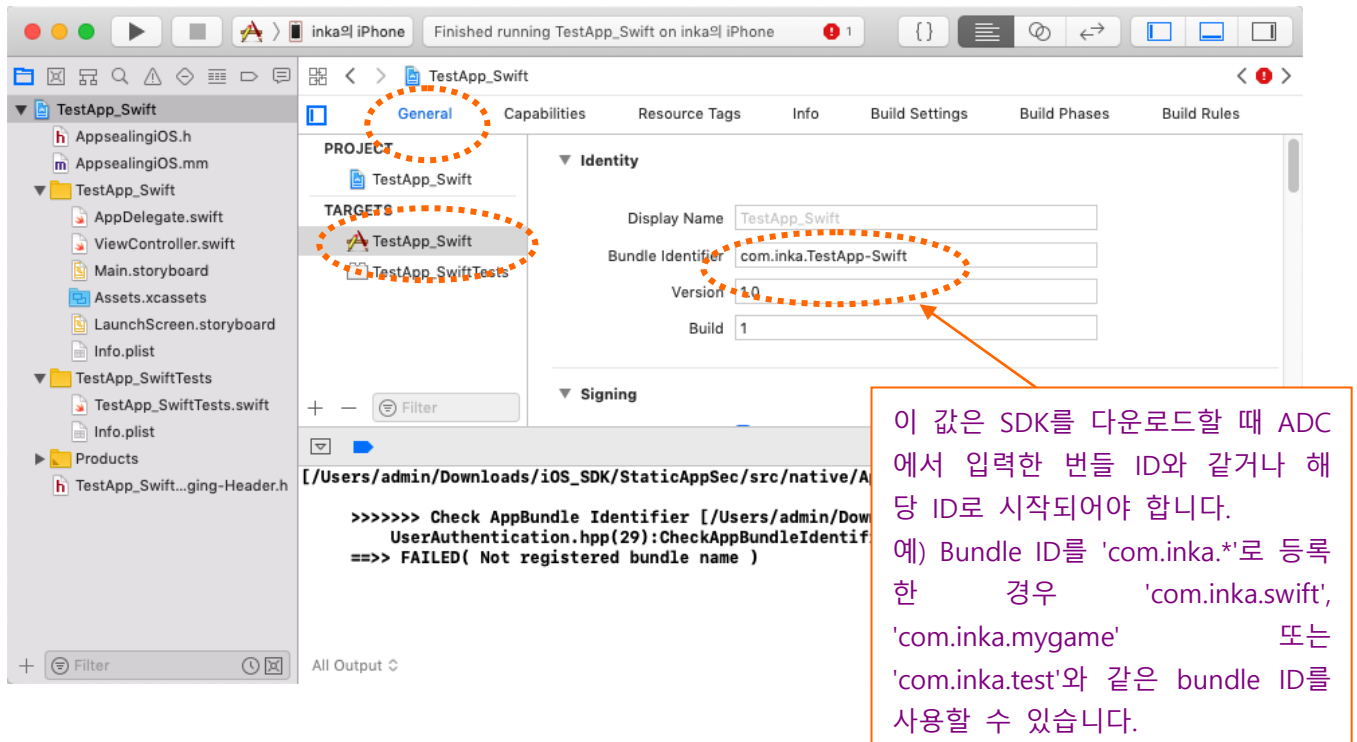
10-2 Execution error (I) 앱이 실행되고 곧바로 종료됨

기기에서 앱이 실행된 직후 바로 종료될 경우, 로그 메시지를 조회하여 원인을 알 수 있습니다.

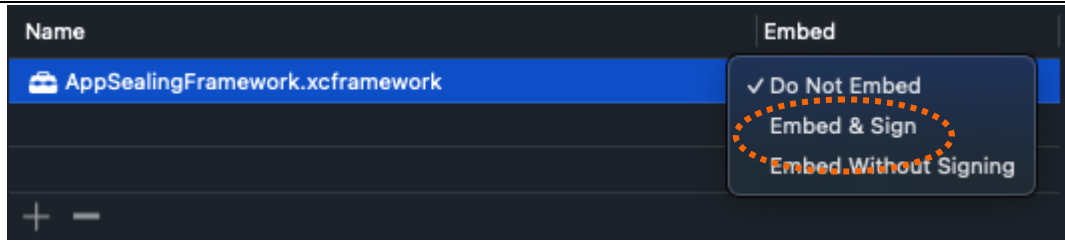
- **Case 1:** bundle ID 불일치



로그 메시지에 "====>>>> FAILED(Not registered bundle name)"와 같은 문자열이 포함되어 있으면 현재 프로젝트의 bundle ID가 AppSealing SDK를 다운로드하는 동안 AppSealing Developer Center (ADC)에 제대로 등록되어 있지 않다는 의미입니다. ADC에서 SDK를 다운로드할 때 입력한 값과 같은 bundle ID를 사용하거나, bundle ID를 바꿀 수 없는 상황이라면 해당 bundle ID에 해당하는 새 프로젝트를 만드십시오.

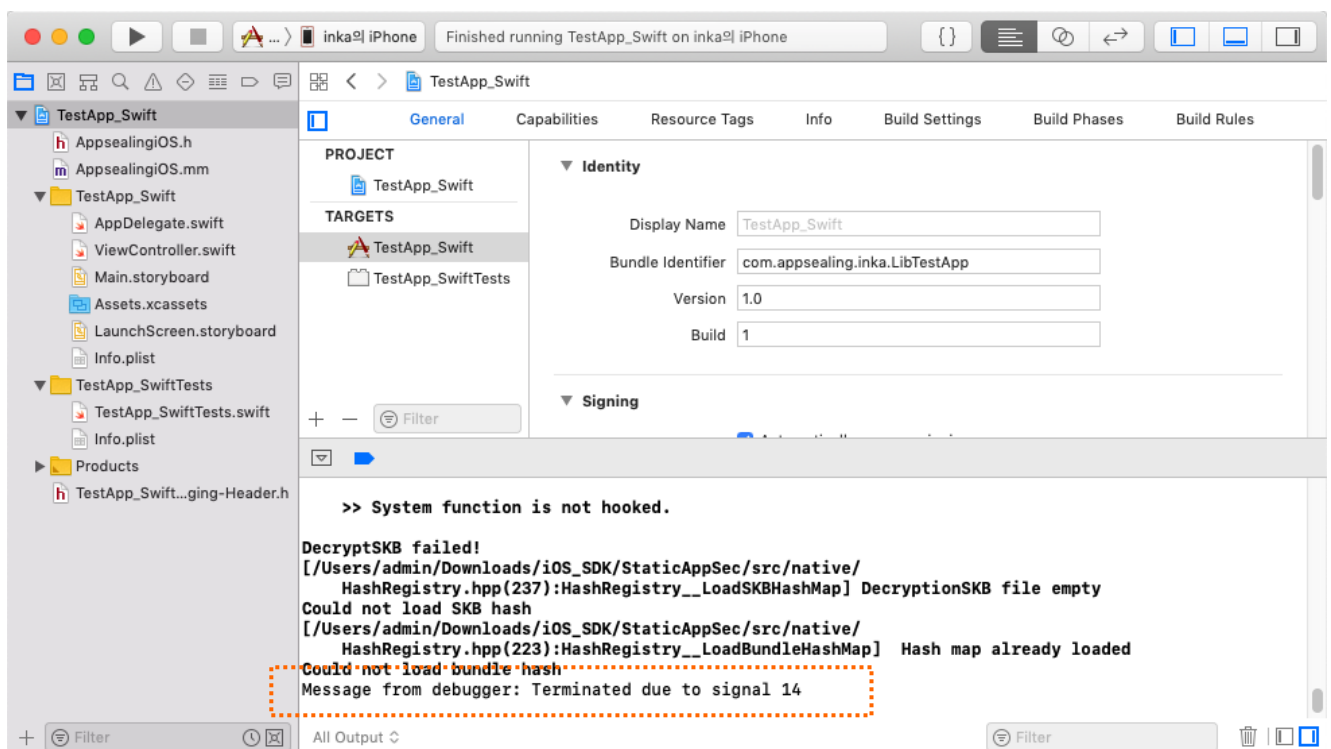


로그 메시지에 " ERROR: ₩"appsealing.lic₩" file not found"와 같은 문자열이 포함되어 있으면 2-3절을 참조하여 프레임워크 설정이 "Embed & Sign"으로 되어 있는지 확인하십시오.



10-3 Execution error (II) 앱이 실행 도중 갑자기 종료됨

실행 후 약 20초 후에 갑자기 앱이 종료된 경우, 실행 로그를 확인하여 앱이 릴리즈 모드로 실행되었는지 확인할 수 있습니다.



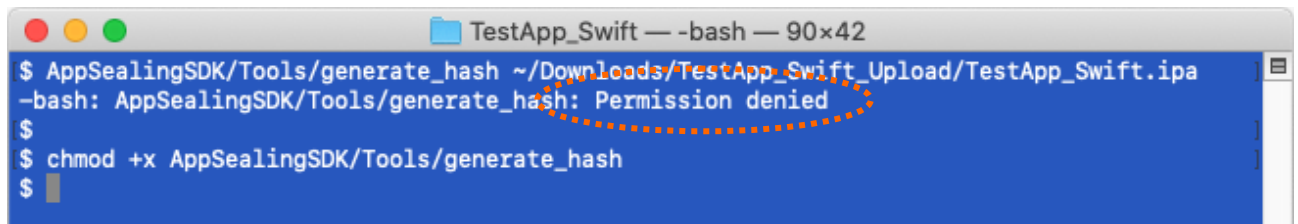
로그 메시지에 "Message from debugger: Terminated due to signal 14" 와 같은 문자열이 포함되어 있는 경우 앱이 release로 빌드, 실행되었고 AppSealing 보안 로직이 활성화되었음을 알 수 있습니다. Release 모드의 AppSealing 보안 로직은 탈옥된 장치, 실행 파일의 복호화 된 상태, 디버거 연결과 같은 보안 침입을 확인하며, 어떤 문제가 있을 경우 사용자의 조치나 기타 상황에 관계없이 20초 후에 앱을 닫습니다.

Release 모드에서는 AppSealing 로직이 활성화되고 앱이 종료되므로 App Store 또는 TestFlight에 배포하기 전까지 디버그 모드로 앱을 테스트하십시오. AppSealing 로직은 release 모드로 빌드 시 Xcode 장치에서 앱을 실행하면 디버깅 환경으로 탐지하

거나 실행 파일이 비정상적으로 해독되어 있음을 감지하고 20초 후에 앱을 종료합니다.

10-4 무결성/인증서 스냅샷 생성 스크립트 실행 시 **Permission denied** 오류

4-4단계의 무결성/인증서 스냅샷 정보 생성 단계에서 아래와 같이 스크립트 실행에 실패하고 "Permission denied" 오류 메시지가 출력되는 경우가 발생할 수 있습니다.

A screenshot of a macOS terminal window titled "TestApp_Swift — -bash — 90x42". The terminal shows the following commands and output:

```
$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa
-bash: AppSealingSDK/Tools/generate_hash: Permission denied
$
$ chmod +x AppSealingSDK/Tools/generate_hash
$
```

An orange dotted circle highlights the error message "-bash: AppSealingSDK/Tools/generate_hash: Permission denied".

이 경우 이하와 같이 스크립트 파일에 실행 권한을 추가하는 명령을 실행하고 다시 시도해 보십시오.

```
$ chmod +x AppSealingSDK/Tools/generate_hash
```